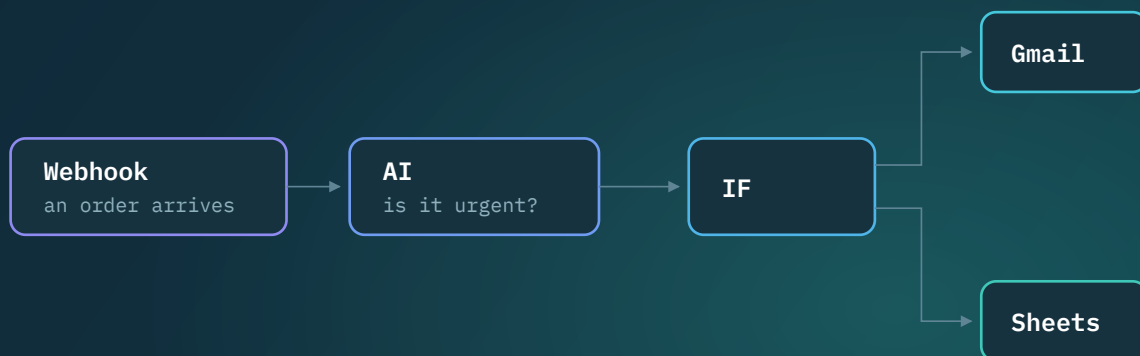


A complete build guide

AI Automation Platform

Building n8n, from an empty folder

Every term explained, every part linked back to something you have already used, and every line of code you need — from a forty-line engine to a live demo.



1 · Make it run

2 · Make it real

3 · Usable

4 · Self-starting

5 · Do things

6 · Ship it

Develop an AI automation platform similar to n8n

Schedule trigger · Webhook · HTTP · Google Docs, Sheets, Drive, Gmail · Code node · AI

Demonstration Week 14 · Report Week 15

8 people

6 phases · 15 stages · 22 diagrams

Contents

PART 1 — THE BIG PICTURE, AND HOW IT WORKS

1.1	Everything we are building, on one page · Figure 1	4
1.2	What a workflow really is · Figure 2	5
1.3	What a node really is · Figure 3	6
1.4	What an item really is · Figure 4	7
1.5	What the engine really is · Figure 5	8
1.6	The canvas and the API — the two halves of the app · Figure 6	9
1.7	The six phases · Figure 7	11
1.8	The toolkit — what we install, and when · Figure 8	12
1.9	Install everything, click by click	13
1.10	VS Code: the program every line of this book is pasted into · Figure 9	14
1.11	The loop you will repeat a hundred times · Figure 10	15
1.12	What the interface should look like · Figure 11	17

PART 2 — BUILDING IT, PHASE BY PHASE

2.1	A workflow runs, from a single file · Figure 12	18
2.1b	Build it: the engine, line by line	19
2.2	Workflows survive a restart, and we can draw them · Figure 13	27
2.2b	Build it: database, server and canvas	29
2.2c	Build it: the canvas, and the two functions that matter · Figure 14	37
2.3	Nodes describe themselves, and fields carry live data · Figures 15–16	45
2.3b	Build it: forms that draw themselves, and expressions · Figure 17	47
2.4	Workflows that run without anyone pressing a button · Figure 18	56
2.4b	Build it: webhooks and schedules · Figure 19	58
2.5	First, permission to use a Google account · Figure 20	66
2.5b	Build it: the Google permission round trip · Figure 21	68
2.6	Then the nodes people actually came for · Figures 22–23	73
2.6b	Build it: one helper, then four Google nodes · Figure 24	75
2.6c	Build it: Gmail, the sandbox, and the AI node · Figure 25	82
2.7	Online, measured, and written up · Figure 26	91
2.7b	Build it: online, and measured against n8n	92
2.8	How all the parts connect to each other · Figure 27	98

PART 3 — PUTTING IT TOGETHER

3.1	How six phases become one working system · Figure 28	99
3.1b	Build it: one repository, one command	100
3.2	Four levels of testing, and what each one catches · Figure 29	106
3.2b	Build it: the four levels, in code	107
3.2c	Build it: the browser test, and running it all automatically · Figure 30	113
3.3	Proving the whole thing works, on the day · Figure 31	117

PART 4 — RUNNING THE PROJECT

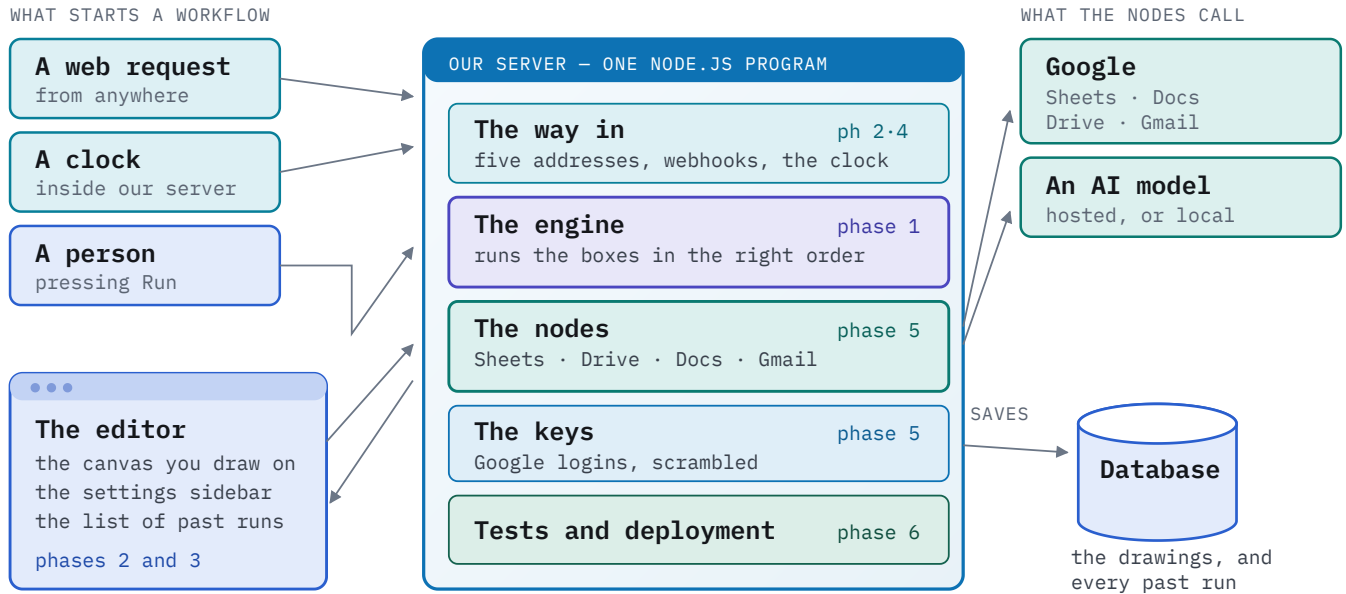
4.1	Eight people, eight areas · Figure 32	118
4.2	The hard part of being eight, and how we solve it · Figure 33	119
4.3	Three risks that end projects, and six habits that save them	120
4.4	What each of us does this week	121

HOW TO READ THIS

Parts 0 and 1 explain how the thing works and are worth reading before anyone writes code. From Part 2 onwards each phase has **two pages**: the first says what we are doing and why, the second gives the actual commands and code. Each phase carries its own colour through every diagram belonging to it, so you always know where you are.

1.1 Everything we are building, on one page

This is the whole project. Nine boxes, and every one of them is built in a phase you can point at. **Nothing else exists.** When you feel lost later, come back to this page and find the box you are working on.



Everything travels the same way: something starts a workflow, the engine walks the drawing box by box, each box does one job — often calling Google or an AI model — and the run is written down.

Figure 1. The whole system. Each box is coloured by the phase that builds it, and every later diagram in this book is a close-up of one of them.

EVERY PHASE, FIRST PAGE

What and why

What we are doing, what it was in n8n, **who builds it**, what you need to know to build it, the tools, how to proceed, and a “done when”.

EVERY PHASE, SECOND PAGE

How, exactly

The commands to type, the files to create, the code to write, and — where a website is involved — **every click, in order**.

THROUGHOUT

Words, where you meet them

Every term is defined the first time it appears, in a box that says what it is, what it does *there*, and how it connects to the rest.

- Phase 1 · the engine
 Phase 2 · database, server, canvas
 Phase 3 · sidebar, expressions
 Phase 4 · webhooks, clock
- Phase 5 · nodes, keys
 Phase 6 · tests, deployment

1.2 What a workflow really is

A workflow feels like a picture. It is actually a **piece of text** — a list of boxes and a list of lines, written down in JSON. The picture is only how we look at it.

FAMILIAR FROM N8N

When you clicked “Download” on a workflow, n8n handed you a `.json` file. That file *was* the workflow — you could email it to a friend, they could import it, and the same picture appeared on their screen. Nothing else was needed. That is the single most important thing to understand before we start.

workflow

What it is. One complete automation — the thing you switch on and leave running.

What it does in this section. It is the JSON on the right of Figure 2: a list of boxes and a list of lines, nothing more.

How it connects. The database stores one row per workflow; the engine is handed exactly this object and nothing else.

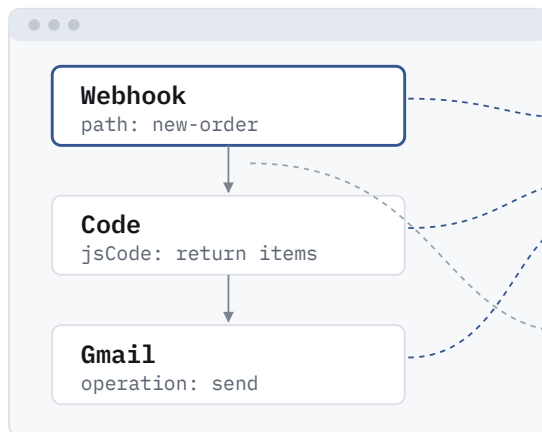
JSON

What it is. A plain-text way of writing data down. Curly braces for a thing, square brackets for a list.

What it does in this section. It is the format the drawing is saved in, and the format every node passes its data along in.

How it connects. The canvas produces it, the database keeps it, the engine reads it — it is the common language of the whole project.

WHAT WE SEE — THE CANVAS



WHAT IS ACTUALLY STORED — ONE PIECE OF TEXT

```

{
  "nodes": [
    { "name": "Webhook", "type": "webhook",
      "parameters": { "path": "new-order" } },
    { "name": "Code", "type": "code", ... },
    { "name": "Gmail", "type": "gmail", ... }
  ],
  "connections": {
    "Webhook": [ [ "Code" ] ],
    "Code": [ [ "Gmail" ] ]
  }
}
  
```

THE BOXES

THE LINES

Figure 2. Every box you drag becomes one entry in `nodes`; every line you draw becomes one entry in `connections`. That is the whole file format — and it is all the engine ever receives.

WHY THIS IS GOOD NEWS

Because a workflow is just text, the two halves of our project can be built by different people at the same time. The canvas only has to *produce* this text; the engine only has to *read* it. Neither needs to know how the other works — they only have to agree on the shape above.

WHAT IT MEANS FOR US

Our database will store this text in a single column. Saving a workflow is one write; loading it is one read; making a copy is copying a row. Big commercial platforms do exactly the same thing, for exactly this reason.

1.3 What a node really is

A node is **one file that does two separate jobs**. Half of it *describes* itself, so the screen knows what settings to draw. The other half is an ordinary function that *does the work* when the engine calls it. Nothing else. Once this is clear, adding a node to our platform stops being mysterious.

node

What it is. One step of a workflow: a small object with a description and one function.

What it does in this section. The description tells the sidebar what settings to draw; the function does the actual job when the engine calls it.

How it connects. Everything in Phase 5 is a node. Because the shape never varies, the engine can run a node written five minutes ago.

parameters

What it is. The settings a node was given — the URL typed into a box, the operation chosen from a dropdown.

What it does in this section. They live on the node inside the workflow JSON, and the node reads them with `getNodeParameter`.

How it connects. Phase 3 makes the sidebar that fills them in; Phase 1 froze the way a node asks for them.



Figure 3. Two halves, two audiences. The screen never runs a node; the engine never draws anything. Neither half needs to know the other exists.

FAMILIAR FROM N8N

Click a Google Sheets node in n8n and the panel on the right appears instantly, with exactly the right fields. Change **Operation** from “Append” to “Read” and the fields change with it. n8n did not hand-build hundreds of panels — every node describes its own fields, and one piece of code turns any such description into a form. We are copying that idea deliberately, and it is the reason four of us can add nodes at the same time.

THE DESCRIPTION HALF

Data, not code

A plain list: “URL is a text box”, “Method is a dropdown with four choices”, “only show Body when Method is POST”. The screen reads that list and draws the panel. Adding a field to a node means adding a line to this list — **never touching the editor**.

THE EXECUTE HALF

One job, done plainly

Items come in, the node does its single job — call an API, reshape some fields, split the flow in two — and items go out. A node never talks to the database, never draws anything, and never decides what runs next. That is the engine's business.

THE RULE THAT KEEPS US OUT OF TROUBLE

Every node in our platform obeys the same shape, whether it sends email through Gmail or simply adds a field. Because the shape never varies, the engine can run a node it has never seen before, and the screen can draw a settings panel for a node written five minutes ago. We agree this shape in Phase 1 and then **freeze it** — changing it later would break everyone's work at once.

1.4 What an item really is

An item is **one thing** — one spreadsheet row, one email, one order. Nodes never pass a single item; they always pass a **list** of them, even when the list is one item long. This sounds like a small detail. It is the decision that shapes every line of code we write.

item

What it is. One single thing travelling between nodes: one row, one email, one order.

What it does in this section. Nodes always pass a *list* of items, even when the list holds one thing.

How it connects. This is why fifty spreadsheet rows become fifty emails with no extra code anywhere.

branch

What it is. One exit from a node. Most nodes have one; a node that splits the flow has two.

What it does in this section. A node returns one list per branch — the outer list is the exits, the inner list is the items.

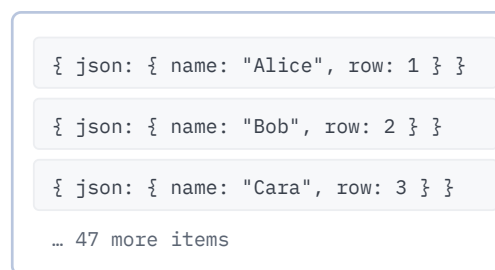
How it connects. An empty branch stops everything after it, which is how the IF node makes half a workflow go quiet.

A REAL SPREADSHEET

name	row
Alice	1
Bob	2
Cara	3
... 47 more	

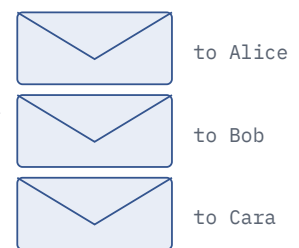
the Sheets node reads it

BECOMES ONE LIST OF 50 ITEMS



one row becomes one item

AND 50 EMAILS GO OUT



... 47 more
one item, one email

The Gmail node is called **ONCE**. It receives all fifty items and loops over them itself — we never write “repeat this node fifty times” anywhere, because the list already says so.

Figure 4. One read, fifty items, one send node, fifty emails — and no special code anywhere to make that happen.

FAMILIAR FROM N8N

Open any node's **OUTPUT** panel in n8n and the heading says something like “5 items”, with little numbered tabs you can click through. That is exactly this list. And it explains something you may have noticed: put a Gmail node after a node that produced five items, and n8n sends five emails without you configuring anything. The list is why.

INSIDE ONE ITEM

A labelled bag of values

Each item holds a `json` part — the ordinary fields, like a name and an email address — and optionally a `binary` part for files riding along, such as a PDF that a later node will attach to an email.

THE LIST OF LISTS

One list per exit

Most nodes have one exit, so they return one list. A node that splits the flow, like IF, has two exits and returns two lists — everything that passed the test, and everything that did not.

THE BUG EVERY ONE OF US WILL HIT AT LEAST ONCE

Returning one list where two were expected, or the other way round. Remember it as: **the outer list is the exits, the inner list is the items**. A normal node returns “one exit, containing all my items”. When a node mysteriously sends nothing onward, or sends everything down the wrong branch, this is almost always why — and printing the data between nodes finds it in about a minute.

1.5 What the engine really is

An **engine** is a program that reads a set of instructions and carries them out in the right order. A chess engine reads a position; a search engine reads a query. **Our engine reads a workflow** — the JSON from section 1.2 — and calls each node's execute function at the right moment, with the right data. It is about forty lines long, and it is the heart of the whole project.

engine

What it is. A program that reads instructions and carries them out in the right order.

What it does in this section. Ours reads a workflow and calls each node's function at the right moment with the right data.

How it connects. Phase 1 builds it and then nobody touches it again — Phases 2, 4 and 6 only add new ways to *call* it.

trigger

What it is. A node with nothing before it: the thing that starts a workflow.

What it does in this section. The engine finds it, hands it one empty item, and the run begins from there.

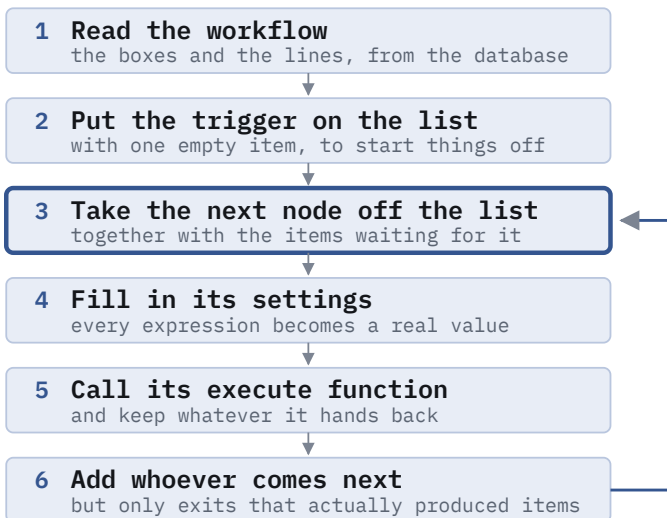
How it connects. Phase 4 builds two real ones — a web address and a clock — and neither requires changing the engine.

execution

What it is. One occasion on which a workflow actually ran, recorded so you can look back at it.

What it does in this section. The engine returns what every node produced; the server writes that down as a row.

How it connects. It is what the run list in the editor shows, and what the benchmark in Phase 6 measures.



THE ARROW GOES BACK TO STEP 3 UNTIL THE WAITING LIST IS EMPTY

THE SAME LOOP, RUNNING A REAL WORKFLOW

TICK	NODE RUN	PRODUCED	WAITING
1	Webhook	1 item	Code
2	Code	3 items	IF
3	IF	3 and 0	Gmail
4	Gmail	3 items	nothing
the list is empty, so the run is finished			

Look at tick 3

The IF sent 3 items down its true exit and 0 down its false exit. Step 6 only adds exits that produced something — so the false side simply never ran.

Figure 5. Left: the loop. Right: the same loop actually running our demo workflow, one tick at a time, until the waiting list empties.

FAMILIAR FROM N8N

Press *Execute Workflow* in n8n and you can watch nodes light up one after another, each turning green as it finishes, branches that were not taken staying grey. You were watching this exact loop run. Our version will do the same thing — and because we wrote it, we will be able to explain every step of it in the report.

1.6 The canvas and the API — the two halves of the app

What looks like one application is really **two programs having a conversation**. The **canvas** runs inside your browser and only draws. The **server** runs somewhere else and does all the real work. They never share memory — they only send each other messages, and the list of messages they agree on is called the **API**.

FAMILIAR FROM N8N

Open `localhost:5678` in a second browser window and your workflows are all still there. Close the browser entirely, reopen it — still there. That is because the workflows were never *in* the page. The page asked the server for them, and the server read them from its database. Every screen you saw in n8n was assembled that way.

server

What it is. A program that waits and answers when something asks it a question. No screen — it listens on a numbered door called a port.

What it does in this section. Ours listens on port 5678 and holds the engine, the nodes and the keys.

How it connects. Everything except the canvas runs inside it, which is why Phase 2 builds it before anything else can be wired up.

API

What it is. The list of questions a server agrees to answer, and the exact wording each needs. A menu, not the kitchen.

What it does in this section. We publish one with five entries; Google publishes one our nodes will call.

How it connects. It is the only thing the canvas and the server have to agree on — freeze it in Phase 2 and both halves can change freely.

endpoint

What it is. One item on that menu: a verb plus a path, such as `POST /rest/workflows/`.

What it does in this section. Each of our five does one job — list, fetch, create, save, run.

How it connects. Phase 4 adds two more of a different shape: `/webhook/:path` and its test twin.

database

What it is. A program whose only job is to remember things after everything else is switched off.

What it does in this section. Ours holds two tables: the drawings, and every past run.

How it connects. It is the reason a workflow is still there tomorrow, and the reason Phase 6 can deploy without losing anything.

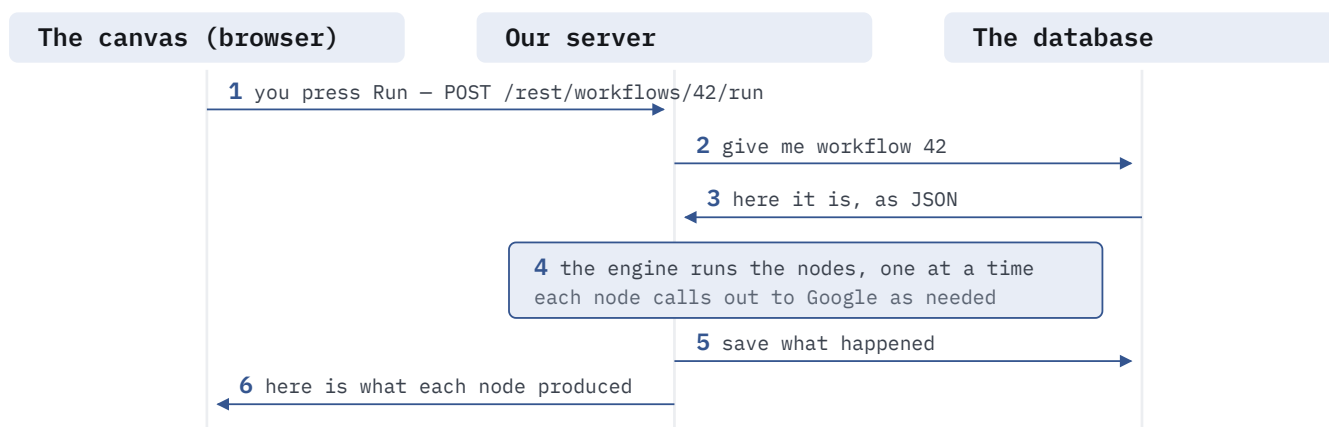


Figure 6. One click, six messages. The canvas never touches the database and never runs a node — it asks, and it displays the answer.

OUR OWN API

Five addresses is enough

List the workflows · fetch one · create one · save one · run one. That is the entire conversation our canvas needs for the first month. Each address has a verb (GET to fetch, POST to create, PATCH to change) and a path such as `/rest/workflows/42`.

OTHER PEOPLE'S APIS

The same idea, pointed outward

Google publishes an address that means “add this row to that spreadsheet”. Our Sheets node sends a request to it and reads the answer. A node is, almost always, one carefully built request to somebody else's API.

WHY WE CARE ABOUT THIS SPLIT

Because it is what lets two people work at once without colliding. Whoever builds the canvas and whoever builds the server only have to agree on those five addresses — the shape of each message, and what comes back. After that, one can redesign the whole screen and the other can rewrite the whole engine, and neither breaks the other. We write that agreement down in Phase 2 and then leave it alone.

1.7 The six phases

Everything from here on is building. The work splits into six phases, and **each one ends in something we can run and show somebody**. From this page onward each phase carries its own colour, used in its heading and in every diagram that belongs to it, so you can always tell at a glance which part of the project you are looking at.

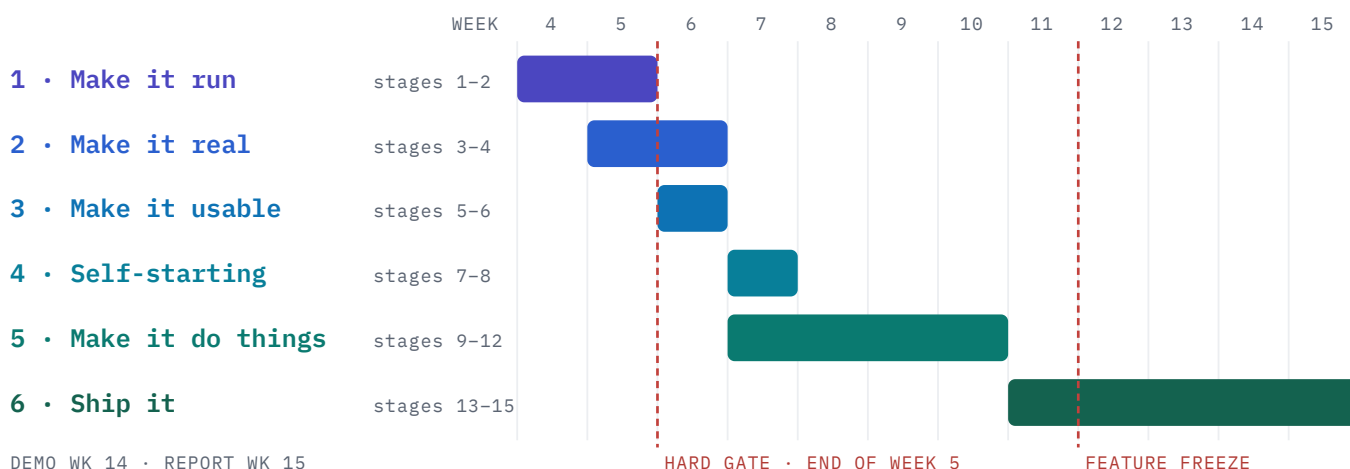


Figure 7. Two dates cannot move. By the end of Week 5 a workflow must run from end to end, however ugly it looks. At the end of Week 11 we stop adding features altogether.

1 · Make it run

A script that executes a workflow and prints the answer. No web page, no database.

2 · Make it real

Workflows are saved in a database and drawn on a canvas.

3 · Make it usable

Settings panels build themselves, and fields can carry live data.

4 · Self-starting

Workflows start on their own, from a web address or from a clock.

5 · Make it do things

Google permission, then the four Google nodes, the Code node and the AI node.

6 · Ship it

Online, measured against n8n, and written up.

THE RULE THAT PROTECTS THE SCHEDULE

If we fall behind we cut *nodes*, never phases. Losing the Drive node costs almost nothing. Losing Phase 3 means every later node needs a hand-built settings panel, and only one person can work at a time — which would cost us the whole of Weeks 8 to 10.

1.8 The toolkit — what we install, and when

The complete list for the semester. **Nobody needs to learn it all now** — each phase introduces two or three things, and about half of them are already built into Node.js. The guiding rule is on the right of the figure: build the thing the assignment is about, and borrow everything else.

We write this ourselves

the engine loop · the node shape · every node
the settings-panel generator · expressions
webhook handling · the schedule registry
the Google permission flow · the sandbox
this is the project — the report comes from it

We deliberately do not

dragging and connecting boxes (React Flow)
the database (Postgres) · HTTPS (Caddy)
running on a schedule (croner)
the code-editing box (Monaco)
solved problems — using them is a good decision

Figure 8. Borrowing the canvas library is not cheating; writing our own would cost a month and earn nothing. Borrowing the *engine* would be the mistake.

TOOL	WHAT IT IS FOR	PHASE
Node.js 22	The language everything is written in — the one we already learned this semester.	1
Git + GitHub	One shared folder with a full history of who changed what. Everyone works on a branch and merges.	1
TypeScript	JavaScript with labels on the data, so a mistake shows up in the editor instead of at three in the morning.	1
Docker Compose	Starts the database and the app together with one command, identically on all eight laptops.	2
PostgreSQL 16	The database. Remembers workflows and every past run.	2
Express	The server framework — it turns “when someone asks for this address, do that” into three lines.	2
React + Vite + React Flow	Builds the page in the browser, and gives us the canvas — dragging, drawing lines, panning and zooming, all solved.	2
Monaco	The code-editing box in the sidebar — literally the editor from VS Code.	3
croner	Runs a function on a schedule, with timezones handled properly.	4
Postman / curl + cloudflared	Send test requests to our own addresses, and give our laptop a temporary public address so outside services can reach it.	4
Google Cloud Console	The website where we switch on the four Google APIs and set up the permission screen. Free.	5
node:crypto, worker_threads	Both built into Node. One scrambles the stored Google keys; the other runs a user's JavaScript without risking the server.	5
Anthropic API	The AI node — one request, one response. Ollama can serve a local model instead, as in our weather project.	5
AWS EC2	One small rented computer that runs the finished platform, about \$20 a month.	6
Caddy	Gets and renews the HTTPS certificate by itself. Three lines of configuration.	6
n8n (Docker)	The platform we compare ourselves against, set up the same way as ours so the comparison is fair.	6
k6	Sends measured traffic at both platforms so the comparison uses real numbers.	6

1.9 Install everything, click by click

WHO DOES THIS PART

all eight

D2 checks everyone finished

WHAT YOU NEED TO KNOW

Nothing. This page assumes an empty laptop. Allow about an hour, once.

Do this before the first Monday. If any step fails, say so in the group chat the same day — half of these have a licence, an account or a restart in the way.

1 Node.js 22 — the language everything is written in

- 1 Go to nodejs.org
- 2 Click the big green **LTS** button — not Current
- 3 Run the installer, accept every default, and let it add Node to your PATH
- 4 Close every terminal window and open a new one — the PATH only updates in new windows

```
$ node -v      # must print v22.something
$ npm -v       # must print 10 or higher
```

2 VS Code, and the four extensions we all use

- 1 code.visualstudio.com → Download → install
- 2 Open it, then press **Ctrl+Shift+X** (**Cmd+Shift+X** on a Mac) for the Extensions panel
- 3 Search and install each: **ESLint** · **Prettier** · **Docker** · **GitLens**
- 4 **File** → **Preferences** → **Settings**, search **format on save**, tick it. Now nobody argues about spacing in a review.

3 Docker Desktop — how we run the database

- 1 docker.com/products/docker-desktop → download for your operating system
- 2 On Windows, when it asks, leave **Use WSL 2** ticked. It will make you restart.
- 3 Open Docker Desktop once and wait until the whale icon stops animating — nothing works until it has started
- 4 **Settings** → **Resources** → give it at least **4 GB** of memory

```
$ docker run --rm hello-world # prints "Hello from Docker!"
```

4 Git and GitHub — one repository, eight people

- 1 Install Git from git-scm.com (on a Mac, **xcode-select --install** is enough)
- 2 Make a GitHub account each. D2 then clicks **New repository** → name it **ai-automation** → **Private** → **Create**
- 3 **Settings** → **Collaborators** → **Add people** → add the other seven
- 4 **Settings** → **Branches** → **Add branch protection rule** → pattern **main** → tick **Require a pull request before merging** and **Require status checks to pass** → **Create**

```
$ git config --global user.name "Your Name"
$ git config --global user.email "you@example.com"
$ git clone https://github.com/<org>/ai-automation && cd ai-automation
$ git checkout -b your-name/first-change # never commit straight to main
```

5 Postman — for poking our own server, and Google's

- 1 postman.com/downloads → install → sign in (free)
- 2 **New** → **Collection** → name it **ai-automation**. Everything we try by hand lives here.
- 3 Inside it, **Add request** → set the verb to **POST**, the URL to **http://localhost:5678/rest/workflows**
- 4 Open the **Body** tab → choose **raw** → change **Text** to **JSON** → paste your JSON → **Send**
- 5 **Collection** → **...** → **Export** and commit the file, so all eight of us share the same requests

DONE WHEN

All five commands above print what they should, on all eight laptops. D2 keeps a tick-list — it is much cheaper to fix a broken install now than on the Thursday somebody is blocked by it.

1.10 VS Code: the program every line of this book is pasted into

WHO NEEDS THIS PAGE

all eight

read it once, then forget it

WHAT YOU NEED TO KNOW

Nothing at all. Every code block in Parts 2 and 3 is pasted into this one program, and this page is how it works.

Whenever this book shows a file name above a block of code, it means: **open that file in VS Code and put this in it.** Whenever it shows a `$` prompt, it means: **type that into the terminal at the bottom of VS Code.** That is the whole convention.

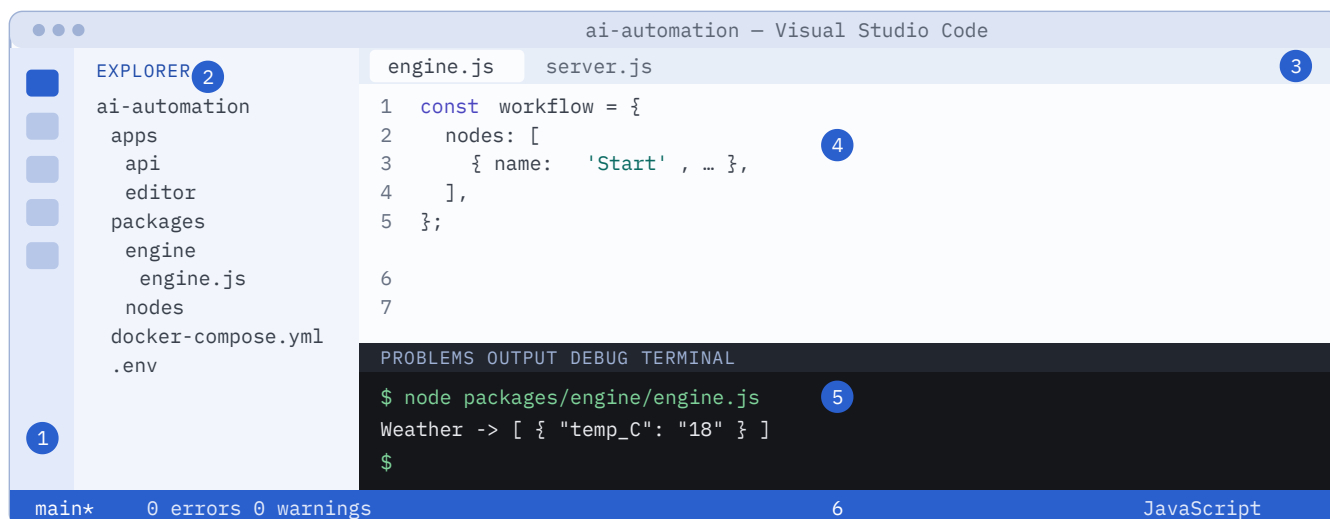


Figure 9. The only six parts of VS Code you need. Everything this book asks you to do happens in region 3 (the editor) or region 5 (the terminal).

1 · Activity bar

Five icons down the left. Top to bottom: **Explorer** (files), Search, **Source Control** (git), Run, **Extensions**. Click one to change what the panel beside it shows.

2 · Explorer

Your project's folders. This is where you create files: **right-click a folder → New File**, type the name with its extension, press Enter.

3 · Editor

Where the code goes. Each open file is a tab. Click into it, select all with `Ctrl A`, paste with `Ctrl V`, save with `Ctrl S`.

4 · Line numbers

When somebody says “line 42 is broken”, this is what they mean. Red squiggles under code are ESLint telling you something before you run it.

5 · Terminal

Open it with `Ctrl `` (the key above Tab). Every `$` line in this book is typed here — without the `$`.

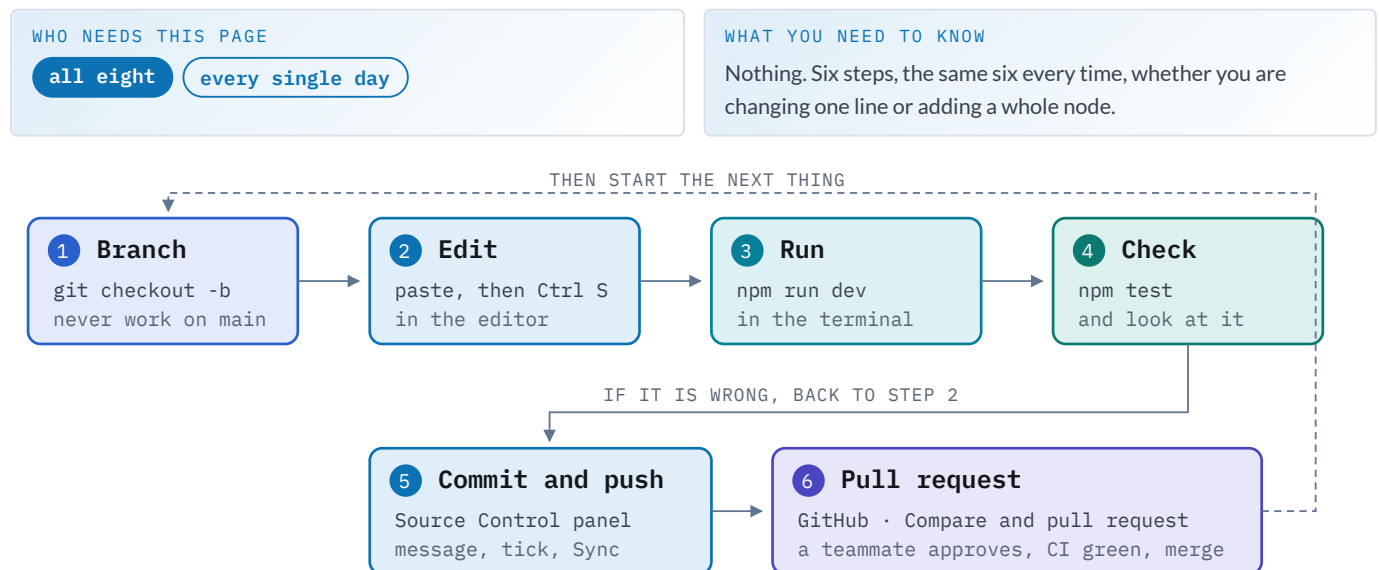
6 · Status bar

Left: the git branch you are on — check it before you type. Right: errors and warnings. If it says `main`, stop and make a branch first.

THE THREE SHORTCUTS WORTH MEMORISING

`Ctrl S` save · `Ctrl `` show or hide the terminal · `Ctrl P` jump to any file by typing part of its name. On a Mac use `Cmd` instead of `Ctrl`. Everything else you can click for.

1.11 The loop you will repeat a hundred times



Steps 1, 5 and 6 take about a minute between them. Steps 2 to 4 are the work. Nothing here changes for the rest of the semester.

Figure 10. The loop. Every code block in Parts 2 and 3 is step 2; every “check it works” box is step 4.

1 Make a branch before you touch anything

```
$ git pull                                # get everyone else's work first
$ git checkout -b amy/webhook-node        # yourname/what-you-are-doing
```

Check the bottom-left of VS Code: it should now show your branch name, not `main`.

2 Create the file, paste, save

- 1 In the **Explorer**, right-click the folder this book names → **New File**
- 2 Type the exact file name including the extension, press **Enter**
- 3 Click into the editor, paste, then **Ctrl S**. Prettier reformats it as it saves — that is expected.

3 Run it and look at the result

```
$ npm run dev          # leave this running in one terminal
# open a SECOND terminal with the + button, and use that one for everything else
```

4 Prove it, before anyone else sees it

```
$ npm test              # the checks for your part
$ npm run lint          # the same checks CI will run
```

If either is red, you are still on step 2. Pushing red work costs somebody else an hour.

5 Commit and push — with buttons, or with typing

- 1 Click the **Source Control** icon (third down the left bar) — your changed files are listed
- 2 Hover a file → **+** to stage it. Review the diff first: VS Code shows old and new side by side.
- 3 Type a message in the box — *what changed and why*, not “update” — then click **✓ Commit**
- 4 Click **Sync Changes** (or **Publish Branch** the first time)

```
$ git add -A && git commit -m "add the webhook node" && git push -u origin HEAD
```

6 Open the pull request

- 1 Open the repository on GitHub — a yellow bar offers `Compare & pull request`
- 2 Title it like the commit; in the body say what you checked
- 3 On the right, `Reviewers` → pick **your pair** (A1↔A2, B1↔B2, C1↔C2, D1↔D2)
- 4 Wait for the green tick from CI, then `Merge pull request` → `Delete branch`

THE TWO MISTAKES EVERYONE MAKES ONCE

Working on `main` without noticing — check the bottom-left corner. And pushing before `npm test` passes, which turns your five-minute problem into somebody else's afternoon.

1.12 What the interface should look like

WHO BUILDS THIS PART

B1

B2

agreed with everyone in Week 5

WHAT YOU NEED TO KNOW

CSS — spacing, borders, and one set of colour variables. No design experience: the numbers below are the design.

A marker sees the screen before they see the code. Four rules make it look considered: **one accent colour**, **one spacing step**, **one corner radius**, and **no drop shadows except on the thing being dragged**. Everything below follows from those.

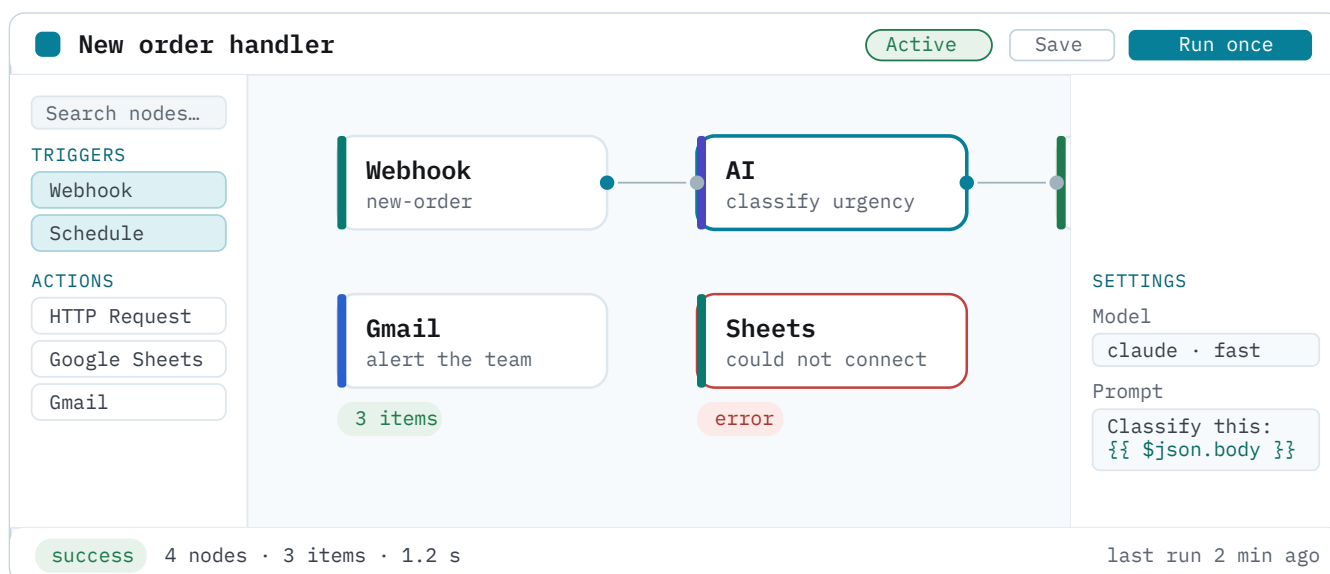


Figure 11. The whole editor. Palette left, canvas centre, settings right, one status strip along the bottom. The selected node has a heavier border; a failed node has a red one; item counts sit on the connection that produced them.

The node card

150 × 52, radius 10, white, 1 px border. A 5 px coloured stripe down the left says which family it belongs to — trigger, transform, Google, AI, control flow.

Title 13 px semibold; subtitle 11 px muted, and the subtitle is always the setting that matters most: the webhook path, the operation, the sheet name.

State, without a legend

Selected = 2 px accent border. **Running** = accent border plus a pulsing dot. **Failed** = red border and a red pill. **Never ran** = no pill at all.

Item counts sit on the *connection*, not the node, because that is where the data actually is — and it teaches the item model without a word of explanation.

TOKEN	VALUE	USED FOR
--accent	#087f99	The Run button, selected borders, focus rings. One accent, nothing else.
--ink / --muted	#14161a / #6b7686	Titles and subtitles. Never pure black, never lighter than the muted value.
--line	#dfe7ec	Every border and divider in the app. One line colour only.
--radius	10px / 4px	Cards use 10, controls use 4. No other radii.
--space	4 · 8 · 12 · 24	Every margin and padding is one of these four. This single rule does most of the work.
status	#1f7a4d / #c2413c	Success and failure pills. Never used for anything decorative.

HOW TO MAKE IT LOOK GOOD WITHOUT A DESIGNER

Pick the spacing from the four values above and never invent a fifth. Give every interactive thing a visible `:focus` ring in the accent colour. Keep one weight of border. And leave more empty space around the canvas than feels necessary — crowding is what makes student projects look unfinished.

PHASE 1 · MAKE IT RUN

STAGES 1-2 · WEEKS 4-5 · LEAD: A1, REVIEWED BY A2

2.1 A workflow runs, from a single file

WHO BUILDS THIS PART

A1

A2 reviews

WHAT YOU NEED TO KNOW

Plain JavaScript — arrays, objects, **async/await**. No framework, no database, no React. If you can read a `for` loop and a promise, you can write this phase.

What we are doing. Writing the loop from section 1.5 as one small program, and agreeing the exact shape every node will have. **No web page, no database, no canvas** — just a file we run from the terminal and watch print real results.

```
engine.js

$ node engine.js

Start -> [ { } ]

Weather -> [ { "temp_C": "18",
  "desc": "Partly cloudy" } ]

Shape -> [ { "city": "Newcastle",
  "temp": 18 } ]

done
```

What just happened

A hard-coded workflow object, three node functions and the loop — all in one file, about forty lines.

It called a real weather API, passed the result to the next node, and printed what each one produced.

everything else is built around this

Figure 12. The whole product, on the first day. Every later phase adds a way to create, store, trigger or display this — never a second way to run it.

FAMILIAR FROM N8N

This is the moment n8n goes from magic to machinery. What you watched happen on the n8n canvas — nodes lighting up in order, data appearing in each output panel — is exactly what those printed lines are. The only difference is that n8n draws it prettily and we are printing it to a terminal.

TOOLS WE USE IN THIS PHASE

Node.js	the language we already learned — nothing new to install	VS Code + Git	one shared repository, everyone on a branch
TypeScript	used lightly, only to write down the node shape	No frameworks	deliberately — this phase is plain JavaScript in one file

HOW WE PROCEED

- 1. One person writes the file.** A hard-coded workflow, two or three node functions, and the waiting-list loop. It should end by printing real data from a public weather API.
- 2. All eight of us read it together** until nobody has a question. This is worth an hour of everyone's time — it is the only genuinely conceptual part of the whole project.
- 3. We write the node shape down** in the repository: what a node receives, what it must hand back, and what it is never allowed to do (touch the database, draw anything, or decide what runs next).
- 4. We build two boring nodes against it** — one that adds fields to each item, one that splits the flow in two. Nothing that can fail for reasons outside our control.
- 5. We freeze the shape.** From here on, changing it needs everyone to agree, because a change breaks work already in progress.

DONE WHEN

Three nodes run in a chain, the splitting node sends data down the correct branch, and every one of us can explain the loop without notes.

WATCH OUT

Do not put eight people on this. A1 writes, A2 reads closely, and the other six work on the independent tracks in section 4.2. Parallel work on an unstable core produces eight rewrites — the most common way a large group fails quietly in week three.

2.1b Build it: the engine, line by line

Everything on this page goes into **three small files**. Type them, run them, and you have a workflow engine that not only works but refuses to lie to you when it breaks — it validates, it times out, it retries, and it tells you which node went wrong.

PROGRAM VS Code — see §1.10 for the window, §1.11 for the loop

FILES `packages/engine/errors.js`, `packages/engine/http.js`, `packages/engine/engine.js`

CREATE THEM In the Explorer, right-click `packages/engine` → **New File** → type the name with its extension → paste → **Ctrl S**

1 Create the workspace

```
$ mkdir ai-automation && cd ai-automation
$ npm init -y
$ npm pkg set type=module workspaces[0]="packages/*" workspaces[1]="apps/*"
$ npm pkg set engines.node=">=22"
$ mkdir -p packages/engine packages/nodes apps/api apps/editor
$ node -v           # must print v22.x — v20 has no AbortSignal.timeout in fetch
```

`type=module` lets us use `import`; `workspaces` is npm's own monorepo support — no extra tool to install. `engines.node` makes npm warn a teammate who is still on an old Node instead of letting them hit strange errors.

2 One error type, so every failure names its node

The single biggest difference between a toy engine and a usable one is that when something breaks, the message says *which box on the canvas* broke. That costs eleven lines.

`packages/engine/errors.js`

```
export class WorkflowError extends Error {
  constructor(message, { node = null, code = 'WORKFLOW_ERROR', cause = null } = {}) {
    super(message);
    this.name = 'WorkflowError';
    this.node = node;      // which node on the canvas
    this.code = code;      // machine-readable: the editor colours the box from this
    if (cause) this.cause = cause;
  }
  toJSON() { return { name: this.name, code: this.code, node: this.node, message: this.message }; }
}
```

`toJSON()` — when you call `JSON.stringify(err)`, JavaScript quietly returns `{}` for a plain `Error`: the message is invisible. Defining `toJSON` is what lets us store the real failure in the database and show it in the sidebar later.

3 One HTTP helper, with a timeout and a retry, used by every node forever

Write this once now and no node author ever has to think about flaky networks again.

```

const sleep = (ms) => new Promise(r => setTimeout(r, ms));

export async function httpRequest(url, opts = {}) {
  const { retries = 3, timeoutMs = 30000, ...init } = opts;
  let lastError;

  for (let attempt = 0; attempt <= retries; attempt++) {
    try {
      const res = await fetch(url, { ...init, signal: AbortSignal.timeout(timeoutMs) });

      // 429 = "you are going too fast", 5xx = "my fault" – both are worth trying again
      if (res.status === 429 || res.status >= 500) {
        const after = Number(res.headers.get('retry-after')) * 1000;
        throw Object.assign(new Error(`HTTP ${res.status}`), { retryAfter: after || 0 });
      }

      // 4xx = "your fault" – retrying changes nothing, so stop immediately
      if (!res.ok) {
        const body = (await res.text()).slice(0, 300);
        throw Object.assign(new Error(`HTTP ${res.status}: ${body}`), { fatal: true });
      }

      const type = res.headers.get('content-type') || '';
      return type.includes('json') ? await res.json() : { data: await res.text() };
    } catch (err) {
      lastError = err;
      if (err.fatal || attempt === retries) break;

      // exponential backoff with jitter: 0.5s, 1s, 2s ... plus a random nudge so eight
      // parallel items do not all wake up and hammer the server at the same millisecond
      await sleep(err.retryAfter || Math.min(8000, 2 ** attempt * 500) + Math.random() * 250);
    }
  }

  throw lastError;
}

```

Jitter — the random extra wait. Without it, every retry in a batch fires at exactly the same moment and knocks the far end over a second time. Two words of code, and it is the difference between a demo that works and a demo that does not.

4 The workflow, and two node types

packages/engine/engine.js

```
import { WorkflowError } from './errors.js';
import { httpRequest } from './http.js';

const workflow = {
  nodes: [
    { name: 'Start', type: 'manualTrigger', parameters: {} },
    { name: 'Weather', type: 'httpRequest',
      parameters: { url: 'https://wttr.in/Newcastle?format=j1' } },
  ],
  // one entry per node: a list of output branches, each listing who it feeds
  connections: { 'Start': [['Weather']] },
};

const nodeTypes = {
  manualTrigger: {
    description: { name: 'manualTrigger', group: 'trigger', properties: [] },
    async execute() { return [{ json: {} }]; },
  },
  httpRequest: {
    description: { name: 'httpRequest', group: 'action', properties: [] },
    async execute(ctx) {
      const out = [];
      const items = ctx.getInputData();
      for (let i = 0; i < items.length; i++) {
        const url = ctx.getNodeParameter('url', i);
        ctx.logger.info(`GET ${url}`);
        out.push({ json: await httpRequest(url, { timeoutMs: 15000 }) });
      }
      return [out]; // one branch, containing every item
    },
  },
};
```

5 Check the workflow makes sense *before* running any of it

A misspelled node name should be a clear sentence, not a crash halfway through a run that has already sent three emails.

```

export function validateWorkflow(workflow, nodeTypes) {
  const problems = [];
  const nodes = Array.isArray(workflow?.nodes) ? workflow.nodes : [];
  if (nodes.length === 0) problems.push('the workflow has no nodes');

  const names = new Set();
  for (const n of nodes) {
    if (!n?.name) problems.push('a node has no name');
    else if (names.has(n.name)) problems.push(`two nodes are both called "${n.name}"`);
    else names.add(n.name);
    if (n?.name && !nodeTypes[n.type]) problems.push(`"${n.name}" has unknown type "${n.type}"`);
  }

  for (const [from, branches] of Object.entries(workflow?.connections ?? {})) {
    if (!names.has(from)) problems.push(`a connection starts at missing node "${from}"`);
    for (const targets of branches ?? [])
      for (const to of targets ?? [])
        if (!names.has(to)) problems.push(`"${from}" points at missing node "${to}"`);
  }

  const triggers = nodes.filter(n => nodeTypes[n.type]?.description.group === 'trigger');
  if (triggers.length !== 1)
    problems.push(`a workflow needs exactly one trigger – this one has ${triggers.length}`);

  if (problems.length)
    throw new WorkflowError('This workflow cannot run:\n  · ' + problems.join('\n  · '),
      { code: 'INVALID_WORKFLOW' });

  return triggers[0];
}

```

6 Two guards the loop cannot do without

packages/engine/engine.js – continued

```
// stop a hung node instead of hanging the whole server with it
function withTimeout(promise, ms, message) {
  let timer;
  const bell = new Promise((_, reject) => {
    timer = setTimeout(() => reject(new WorkflowError(message, { code: 'NODE_TIMEOUT' })), ms);
  });
  return Promise.race([promise, bell]).finally(() => clearTimeout(timer));
}

// a node may return sloppy data; the rest of the engine must never see it
function normalise(branches, nodeName, maxItems) {
  if (!Array.isArray(branches))
    throw new WorkflowError(`"${nodeName}" must return an array of branches`,
      { node: nodeName, code: 'BAD_NODE_OUTPUT' });

  let total = 0;
  return branches.map((branch, i) => {
    if (!Array.isArray(branch))
      throw new WorkflowError(`branch ${i} of "${nodeName}" is not an array`,
        { node: nodeName, code: 'BAD_NODE_OUTPUT' });

    total += branch.length;
    if (total > maxItems)
      throw new WorkflowError(`"${nodeName}" produced more than ${maxItems} items – refusing to go on`,
        { node: nodeName, code: 'TOO_MANY_ITEMS' });

    return branch.map(item =>
      item && typeof item === 'object' && 'json' in item ? item : { json: item ?? {} });
  });
}
```

7 The loop from Figure 5 — now with a seatbelt

packages/engine/engine.js — continued


```

const DEFAULTS = { maxSteps: 1000, maxItems: 10000, nodeTimeoutMs: 60000 };

export async function runWorkflow(workflow, nodeTypes, options = {}) {
  const cfg = { ...DEFAULTS, ...options };
  const trigger = validateWorkflow(workflow, nodeTypes); // 1 · refuse bad input early
  const byName = Object.fromEntries(workflow.nodes.map(n => [n.name, n]));
  const results = {};
  const log = [];
  const ready = [{ nodeName: trigger.name, items: cfg.triggerItems ?? [{ json: {} }] }];
  let steps = 0;

  while (ready.length > 0) {
    if (++steps > cfg.maxSteps) // 2 · a loop cannot run forever
      throw new WorkflowError(`Stopped after ${cfg.maxSteps} steps - this workflow loops`,
        { code: 'STEP_LIMIT' });

    const { nodeName, items } = ready.shift(); // 3 · take the next node
    const node = byName[nodeName];
    const type = nodeTypes[node.type];
    const startedAt = Date.now();

    const ctx = { // 4 · what the node may ask for
      getInputData: () => items,
      getNodeParameter(name, i = 0, fallback = undefined) {
        const value = node.parameters?.[name] ?? fallback;
        if (value === undefined)
          throw new WorkflowError(`${nodeName} needs its "${name}" setting filled in`,
            { node: nodeName, code: 'MISSING_PARAMETER' });
        return value;
      },
      helpers: { httpRequest },
      logger: { info: (msg) => log.push({ node: nodeName, level: 'info', msg }) },
    };

    let branches;
    try { // 5 · run it, safely
      branches = await withTimeout(Promise.resolve(type.execute(ctx)), cfg.nodeTimeoutMs,
        `${nodeName} ran longer than ${cfg.nodeTimeoutMs} ms`);
      branches = normalise(branches, nodeName, cfg.maxItems);
    } catch (err) {
      log.push({ node: nodeName, level: 'error', ms: Date.now() - startedAt, msg: err.message });
      if (!node.continueOnFail)
        throw err instanceof WorkflowError
          ? err
          : new WorkflowError(`${nodeName} failed: ${err.message}`,
            { node: nodeName, code: 'NODE_FAILED', cause: err });
      branches = [{ json: {}, error: err.message }]; // carry the failure downstream
    }

    results[nodeName] = branches;
    log.push({ node: nodeName, level: 'done', ms: Date.now() - startedAt,
      items: branches.reduce((n, b) => n + b.length, 0) });

    const targets = workflow.connections[nodeName] ?? []; // 6 · push what comes next
    branches.forEach((branchItems, i) => {

```

```

    if (branchItems.length === 0) return; // an empty branch goes quiet
    for (const to of targets[i] ?? []) ready.push({ nodeName: to, items: branchItems });
  });
}
return { results, log, steps };
}

```

8 Run it

packages/engine/engine.js – last three lines

```

const { results, log } = await runWorkflow(workflow, nodeTypes);
for (const l of log) console.log(`${l.level.padEnd(5)} ${l.node.padEnd(8)}`,
    l.msg ?? `${l.items} item(s) in ${l.ms} ms`);
console.log('Weather →', JSON.stringify(results.Weather[0][0].json).slice(0, 60), '...');

```

```
$ node packages/engine/engine.js
```

WHAT YOU SHOULD SEE

```

Terminal – node packages/engine/engine.js

done Start      1 item(s) in 0 ms
info Weather GET https://wttr.in/Newcastle?format=j1
done Weather  1 item(s) in 412 ms
Weather → {"current_condition":[{"temp_C":"18",...}] ...
$ _

```

```

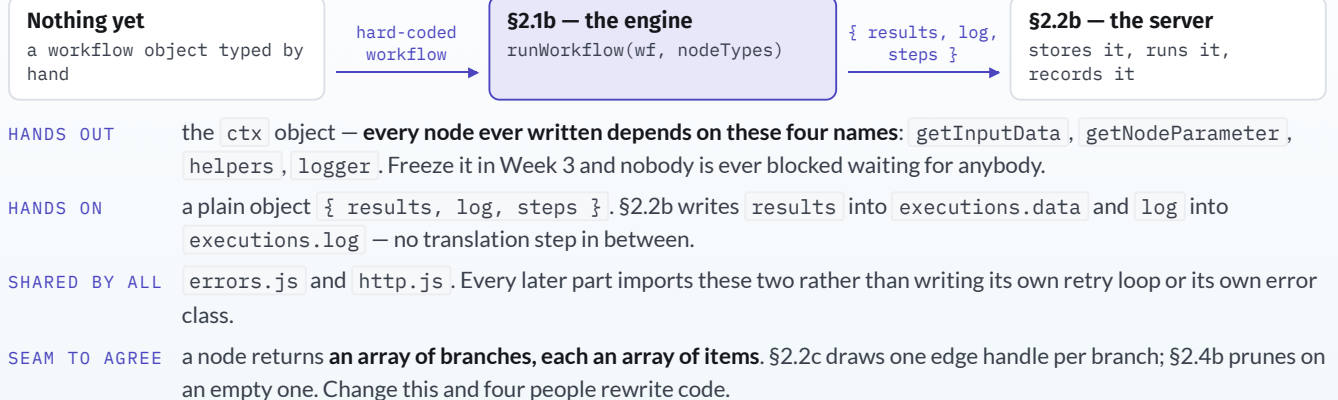
Terminal – after you misspell a node name

WorkflowError: This workflow cannot run:
  · "Start" points at missing node "Wether"
  · a workflow needs exactly one trigger – this one has 0
    at validateWorkflow (engine.js:41:11)
code: 'INVALID_WORKFLOW'

```

Read the left screen as a receipt. One line per node, in the order the loop chose them, with a millisecond count you can quote in the report. The right screen is the whole point of step 5: nothing ran, nothing was sent, and the message names the exact typo.

HOW THIS PART CONNECTS TO THE OTHERS



CHECK IT WORKS

Real weather appears, and each line of the log names a node. Then break it on purpose, three ways, and confirm each guard fires: (a) make `httpRequest` return `[]` and everything after it goes quiet – that is branch pruning; (b) point the URL at `https://httpstat.us/503` and watch three retries then a clean failure; (c) add `connections: { Weather: [['Start']] }` to make a loop and confirm the step limit stops it instead of your laptop's fan.

PHASE 2 · MAKE IT REAL STAGES 3-4 · WEEKS 5-6 · LEAD: C1 · B1

2.2 Workflows survive a restart, and we can draw them

WHO BUILDS THIS PART

C1

B1

D2 helps with Docker

WHAT YOU NEED TO KNOW

SQL enough to create a table and write five queries · **HTTP verbs** (GET, POST, PATCH) · **React** basics — components, state, effects · running one Docker command.

What we are doing. Giving the engine a home. A **database** remembers workflows, a **server** hands them out through the five addresses from section 1.6, and a **canvas** lets us draw one instead of typing JSON by hand.

container

What it is. A program packed with everything it needs, so it behaves identically on every machine.

What it does in this section. We start Postgres as one, so nobody installs a database by hand.

How it connects. The same idea deploys the finished platform in Phase 6 — the container that runs on your laptop is the one that runs on the server.

migration

What it is. A file of SQL that creates or changes tables, kept in the repository and run in order.

What it does in this section. `schema.sql` is our first one; every later table change adds another.

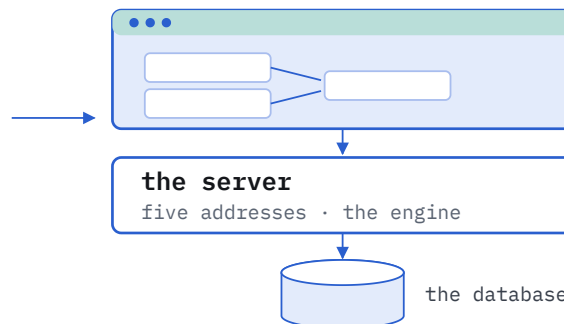
How it connects. It means all eight laptops and the live server end up with exactly the same database shape.

AFTER PHASE 1

PHASE 2 AFTER PHASE 2

```
$ node engine.js
Start -> [ { } ]
Weather -> [ ... ]
done
```

one file, hard-coded,
gone when it finishes



What is new

a page you can draw on
five addresses to call
two tables that remember
a Run button that works

the engine itself does
not change — it is just
called from elsewhere

Figure 13. Phase 2 wraps phase 1 in the four things every web application has: a screen, a server, a database, and a way back. Nothing about the engine is rewritten.

FAMILIAR FROM N8N

Everything on this page is what n8n already gave you for free: a canvas that saved when you clicked away, a workflow list that survived a restart, and a Run button. You never thought about the database behind it — which is exactly the experience we are now building for ourselves.

TOOLS WE USE IN THIS PHASE

Docker Compose	starts the database with one command — nobody installs Postgres by hand	PostgreSQL	remembers the workflows and every past run
Express	the smallest useful server — readable end to end in an afternoon	node-postgres	talks to the database in plain SQL; no extra layer to learn
React + Vite	builds the page the canvas lives in	React Flow	dragging, connecting, panning and zooming — already solved

HOW WE PROCEED

1. **Start the database in a container.** One configuration file, one command, the same on every laptop.
2. **Two tables only:** one for workflows, one for runs. The whole drawing goes into a single column as the text from Figure 2 — which is what makes saving one write and loading one read.
3. **Build the five addresses:** list, fetch, create, save, run. That really is enough for the next month.
4. **Put React Flow on the page.** We write the boxes and the sidebar; we do not write dragging, zooming or line-drawing.

5. **Join the two halves** so the drawing saves to the database and the Run button executes it, showing what each node produced.

DONE WHEN

We can drag out two nodes, connect them, refresh the browser and find them still there — then press Run and see the result of each one.

WATCH OUT

Converting between React Flow's format and ours is the fiddly job here, and essentially every canvas bug for the rest of the semester will live in it. Write it once, carefully, with someone reading over your shoulder.

2.2b Build it: database, server and canvas

PROGRAM	VS Code — see §1.10 for the window, §1.11 for the loop
FILES	<code>docker-compose.yml</code> , <code>schema.sql</code> , <code>apps/api/db.js</code> , <code>apps/api/server.js</code>
CREATE THEM	In the Explorer, right-click the project root, then <code>apps/api</code> → <code>New File</code> → type the name with its extension → paste → <code>Ctrl S</code>

1 Start Postgres in a container

`docker-compose.yml`

```
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: dev
      POSTGRES_DB: automation
    ports: ["5432:5432"]
    volumes: ["pgdata:/var/lib/postgresql/data"]
    # docker will not report the container "ready" until Postgres actually answers,
    # so 'npm run dev' never races ahead of the database on a cold laptop
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d automation"]
      interval: 5s
      timeout: 3s
      retries: 10
volumes:
  pgdata: {}
```

```
$ docker compose up -d
$ docker compose ps           # wait until STATUS says (healthy)
$ psql postgresql://postgres:dev@localhost:5432/automation -f schema.sql
```

2 Two tables — with the constraints that stop bad rows existing at all

schema.sql

```
CREATE TABLE IF NOT EXISTS workflows (  
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  name        text NOT NULL CHECK (length(trim(name)) BETWEEN 1 AND 120),  
  active       boolean NOT NULL DEFAULT false,  
  nodes        jsonb NOT NULL DEFAULT '[]' CHECK (jsonb_typeof(nodes) = 'array'),  
  connections  jsonb NOT NULL DEFAULT '{}' CHECK (jsonb_typeof(connections) = 'object'),  
  version      integer NOT NULL DEFAULT 1,          -- bumped on every save; stops silent overwrites  
  created_at   timestampz NOT NULL DEFAULT now(),  
  updated_at   timestampz NOT NULL DEFAULT now()  
);  
  
CREATE TABLE IF NOT EXISTS executions (  
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  workflow_id uuid NOT NULL REFERENCES workflows(id) ON DELETE CASCADE,  
  mode        text NOT NULL DEFAULT 'manual'  
              CHECK (mode IN ('manual', 'webhook', 'schedule')),  
  status      text NOT NULL CHECK (status IN ('running', 'success', 'error', 'cancelled')),  
  data        jsonb,          -- what each node produced  
  log         jsonb NOT NULL DEFAULT '[]',          -- the receipt from the engine  
  error       jsonb,  
  started_at  timestampz NOT NULL DEFAULT now(),  
  finished_at timestampz  
);  
  
-- the executions list is the most-hit query in the whole product: index it now  
CREATE INDEX IF NOT EXISTS executions_by_workflow  
  ON executions (workflow_id, started_at DESC);  
CREATE INDEX IF NOT EXISTS executions_running  
  ON executions (status) WHERE status = 'running';  
  
-- keep updated_at honest without every route remembering to set it  
CREATE OR REPLACE FUNCTION touch_updated_at() RETURNS trigger AS $$  
BEGIN NEW.updated_at = now(); NEW.version = OLD.version + 1; RETURN NEW; END;  
$$ LANGUAGE plpgsql;  
  
DROP TRIGGER IF EXISTS workflows_touch ON workflows;  
CREATE TRIGGER workflows_touch BEFORE UPDATE ON workflows  
  FOR EACH ROW EXECUTE FUNCTION touch_updated_at();
```

CHECK constraint — a rule the database itself enforces. Any code path, any teammate, any hand-typed `psql` command: a row that breaks the rule is simply rejected. It is the cheapest bug prevention in the project.

3 One connection pool, configured, in its own file

```
$ npm i express pg helmet cors express-rate-limit --workspace apps/api
```

```
import pg from 'pg';

if (!process.env.DATABASE_URL) {
  console.error('DATABASE_URL is not set - copy .env.example to .env first');
  process.exit(1);
}

export const db = new pg.Pool({
  connectionString: process.env.DATABASE_URL,
  max: 10, // never open more than 10 sockets to Postgres
  idleTimeoutMillis: 30_000,
  connectionTimeoutMillis: 5_000, // fail fast instead of hanging a request forever
});

// a dead pooled socket fires 'error' with nobody listening -> Node kills the process.
// One line stops the whole API dying because Postgres restarted.
db.on('error', (err) => console.error('[db] idle client error:', err.message));

export async function withTransaction(fn) {
  const client = await db.connect();
  try {
    await client.query('BEGIN');
    const out = await fn(client);
    await client.query('COMMIT');
    return out;
  } catch (err) {
    await client.query('ROLLBACK');
    throw err;
  } finally {
    client.release(); // ALWAYS - a leaked client is a frozen server at demo time
  }
}
```

4 The five routes, with validation and a single error handler

```
apps/api/server.js
```



```

import express from 'express';
import helmet from 'helmet';
import cors from 'cors';
import rateLimit from 'express-rate-limit';
import { db, withTransaction } from './db.js';
import { runWorkflow } from '../../packages/engine/engine.js';
import { WorkflowError } from '../../packages/engine/errors.js';
import { nodeTypes } from '../../packages/nodes/index.js';

const app = express();
app.use(helmet()); // sane security headers, free
app.use(cors({ origin: process.env.EDITOR_ORIGIN ?? 'http://localhost:5173' }));
app.use(express.json({ limit: '2mb' })); // a 900 MB body must not eat the server
app.use('/rest', rateLimit({ windowMs: 60_000, limit: 300 }));

// ---- small helpers -----
const UUID = /^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$/i;

class HttpError extends Error {
  constructor(status, message) { super(message); this.status = status; }
}

// every route is wrapped: a rejected promise becomes a clean 500, never a crash
const route = (fn) => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);

function requireId(req) {
  if (!UUID.test(req.params.id)) throw new HttpError(400, 'That is not a valid workflow id');
  return req.params.id;
}

function validateWorkflowBody(body, { partial = false } = {}) {
  const out = {};
  if (!partial || body.name !== undefined) {
    if (typeof body.name !== 'string' || !body.name.trim())
      throw new HttpError(400, 'name is required');
    if (body.name.length > 120) throw new HttpError(400, 'name must be 120 characters or fewer');
    out.name = body.name.trim();
  }
  if (!partial || body.nodes !== undefined) {
    if (!Array.isArray(body.nodes)) throw new HttpError(400, 'nodes must be an array');
    out.nodes = body.nodes;
  }
  if (!partial || body.connections !== undefined) {
    if (typeof body.connections !== 'object' || body.connections === null
      || Array.isArray(body.connections))
      throw new HttpError(400, 'connections must be an object');
    out.connections = body.connections;
  }
  return out;
}

// ---- the routes -----
app.get('/healthz', route(async (req, res) => {
  await db.query('SELECT 1');
  res.json({ ok: true, uptime: Math.round(process.uptime()) });
}));

```

```

    }));

app.get('/rest/workflows', route(async (req, res) => {
  const limit = Math.min(Number(req.query.limit) || 50, 200);
  const offset = Math.max(Number(req.query.offset) || 0, 0);
  const { rows } = await db.query(
    'SELECT id, name, active, updated_at FROM workflows ORDER BY updated_at DESC LIMIT $1 OFFSET $2',
    [limit, offset]);
  res.json({ data: rows, limit, offset });
}));

app.get('/rest/workflows/:id', route(async (req, res) => {
  const { rows } = await db.query('SELECT * FROM workflows WHERE id = $1', [requireId(req)]);
  if (!rows[0]) throw new HttpError(404, 'No workflow with that id');
  res.json(rows[0]);
}));

app.post('/rest/workflows', route(async (req, res) => {
  const wf = validateWorkflowBody(req.body);
  const { rows } = await db.query(
    'INSERT INTO workflows (name, nodes, connections) VALUES ($1, $2, $3) RETURNING *',
    [wf.name, JSON.stringify(wf.nodes ?? []), JSON.stringify(wf.connections ?? {})]);
  res.status(201).json(rows[0]);
}));

app.patch('/rest/workflows/:id', route(async (req, res) => {
  const id = requireId(req);
  const wf = validateWorkflowBody(req.body, { partial: true });
  // optimistic locking: the editor sends the version it loaded. If somebody else saved
  // in between, the UPDATE matches nothing and we say so instead of eating their work.
  const { rows } = await db.query(
    `UPDATE workflows SET name          = COALESCE($2, name),
                        nodes           = COALESCE($3, nodes),
                        connections     = COALESCE($4, connections),
                        active          = COALESCE($5, active)

    WHERE id = $1 AND ($6::int IS NULL OR version = $6) RETURNING *`,
    [id, wf.name ?? null,
     wf.nodes ? JSON.stringify(wf.nodes) : null,
     wf.connections ? JSON.stringify(wf.connections) : null,
     typeof req.body.active === 'boolean' ? req.body.active : null,
     req.body.version ?? null]);
  if (!rows[0]) throw new HttpError(409, 'Somebody else saved this workflow – reload before saving');
  res.json(rows[0]);
}));

app.post('/rest/workflows/:id/run', route(async (req, res) => {
  const id = requireId(req);
  const { rows: [wf] } = await db.query('SELECT * FROM workflows WHERE id = $1', [id]);
  if (!wf) throw new HttpError(404, 'No workflow with that id');

  const { rows: [run] } = await db.query(
    'INSERT INTO executions (workflow_id, status, mode) VALUES ($1, 'running', $2) RETURNING id',
    [wf.id, req.body?.mode ?? 'manual']);

  try {

```

```

const { results, log } = await runWorkflow(wf, nodeTypes, { nodeTimeoutMs: 60_000 });
await db.query(
  "UPDATE executions SET status='success', data=$1, log=$2, finished_at=now() WHERE id=$3",
  [JSON.stringify(results), JSON.stringify(log), run.id]);
res.json({ executionId: run.id, status: 'success', data: results, log });
} catch (err) {
const payload = err instanceof WorkflowError ? err.toJSON() : { message: String(err.message) };
await db.query(
  "UPDATE executions SET status='error', error=$1, finished_at=now() WHERE id=$2",
  [JSON.stringify(payload), run.id]);
res.status(422).json({ executionId: run.id, status: 'error', error: payload });
}
});

// ---- the two middlewares every Express app should end with -----
app.use((req, res) => res.status(404).json({ message: `No route for ${req.method} ${req.path}` }));

app.use((err, req, res, next) => {
const status = err.status ?? 500;
if (status >= 500) console.error('[api]', err); // log the stack, show the user nothing
res.status(status).json({ message: status >= 500 ? 'Internal error' : err.message });
});

// ---- start, and stop politely -----
const server = app.listen(process.env.PORT ?? 5678,
  () => console.log(`api on http://localhost:${process.env.PORT ?? 5678}`));

for (const signal of ['SIGINT', 'SIGTERM']) {
process.on(signal, () => {
console.log(`\n${signal} - finishing in-flight requests...`);
server.close(async () => { await db.end(); process.exit(0); });
setTimeout(() => process.exit(1), 10_000).unref(); // but never hang forever
});
}

```

Error middleware — an Express function with *four* arguments (`err`, `req`, `res`, `next`). Express treats the four-argument shape as special and sends every thrown error here. Miss the fourth argument and it silently becomes an ordinary route.

WHAT YOU SHOULD SEE

```

Terminal - bringing the database up

$ docker compose ps
NAME                IMAGE                STATUS                PORTS
ai-automation-db    postgres:16          Up 12 seconds (healthy) 0.0.0.0:5432->5432/tcp

$ psql $DATABASE_URL -c '\dt'
List of relations
Schema | Name      | Type  | Owner
-----+-----+-----+-----
public | executions | table | postgres
public | workflows  | table | postgres
(2 rows)

```

```

Terminal - a good request

$ curl -s -X POST localhost:5678/rest/workflows \
  -H 'content-type: application/json' \
  -d '{"name": "first", "nodes": [], "connections": {}}'

HTTP/1.1 201 Created
{
  "id": "3f9a...c1", "name": "first",
  "active": false, "version": 1,
  "nodes": [], "connections": {}
}

```

```

Terminal - every guard, in order

# no name
400 {"message": "name is required"}

# id is not a uuid
400 {"message": "That is not a valid workflow id"}

# unknown id
404 {"message": "No workflow with that id"}

# stale version - somebody else saved
409 {"message": "Somebody else saved this
      workflow - reload before saving"}

# the engine refused the workflow
422 {"status": "error", "error": {"code":
      "INVALID_WORKFLOW", "node": null, ...}}

```

Those five numbers are the deliverable, not the 201. Anyone can make a route that works; a marker looks for what happens when it does not. Screenshot this pane for the report.

HOW THIS PART CONNECTS TO THE OTHERS

\$2.1b — the engine
runWorkflow()

imported
directly

\$2.2b — server + database
5 routes · 2 tables

HTTP + JSON

\$2.2c — the canvas
fetch('/rest/workflows/...')

RECEIVES	the engine as a normal <code>import</code> — same process, no queue. \$2.4b later reuses the very same <code>runById()</code> wrapper for webhooks and schedules.
HANDS ON	the workflow JSON <code>{ id, name, nodes[], connections{}, version }</code> . That object is the single shared vocabulary between the server, the canvas and the engine.
SHARED BY ALL	<code>db.js</code> — one pool, one <code>withTransaction()</code> . \$2.4b, \$2.5b and \$2.6b all import it instead of opening connections of their own.
SEAM TO AGREE	<code>version</code> . The canvas sends the number it loaded; the server refuses a stale save with 409. Agree it in Week 5, or two people overwrite each other in Week 11.

CHECK IT WORKS

Create a workflow, restart the container with `docker compose restart`, fetch it back — it survived. Then run each failing request above and confirm the status code matches. Finally press `Ctrl C` in the API terminal: it should print `SIGINT — finishing in-flight requests...` and exit within a second, not die mid-query. That is Phase 2's hardest half done.

ELEC 3111 · Group 2 · AI Automation Platform

36

2.2c Build it: the canvas, and the two functions that matter

PROGRAM VS Code — see §1.10 for the window, §1.11 for the loop

FILES `apps/editor/src/convert.ts`, `api.ts`, `Editor.tsx`

CREATE THEM In the Explorer, right-click `apps/editor/src` → **New File** → type the name with its extension → paste → **Ctrl S**

4 Start the editor

```
$ npm create vite@latest apps/editor -- --template react-ts
$ cd apps/editor && npm i reactflow
$ npm run dev                # http://localhost:5173
```

Add the proxy line to `vite.config.ts` now, before anything else, or every request will fail with a cross-origin error that looks like a bug in your code:

```
apps/editor/vite.config.ts

export default defineConfig({
  plugins: [react()],
  server: { proxy: { '/rest': 'http://localhost:5678' } },
});
```

5 Our format → React Flow's, and back

These two functions are where *every* canvas bug will live for the rest of the semester. Write them once, carefully, defensively, and give them a test.


```

import type { Node as RfNode, Edge } from 'reactflow';

export type WorkflowNode = {
  name: string; type: string;
  parameters: Record<string, unknown>;
  position?: { x: number; y: number };
  disabled?: boolean; continueOnFail?: boolean; notes?: string;
};

export type Workflow = { id?: string; name: string; version?: number;
  nodes: WorkflowNode[]; connections: Record<string, string[][]> };

// laid out in a diagonal so a workflow with no saved positions is still readable
const fallbackPosition = (i: number) => ({ x: 80 + i * 240, y: 80 + (i % 2) * 120 });

export function toReactFlow(wf: Workflow) {
  const nodes: RfNode[] = (wf.nodes ?? []).map((n, i) => ({
    id: n.name,
    type: 'workflowNode',
    position: n.position ?? fallbackPosition(i),
    data: { node: n },
  }));
  const known = new Set(nodes.map(n => n.id));

  const edges: Edge[] = Object.entries(wf.connections ?? {}).flatMap(([from, branches]) =>
    (branches ?? []).flatMap((targets, branchIndex) =>
      (targets ?? [])
        // a connection to a deleted node would crash React Flow - drop it quietly
        .filter(to => known.has(to) && known.has(from))
        .map(to => ({
          id: `${from}:${branchIndex}->${to}`,
          source: from, target: to,
          sourceHandle: String(branchIndex), targetHandle: 'in',
        })))));

  return { nodes, edges };
}

export function fromReactFlow(nodes: RfNode[], edges: Edge[]): Pick<Workflow, 'nodes'|'connections'> {
  const connections: Record<string, string[][]> = {};
  for (const e of edges) {
    const branch = Number.isFinite(Number(e.sourceHandle)) ? Number(e.sourceHandle) : 0;
    connections[e.source] ??= [];
    for (let b = 0; b <= branch; b++) connections[e.source][b] ??= []; // no holes in the array
    if (!connections[e.source][branch].includes(e.target)) // no duplicate edges
      connections[e.source][branch].push(e.target);
  }
  return {
    nodes: nodes.map(n => ({
      ...(n.data.node as WorkflowNode),
      position: { x: Math.round(n.position.x), y: Math.round(n.position.y) },
    })),
    connections,
  };
}

```

```
// round-trip guard – call it in a test, and in dev after every save
export function isLossless(wf: Workflow) {
  const { nodes, edges } = toReactFlow(wf);
  const back = fromReactFlow(nodes, edges);
  return JSON.stringify(back.connections) === JSON.stringify(wf.connections);
}
```

`??=` — "assign only if this is null or undefined". It is what lets us build a nested array without four lines of `if` checks. `Pick<...>` is TypeScript for "the same type, but only these two fields" — so the compiler stops us returning a half-built workflow by accident.

6 One place that talks to the server, so errors are handled once

apps/editor/src/api.ts

```
export class ApiError extends Error {
  constructor(public status: number, message: string) { super(message); }
}

export async function api<T>(path: string, init: RequestInit = {}): Promise<T> {
  const res = await fetch(path, {
    ...init,
    headers: { 'content-type': 'application/json', ...init.headers },
    signal: init.signal ?? AbortSignal.timeout(20_000),
  });
  const text = await res.text();
  const body = text ? JSON.parse(text) : null;
  if (!res.ok) throw new ApiError(res.status, body?.message ?? `Request failed (${res.status})`);
  return body as T;
}
```


7 Wire the canvas to the server — load, autosave, run

`apps/editor/src/Editor.tsx` — the parts that matter

```

const [nodes, setNodes, onNodesChange] = useNodesState([]);
const [edges, setEdges, onEdgesChange] = useEdgesState([]);
const [status, setStatus] = useState<'idle' | 'saving' | 'saved' | 'error'>('idle');
const [message, setMessage] = useState('');
const version = useRef<number>(1);
const dirty = useRef(false);

useEffect(() => { // load
  const stop = new AbortController();
  api<Workflow>(`/rest/workflows/${id}`, { signal: stop.signal })
    .then(wf => {
      version.current = wf.version ?? 1;
      const { nodes, edges } = toReactFlow(wf);
      setNodes(nodes); setEdges(edges);
    })
    .catch(e => { if (e.name !== 'AbortError') { setStatus('error'); setMessage(e.message); } });
  return () => stop.abort(); // React 18 mounts twice in dev - this matters
}, [id]);

const save = useCallback(async () => {
  setStatus('saving');
  try {
    const saved = await api<Workflow>(`/rest/workflows/${id}`, {
      method: 'PATCH',
      body: JSON.stringify({ ...fromReactFlow(nodes, edges), version: version.current }),
    });
    version.current = saved.version ?? version.current + 1;
    dirty.current = false;
    setStatus('saved'); setMessage('');
  } catch (e) {
    setStatus('error');
    setMessage(e instanceof ApiError && e.status === 409
      ? 'Someone else saved this workflow. Reload to get their changes.'
      : (e as Error).message);
  }
}, [id, nodes, edges]);

useEffect(() => { // autosave, 1.2 s after you stop moving things
  if (!dirty.current) return;
  const t = setTimeout(save, 1200);
  return () => clearTimeout(t);
}, [nodes, edges, save]);

useEffect(() => { // Ctrl/Cmd-S, and a browser-close warning
  const onKey = (e: KeyboardEvent) => {
    if ((e.ctrlKey || e.metaKey) && e.key === 's') { e.preventDefault(); save(); }
  };
  const onLeave = (e: BeforeUnloadEvent) => { if (dirty.current) e.preventDefault(); };
  window.addEventListener('keydown', onKey);
  window.addEventListener('beforeunload', onLeave);
  return () => { window.removeEventListener('keydown', onKey);
    window.removeEventListener('beforeunload', onLeave); };
}, [save]);

const run = async () => {

```

```

if (dirty.current) await save(); // never run a version the server has not got
setStatus('saving'); setMessage('Running...');
try {
  const r = await api<RunResult>(`/rest/workflows/${id}/run`, { method: 'POST' });
  setRunResult(r); setStatus('saved'); setMessage(`Finished in ${totalMs(r.log)} ms`);
} catch (e) {
  setStatus('error');
  setMessage((e as ApiError).message);
  if ((e as any).error?.node) highlightFailedNode((e as any).error.node); // colour the box red
}
};

```

Set `dirty.current = true` inside your `onNodesChange` and `onEdgesChange` wrappers. Those eleven characters are what turn a Save button into an editor nobody can lose work in.

WHAT YOU SHOULD SEE

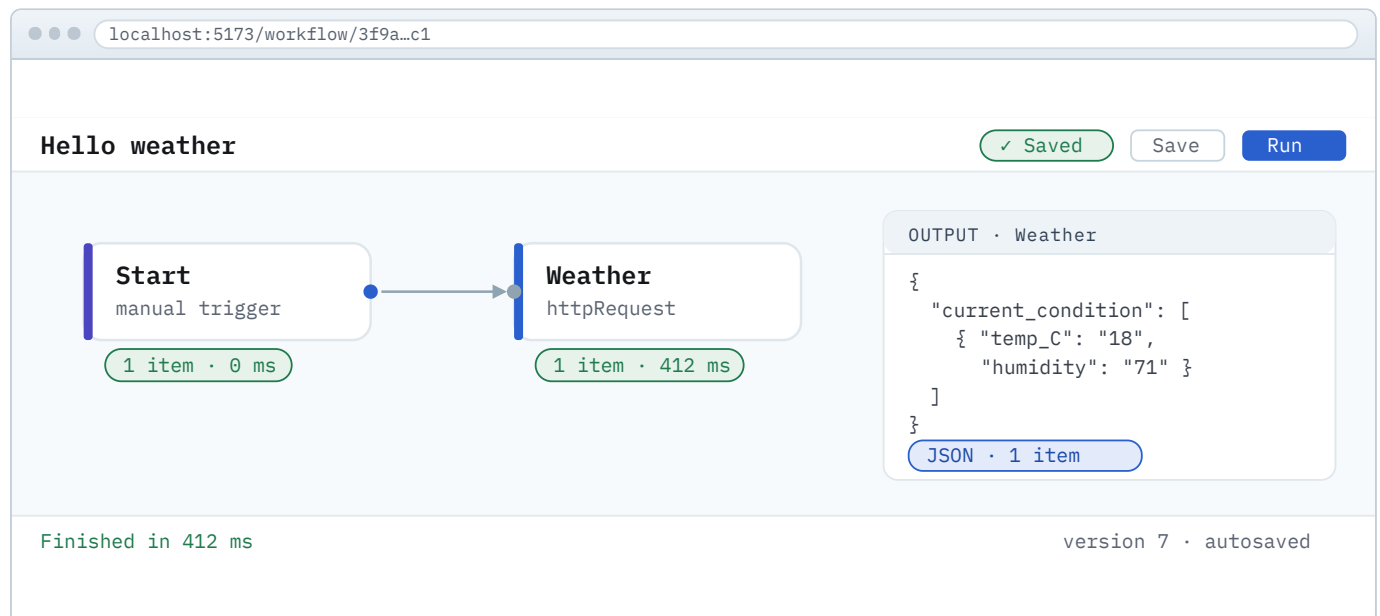


Figure 14. The Week 5 hard gate, on screen. Two nodes, one connection, a green item count under each box, the last node's output on the right, and an autosave that already happened — no Save button was pressed.

The three things a marker looks for are all visible here: the item count under each node (the engine reported it), the millisecond timing (the log carried it), and the version number in the status bar (optimistic locking is real). If a save collides, the green pill turns into a red one reading "Someone else saved this workflow."

HOW THIS PART CONNECTS TO THE OTHERS

\$2.2b — the server
GET / PATCH
`/rest/workflows/:id`

workflow
JSON

\$2.2c — the canvas
`toReactFlow` ≠ `fromReactFlow`

`node.parameters`

\$2.3b — the settings panel
renders `properties[]`

- RECEIVES** exactly the row \$2.2b returns. Nothing is reshaped on the way in — `toReactFlow` is the only translator in the project, and \$3.2b tests it both ways.
- HANDS ON** each node object to the panel of \$2.3b, which writes straight into `node.parameters`. The canvas never needs to know what a setting *means*.
- TALKS TO** \$2.4b: the executions list is the same `api()` helper pointed at `/rest/executions`, so error handling and timeouts were written once.
- SEAM TO AGREE** branch index = React Flow `sourceHandle`, and branch 0 is the IF node's *true* output. B1/B2 and A2 must write this down in Week 6.

CHECK IT WORKS

Drag two nodes out, connect them, wait 1.2 seconds without touching anything — the pill flicks to **Saving...** then **Saved**. Refresh the browser: they are still there, in the same positions. Press Run and each box grows its item count. Then open the same workflow in a second browser tab, save in one, save in the other — the second gets the red conflict message instead of silently destroying the first person's work.

PHASE 3 · MAKE IT USABLE

STAGES 5-6 · WEEK 6 · LEAD: B2 · A1

2.3 Nodes describe themselves, and fields carry live data

WHO BUILDS THIS PART

B2

A1

B1 reviews

WHAT YOU NEED TO KNOW

React conditional rendering and controlled inputs · a little **regular-expression** reading for the expression resolver · above all, patience for detail: this is the fiddliest phase and the most valuable.

What we are doing. The two mechanisms that look small and decide everything: the settings panel that **draws itself**, and **expressions**.

expression

What it is. A small piece of text in a settings box, in double braces, replaced with real data just before the node runs.

What it does in this section. It is how one field becomes a different value for every item.

How it connects. The engine resolves it inside `getNodeParameter`, so nodes never see the braces — which is why Phase 1's contract did not need to change.

schema-driven UI

What it is. A screen built from a description of the data rather than hand-written for each case.

What it does in this section. Our sidebar reads a node's property list and draws the form from it.

How it connects. It is the single reason six people can add six nodes in Weeks 8–10 without touching the editor once.

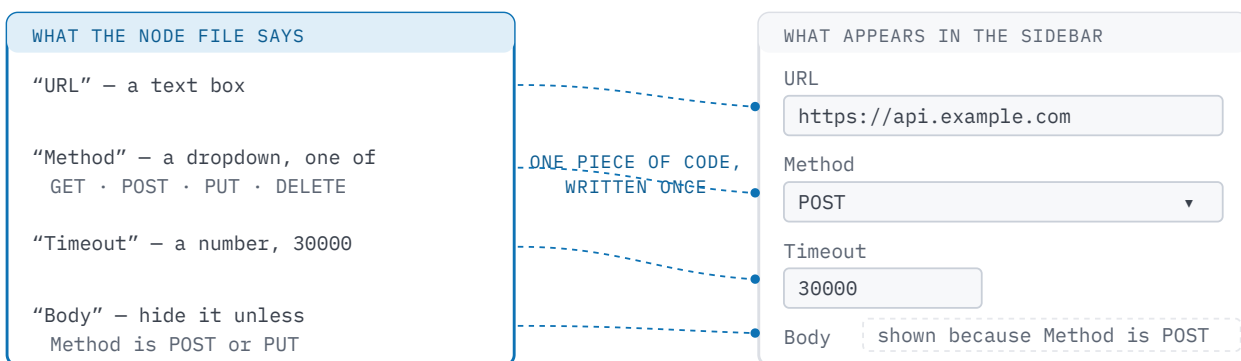


Figure 15. Four lines of description, four fields on screen — and the fourth appears only because Method happens to be POST. Nobody hand-builds a panel, ever.

AND ONE FIELD BECOMES A DIFFERENT VALUE FOR EACH ITEM

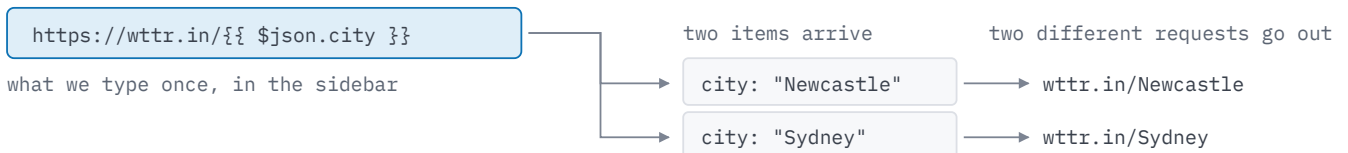


Figure 16. The engine fills the field in for every item separately, just before the node runs — the node never sees the braces.

FAMILIAR FROM N8N

Changing a node's **Operation** made half the fields disappear — that is the top figure. Dragging a field into a box inserted `{{ $json.something }}` — that is the bottom one.

TOOLS WE USE IN THIS PHASE

React	the one component that turns a description into a form	Our own code	no library does this for us — about a hundred lines, and the cleverest part of the project
Monaco	the editor from VS Code, dropped in later as the Code node's text box		

HOW WE PROCEED

1. **Agree the kinds of setting we support** — text, number, on/off, dropdown, code box — then write the single component that reads a description and produces those inputs.
2. **Add “only show this when...”**, so a panel changes as you pick an operation — this is what makes the Google nodes feel finished rather than overwhelming.
3. **Add expressions**, filled in for each item just before the node runs.

DONE WHEN

A brand-new node file appears in the sidebar with a full settings panel, and nobody touched the canvas code.

WATCH OUT

Skip this “to save time” and each of the next ten nodes needs a hand-built panel — turning Weeks 8–10 into a queue behind one person.

2.3b Build it: forms that draw themselves, and expressions

PROGRAM	VS Code — see §1.10 for the window, §1.11 for the loop
FILES	<code>packages/nodes/core/httpRequest.ts</code> , <code>packages/engine/expressions.js</code> , <code>apps/editor/src/ParameterPanel.tsx</code>
CREATE THEM	In the Explorer, right-click <code>packages/nodes/core</code> and <code>apps/editor/src</code> → <code>New File</code> → type the name with its extension → paste → <code>Ctrl S</code>

1 A node describes its own settings — all of them

This one description is simultaneously the form in the sidebar, the validation rules, and the documentation. Every field you add here appears in the UI with no UI work at all.


```

import { WorkflowError } from '../engine/errors.js';

export const httpRequestNode = {
  description: {
    name: 'httpRequest', displayName: 'HTTP Request', group: 'action',
    icon: 'globe', colour: '#0d72b4', inputs: ['main'], outputs: ['main'],
    properties: [
      { displayName: 'Method', name: 'method', type: 'options', default: 'GET', required: true,
        options: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE'].map(v => ({ name: v, value: v })) },

      { displayName: 'URL', name: 'url', type: 'string', default: '', required: true,
        placeholder: 'https://api.example.com/orders',
        validate: { pattern: '^https?:/', message: 'must start with http:// or https://' } },

      { displayName: 'Send headers', name: 'sendHeaders', type: 'boolean', default: false },
      { displayName: 'Headers', name: 'headers', type: 'json', default: '{}',
        displayOptions: { show: { sendHeaders: [true] } } },

      { displayName: 'Body', name: 'body', type: 'json', default: '{}',
        displayOptions: { show: { method: ['POST', 'PUT', 'PATCH'] } } }, // hidden otherwise

      { displayName: 'Timeout (ms)', name: 'timeout', type: 'number', default: 30000,
        validate: { min: 100, max: 120000 } },
      { displayName: 'Retries', name: 'retries', type: 'number', default: 3,
        validate: { min: 0, max: 5 } },
      { displayName: 'Response', name: 'responseFormat', type: 'options', default: 'auto',
        options: [{ name: 'Auto-detect', value: 'auto' }, { name: 'JSON', value: 'json' },
          { name: 'Text', value: 'text' } ] },
      { displayName: 'Continue on failure', name: 'continueOnFail', type: 'boolean',
        default: false, hint: 'Send the error down the branch instead of stopping the workflow' },
    ],
  },
};

async execute(ctx) {
  const out = [];
  const items = ctx.getInputData();

  for (let i = 0; i < items.length; i++) {
    const url = String(ctx.getNodeParameter('url', i));
    if (!/^https?:\/\//i.test(url))
      throw new WorkflowError(`URL must start with http:// or https:// - got "${url}"`,
        { code: 'BAD_PARAMETER' });

    const method = ctx.getNodeParameter('method', i, 'GET');
    const headers = ctx.getNodeParameter('sendHeaders', i, false)
      ? parseJsonParameter(ctx, 'headers', i) : {};
    const body = ['POST', 'PUT', 'PATCH'].includes(method)
      ? JSON.stringify(parseJsonParameter(ctx, 'body', i)) : undefined;

    try {
      const json = await ctx.helpers.httpRequest(url, {
        method, body,
        headers: { 'content-type': 'application/json', ...headers },
        timeoutMs: ctx.getNodeParameter('timeout', i, 30000),
        retries: ctx.getNodeParameter('retries', i, 3),
      });
    }
  }
}

```

```

    });
    out.push({ json, pairedItem: i }); // remember which input made this output
  } catch (err) {
    if (!ctx.getNodeParameter('continueOnFail', i, false)) throw err;
    out.push({ json: { error: err.message }, pairedItem: i });
  }
}
return [out];
},
};

// a JSON field the user typed is a string; a bad one must name the field, not throw "Unexpected token"
function parseJsonParameter(ctx, name, i) {
  const raw = ctx.getNodeParameter(name, i, '{}');
  if (typeof raw === 'object') return raw;
  try { return JSON.parse(raw); }
  catch { throw new WorkflowError(`The "${name}" field is not valid JSON`, { code: 'BAD_PARAMETER' }); }
}

```

pairedItem — the index of the input item that produced this output. Nobody needs it on day one, and everybody needs it the first time a workflow fans out to fifty items and one of them fails. Recording it costs one property.

2 Hand every description to the editor

apps/api/server.js — one more route

```

import { nodeTypes } from '../../packages/nodes/index.js';

app.get('/rest/node-types', (req, res) => {
  res.set('cache-control', 'public, max-age=60'); // it changes only when we deploy
  res.json(Object.values(nodeTypes).map(n => n.description));
});

```

3 One component turns any description into a form — and validates it

```
apps/editor/src/ParameterPanel.tsx
```

```

function isVisible(p, values) {
  const { show, hide } = p.displayOptions ?? {};
  const matches = (rules) => Object.entries(rules)
    .every(([field, allowed]) => (allowed as unknown[]).includes(values[field]));
  if (show && !matches(show)) return false;
  if (hide && matches(hide)) return false;
  return true;
}

export function validateParameter(p, value): string | null {
  const isExpression = typeof value === 'string' && value.includes('{{');
  if (isExpression) return null; // we cannot know the value until it runs
  if (p.required && (value === '' || value == null)) return `${p.displayName} is required`;
  const v = p.validate ?? {};
  if (v.pattern && !new RegExp(v.pattern).test(String(value ?? '')))
    return v.message ?? `${p.displayName} is not in the right format`;
  if (p.type === 'number') {
    if (!Number.isFinite(Number(value))) return `${p.displayName} must be a number`;
    if (v.min != null && Number(value) < v.min) return `${p.displayName} must be at least ${v.min}`;
    if (v.max != null && Number(value) > v.max) return `${p.displayName} must be at most ${v.max}`;
  }
  if (p.type === 'json') {
    try { JSON.parse(typeof value === 'string' ? value : JSON.stringify(value)); }
    catch { return `${p.displayName} is not valid JSON`; }
  }
  return null;
}

export function ParameterPanel({ properties, values, sampleItem, onChange, onValidity }) {
  const visible = properties.filter(p => isVisible(p, values));
  const errors = Object.fromEntries(visible
    .map(p => [p.name, validateParameter(p, values[p.name] ?? p.default)])
    .filter(([, e]) => e));

  useEffect(() => onValidity(Object.keys(errors).length === 0), [JSON.stringify(errors)]);

  return visible.map(p => {
    const v = values[p.name] ?? p.default;
    const set = (val) => onChange({ ...values, [p.name]: val });
    const common = { id: p.name, 'aria-invalid': !!errors[p.name], value: v ?? '' };

    return (
      <Field key={p.name} label={p.displayName} hint={p.hint}
        error={errors[p.name]} preview={previewOf(v, sampleItem)}>
        {p.type === 'string' && <input {...common} placeholder={p.placeholder}
          onChange={e => set(e.target.value)} />}
        {p.type === 'number' && <input {...common} type="number"
          onChange={e => set(e.target.value === '' ? '' : +e.target.value)} />}
        {p.type === 'boolean' && <input id={p.name} type="checkbox" checked={!!v}
          onChange={e => set(e.target.checked)} />}
        {p.type === 'json' && <textarea {...common} rows={4}
          onChange={e => set(e.target.value)} />}
        {p.type === 'options' && <select {...common} onChange={e => set(e.target.value)}>
          {p.options.map(o =>
            <option key={o.value} value={o.value}>{o.name}</option>)}
        </select>}
      </Field>
    )
  });
}

```

```

        </select>
    </Field>
    );
});
}

// show what an expression will actually become, using the last item that ran
function previewOf(value, sampleItem) {
    if (typeof value !== 'string' || !value.includes('{{') || !sampleItem) return null;
    try { return String(resolve(value, sampleItem, {})); }
    catch (e) { return `⚠️ ${e.message}`; }
}

```

Disable the Run button while `onValidity` is false. A workflow that cannot possibly work should never reach the engine — the user finds out in the sidebar, in red, in the field that is wrong.

4 Expressions, resolved per item — and unable to reach the server

packages/engine/expressions.js

```
import { WorkflowError } from './errors.js';

// names an expression must never see. They are passed in as parameters with no
// arguments, so inside the expression each one is simply 'undefined'.
const SHADOWED = ['process', 'require', 'module', 'exports', 'globalThis', 'global',
  'fetch', 'eval', 'Function', 'setTimeout', 'setInterval',
  'XMLHttpRequest', 'WebSocket', 'import'];

const cache = new Map(); // compiling is the slow part; do it once per string

function compile(expr) {
  if (cache.has(expr)) return cache.get(expr);
  if (expr.length > 2000) throw new WorkflowError('That expression is too long');
  let fn;
  try {
    fn = new Function('$json', '$node', '$now', '$itemIndex', ...SHADOWED,
      `"use strict"; return (${expr});`);
  } catch (e) {
    throw new WorkflowError(`Expression will not compile: ${e.message}`, { code: 'BAD_EXPRESSION' });
  }
  if (cache.size > 500) cache.clear();
  cache.set(expr, fn);
  return fn;
}

export function resolve(value, item, results, itemIndex = 0) {
  if (typeof value !== 'string' || !value.includes('{{}}')) return value;

  const evaluate = (expr) => {
    const $json = item?.json ?? {};
    const $node = new Proxy({}, {
      get: (_, name) => {
        const branches = results[name];
        if (!branches) throw new WorkflowError(`$node["${String(name)}"] has not run yet`,
          { code: 'BAD_EXPRESSION' });
        return { json: branches[0]?.[0]?.json ?? {} };
      },
    });
    try {
      return compile(expr.trim())($json, $node, new Date(), itemIndex);
    } catch (e) {
      if (e instanceof WorkflowError) throw e;
      throw new WorkflowError(`${expr.trim()} -> ${e.message}`, { code: 'BAD_EXPRESSION' });
    }
  };

  const whole = value.match(/^(s*\{([s\S]+)\}\s*$/); // the field is ONE expression
  if (whole) return evaluate(whole[1]); // ...so keep the real type
  return value.replace(/\{([s\S]+)\}/g, (_, e) => String(evaluate(e) ?? ''));
}
```

Now wire it into the engine. This is the whole change to \$2.1b — three lines, and from this moment every setting of every node supports expressions, forever:

packages/engine/engine.js – inside the ctx object

```
getNodeParameter(name, i = 0, fallback = undefined) {  
  const raw = node.parameters?.[name] ?? fallback;  
  if (raw === undefined)  
    throw new WorkflowError(`${nodeName} needs its "${name}" setting filled in`,  
      { node: nodeName, code: 'MISSING_PARAMETER' });  
  return resolve(raw, items[i], results, i); // <- the only new line  
},
```

Node authors never see braces. Say plainly in the report that shadowing is a speed bump, not a sandbox — the real isolation story is the Code node in §2.6c.

WHAT YOU SHOULD SEE

METHOD = GET

HTTP Request

METHOD
GET

URL
https://wttr.in/...

SEND HEADERS
☐

TIMEOUT (MS)
30000

No Body field – GET has no body. Nobody hid it.

METHOD = POST

HTTP Request

METHOD
POST

URL
https://api.example.com/...

BODY (JSON)
{"city": "{{ \$json.city }}"}

preview, last item:
{"city": "Newcastle"}

Shown because Method = POST.

A FIELD IS WRONG

HTTP Request

URL
wttr.in/Newcastle
must start with https://

TIMEOUT (MS)
999999
must be at most 120000

Run (disabled)

Two red fields, one disabled button. The engine never ran.

Figure 17. The same component, three times. Nobody wrote a form: each panel is the node's `properties` array rendered, validated and previewed.

Middle panel, bottom half, is the feature to demo. Typing `{{ $json.city }}` shows what it will become *before* you run anything — that is what makes expressions feel safe to a first-time user.

HOW THIS PART CONNECTS TO THE OTHERS

§2.2b — /rest/node-types
every description, as JSON

properties[]

§2.3b — schema-driven UI
one panel · expressions

parameters{{

§2.6b / §2.6c — six new nodes
written by six people at once

RECEIVES

the `description` of every registered node. Add a file to `packages/nodes`, restart, refresh — the form exists. **This is the single thing that makes the Week 8 node surge possible.**

HANDS ON

a filled `parameters` object into the workflow JSON of §2.2b, saved by the canvas of §2.2c.

CHANGES THE ENGINE

by exactly one line: `getNodeParameter` now calls `resolve()`. From here, every node in §2.6b and §2.6c gets expressions for free without knowing they exist.

SEAM TO AGREE

the property `type` vocabulary — `string · number · boolean · options · json`. A node author who invents a seventh type gets a blank field, so add it to the panel first.

CHECK IT WORKS

Add a new node file, restart the API, refresh the editor — its whole settings panel is there, and switching Method to POST makes Body appear. Nobody touched the editor. Then type `ftp://x` into URL and confirm the field turns red and Run greys out; type `{{ $json.nothing.here }}` and confirm the preview shows a warning instead of crashing the panel.

PHASE 4 · MAKE IT START BY ITSELF

STAGES 7-8 · WEEK 7 · LEAD: C2

2.4 Workflows that run without anyone pressing a button

WHO BUILDS THIS PART

C2

C1 reviews

WHAT YOU NEED TO KNOW

HTTP request and response, in detail · **cron** syntax · timezones · comfort with networking that fails for boring reasons (ports, tunnels, firewalls).

What we are doing. Building the two **triggers** from section 0.1. One is a **web address** that anything on the internet can call; the other is a **clock** inside our own server. These are the two the assignment names, and together they are what turns a tool into a platform.

webhook

What it is. A web address that belongs to one workflow. Anything that can send a request can start it.

What it does in this section. The server looks the address up, then starts the workflow with the request as its first item.

How it connects. Each workflow gets two — a test one while editing and a live one once switched on — so experiments never send real email.

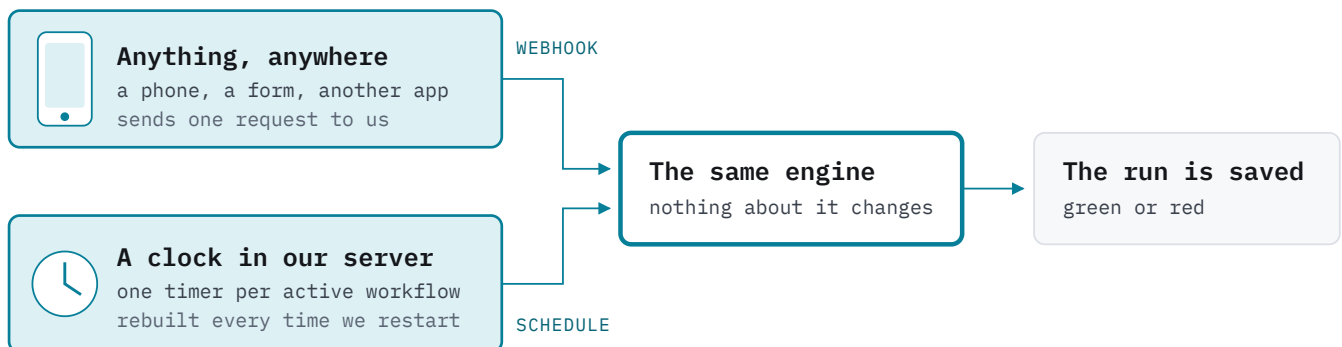
cron

What it is. A five-field way of writing a repeating time. * * * * *

* means every minute.

What it does in this section. It is what a Schedule trigger stores, alongside a timezone.

How it connects. The scheduler keeps one timer per active workflow and rebuilds them all on boot — otherwise schedules stop silently after a deploy.



Each workflow gets two addresses, exactly like n8n: a test one while we are editing it, and the real one once it has been switched on.

Figure 18. Both routes arrive at the same loop. Adding a trigger never means changing the engine — that is the payoff for getting Phase 1 right.

FAMILIAR FROM N8N

The Webhook node in n8n showed you two URLs, a Test one and a Production one, and made you press “Listen for test event” before the test one would work. That is precisely the mechanism on this page, and we are copying the two-URL idea because it stops us sending real emails while we are still experimenting.

TOOLS WE USE IN THIS PHASE

croner

one small library that runs a function on a schedule, with timezones handled

Postman or curl

to fire test requests at our own addresses without a real caller

cloudflared

a free tunnel giving our laptop a public address while we develop

HOW WE PROCEED

1. **Give each workflow its own web address.** When a request arrives we look up which workflow owns that address, and start it with the request handed in as the first item.
2. **Two addresses per workflow** — a test one while editing, a live one once activated.
3. **Decide what the caller gets back:** an instant “received”, or wait for the workflow to finish and return its result. Both are useful, and both are easy once the first works.

4. **For schedules, keep one timer per active workflow**, and rebuild the whole set whenever the server restarts — otherwise schedules stop silently after every deploy.
5. **Always store a timezone**. Our laptops are on Australian time and the server will be on UTC, so a workflow set for 9am would otherwise fire in the evening.

DONE WHEN

A request sent from someone's phone runs a workflow on the laptop, and a scheduled workflow produces exactly one run per minute — stopping the moment it is switched off.

WATCH OUT

To test from outside while developing, use a tunnel rather than trying to open ports on the university network. It is one command and saves an afternoon.

2.4b Build it: webhooks and schedules

PROGRAM VS Code — see §1.10 for the window, §1.11 for the loop

FILES apps/api/webhooks.js, apps/api/schedules.js, apps/api/runner.js

CREATE THEM In the Explorer, right-click apps/api → New File → type the name with its extension → paste → **Ctrl S**

1 Two more tables, for the addresses and the timers in use

schema.sql — additions

```
CREATE TABLE IF NOT EXISTS webhooks (
  path          text NOT NULL CHECK (path ~ '^[a-z0-9][a-z0-9-]{0,62}$'),
  method        text NOT NULL CHECK (method IN ('GET', 'POST', 'PUT', 'PATCH', 'DELETE')),
  workflow_id   uuid NOT NULL REFERENCES workflows(id) ON DELETE CASCADE,
  node_name     text NOT NULL,
  is_test       boolean NOT NULL DEFAULT false,
  response_mode text NOT NULL DEFAULT 'lastNode'
                      CHECK (response_mode IN ('immediately', 'lastNode')),
  secret        bytea,                      -- optional shared secret, encrypted
  PRIMARY KEY (path, method, is_test)      -- two workflows cannot claim one address
);

CREATE TABLE IF NOT EXISTS schedules (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  workflow_id   uuid NOT NULL UNIQUE REFERENCES workflows(id) ON DELETE CASCADE,
  cron          text NOT NULL,
  timezone      text NOT NULL DEFAULT 'Australia/Sydney'
);

-- a webhook delivered twice must not do the work twice
CREATE TABLE IF NOT EXISTS webhook_deliveries (
  idempotency_key text PRIMARY KEY,
  execution_id    uuid NOT NULL REFERENCES executions(id) ON DELETE CASCADE,
  received_at     timestampz NOT NULL DEFAULT now()
);
```

Idempotency — "doing it twice has the same effect as doing it once". Stripe, GitHub and Shopify all retry a webhook they think failed. Without this table a retried order becomes two rows in the spreadsheet and two emails to the customer.

2 One runner, so every trigger starts a workflow the same way

apps/api/runner.js

```
import { db } from './db.js';
import { runWorkflow } from '../../packages/engine/engine.js';
import { WorkflowError } from '../../packages/engine/errors.js';
import { nodeTypes } from '../../packages/nodes/index.js';

const MAX_CONCURRENT = 5;
let inFlight = 0;
const queue = [];

// a tiny queue: five workflows at once, the rest wait their turn. Without it, one
// popular webhook opens 200 database clients and the whole server stops answering.
function withSlot(fn) {
  return new Promise((resolve, reject) => {
    const start = async () => {
      inFlight++;
      try { resolve(await fn()); } catch (e) { reject(e); }
      finally { inFlight--; queue.shift()?.(); }
    };
    inFlight < MAX_CONCURRENT ? start() : queue.push(start);
  });
}

export async function runById(workflowId, triggerItems, mode = 'manual') {
  return withSlot(async () => {
    const { rows: [wf] } = await db.query('SELECT * FROM workflows WHERE id = $1', [workflowId]);
    if (!wf) throw new WorkflowError('That workflow no longer exists', { code: 'NOT_FOUND' });

    const { rows: [run] } = await db.query(
      "INSERT INTO executions (workflow_id, status, mode) VALUES ($1,'running',$2) RETURNING id",
      [workflowId, mode]);

    try {
      const { results, log } = await runWorkflow(wf, nodeTypes, { triggerItems });
      await db.query("UPDATE executions SET status='success', data=$1, log=$2, "
        + "finished_at=now() WHERE id=$3",
        [JSON.stringify(results), JSON.stringify(log), run.id]);
      return { executionId: run.id, results, log };
    } catch (err) {
      const payload = err instanceof WorkflowError ? err.toJSON() : { message: err.message };
      await db.query("UPDATE executions SET status='error', error=$1, finished_at=now() WHERE id=$2",
        [JSON.stringify(payload), run.id]);
      throw Object.assign(err, { executionId: run.id });
    }
  });
}

// anything still 'running' when the server died is a lie - clean it up on boot
export async function reapStuckExecutions() {
  const { rowCount } = await db.query(
    "UPDATE executions SET status='cancelled', finished_at=now() WHERE status='running'");
  if (rowCount) console.log(`marked ${rowCount} interrupted execution(s) as cancelled`);
}
```

3 The webhook route — the one endpoint strangers can reach

Everything else in this project is behind our own editor. This is not. Treat every line of it as hostile input, because it is.


```

import crypto from 'node:crypto';
import express from 'express';
import rateLimit from 'express-rate-limit';
import { db } from './db.js';
import { decrypt } from './crypto.js';
import { runById } from './runner.js';

// keep the raw bytes: a signature is computed over the exact body, not over re-encoded JSON
const rawJson = express.json({
  limit: '1mb',
  verify: (req, _res, buf) => { req.rawBody = buf; },
});

const hookLimit = rateLimit({ windowMs: 60_000, limit: 120,
  message: { message: 'Too many requests' } });

export function mountWebhooks(app) {
  app.all('/webhook/:path', hookLimit, rawJson, async (req, res) => {
    try {
      const isTest = req.query.test === '1';
      const { rows: [hook] } = await db.query(
        'SELECT * FROM webhooks WHERE path = $1 AND method = $2 AND is_test = $3',
        [req.params.path, req.method, isTest]);

      if (!hook) return res.status(404).json({ message: 'No workflow is listening here' });

      // optional shared secret, compared in constant time so timing cannot leak it
      if (hook.secret) {
        const expected = crypto.createHmac('sha256', decrypt(hook.secret))
          .update(req.rawBody ?? Buffer.alloc(0)).digest('hex');
        const given = String(req.get('x-signature') ?? '');
        const ok = given.length === expected.length
          && crypto.timingSafeEqual(Buffer.from(given), Buffer.from(expected));
        if (!ok) return res.status(401).json({ message: 'Bad or missing signature' });
      }

      // deduplicate retries from the sender
      const key = req.get('idempotency-key') || req.get('x-github-delivery');
      if (key) {
        const { rows } = await db.query(
          'SELECT execution_id FROM webhook_deliveries WHERE idempotency_key = $1', [key]);
        if (rows[0]) return res.status(200).json({ message: 'Already handled',
          executionId: rows[0].execution_id });
      }

      const triggerItems = [{ json: {
        headers: req.headers, query: req.query, body: req.body,
        method: req.method, receivedAt: new Date().toISOString(),
      } }];

      if (hook.response_mode === 'immediately') {
        res.status(202).json({ message: 'Workflow was started' }); // answer in ~5 ms
        runById(hook.workflow_id, triggerItems, 'webhook')
          .then(r => key && remember(key, r.executionId))
          .catch(err => console.error('[webhook]', err.message)); // never crash the process
      }
    }
  });
}

```

```

        return;
    }

    const result = await runById(hook.workflow_id, triggerItems, 'webhook');
    if (key) await remember(key, result.executionId);
    res.json(lastNodeOutput(result.results));

    } catch (err) {
        console.error('[webhook]', err);
        res.status(500).json({ message: 'The workflow failed', executionId: err.executionId ?? null });
    }
});
}

const remember = (key, executionId) => db.query(
    'INSERT INTO webhook_deliveries (idempotency_key, execution_id) VALUES ($1,$2) '
    + 'ON CONFLICT DO NOTHING', [key, executionId]);

```

The test address is the same handler with `?test=1`, so experiments never fire real emails.

4 The schedule registry — with the three guards a cron always needs

```
$ npm i croner --workspace apps/api
```

```

import { Cron } from 'croner';
import { db } from './db.js';
import { runById } from './runner.js';

const jobs = new Map();

export function validateSchedule({ cron, timezone }) {
  try { Intl.DateTimeFormat(undefined, { timeZone: timezone }); }
  catch { throw new Error(`"${timezone}" is not a time zone`); }
  const probe = new Cron(cron, { timezone, paused: true });
  if (!probe.nextRun()) throw new Error(`"${cron}" will never run`);
  return probe.nextRun();
}

export function activate(workflow, schedule) {
  deactivate(workflow.id);
  validateSchedule(schedule);
  const job = new Cron(schedule.cron, {
    timezone: schedule.timezone,
    protect: true, // 1 · if the last run is still going, SKIP this tick – never pile up
    catch: (err) => console.error(`[schedule ${workflow.name}]`, err.message), // 2 · never crash
    maxRuns: Infinity,
  }, () => runById(workflow.id, [{ json: {
    timestamp: new Date().toISOString(),
    scheduledFor: job.currentRun()?.toISOString() ?? null,
  } }], 'schedule'));
  jobs.set(workflow.id, job);
  console.log(`[schedule] ${workflow.name} → next ${job.nextRun()?.toISOString()}`);
}

export function deactivate(workflowId) { // 3 · always stop the old timer first
  jobs.get(workflowId)?.stop();
  jobs.delete(workflowId);
}

// rebuild every timer on boot – otherwise schedules stop silently after each deploy
export async function restoreAll() {
  const { rows } = await db.query(
    'SELECT w.*, s.cron, s.timezone FROM workflows w JOIN schedules s ON s.workflow_id = w.id '
    + 'WHERE w.active = true');
  for (const r of rows) {
    try { activate(r, r); }
    catch (e) { console.error(`[schedule] ${r.name} is broken: ${e.message}`); }
  }
  console.log(`restored ${rows.length} schedule(s)`);
}

export const listSchedules = () => [...jobs.entries()].map(([id, job]) =>
  ({ workflowId: id, next: job.nextRun()?.toISOString(), running: job.isBusy() }));

```

protect: true — the single most valuable option on this page. A workflow set to every minute that takes ninety seconds will, without it, start a second copy before the first finishes, then a third, until the machine dies. With it, the tick is skipped and logged.

5 Reach your laptop from outside

```
$ npx cloudflared tunnel --url http://localhost:5678
# prints https://something-random.trycloudflare.com — use that in Postman,
# or paste it into a phone browser to fire the webhook from another network
```

WHAT YOU SHOULD SEE

Terminal — the API log

```
api on http://localhost:5678
marked 2 interrupted execution(s) as cancelled
[schedule] Daily digest → next 2026-03-14T09:00:00Z
restored 1 schedule(s)
POST /webhook/new-order 202 4 ms
POST /webhook/new-order 200 0 ms (duplicate key)
POST /webhook/new-order 401 Bad or missing signature
POST /webhook/nope 404 No workflow listening
[schedule Daily digest] skipped — previous run still going
```

Terminal — firing it from your phone

```
$ curl -i -X POST \
  https://tidy-owl.trycloudflare.com/webhook/new-order \
  -H 'content-type: application/json' \
  -H 'idempotency-key: order-1042' \
  -d '{"customer": "Alice", "total": 42}'

HTTP/1.1 200 OK
{
  "status": "ok",
  "sheetRow": 18,
  "executionId": "7c2e...9b"
}
```

localhost:5173/workflow/3f9a...c1/executions

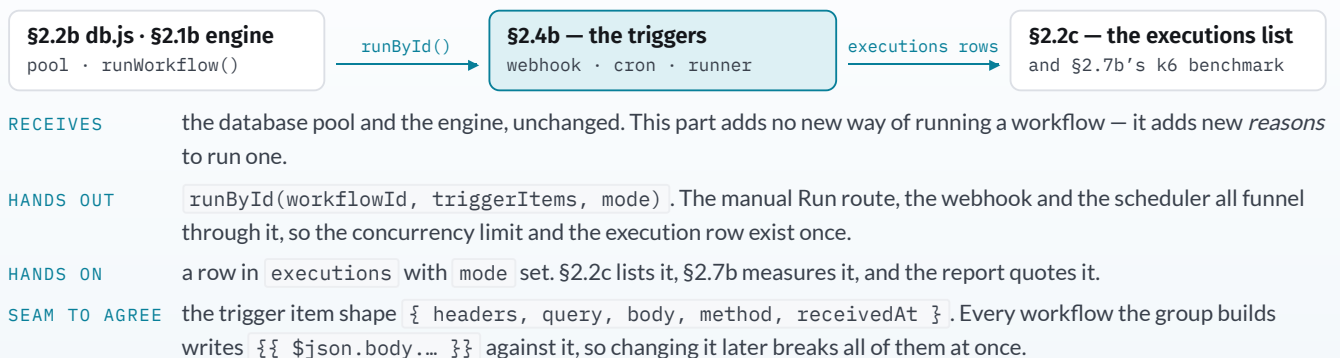
STATUS	STARTED	MODE	TOOK	TRIGGERED BY
Success	14:32:07	webhook	612 ms	order-1042
Success	14:31:00	schedule	1.4 s	* * * * *
Error	14:30:02	webhook	31 s	order-1041

Sheets: HTTP 429 after 3 retries — node "Append row"

Figure 19. The executions table after ten minutes of Phase 4. Mode says which trigger started it, and a failed row carries the engine's message with the node's name in it.

The red row is the one to demonstrate. It proves three separate pieces of Phase 1 and 4 are wired together: the HTTP helper retried, the engine attached the node name, and the runner wrote a real `error` column instead of losing it in a console somewhere.

HOW THIS PART CONNECTS TO THE OTHERS



CHECK IT WORKS

A POST from your phone through the tunnel creates an execution row within a second. Send the exact same request again with the same `idempotency-key` — you get `Already handled` and **no** second row. A workflow on `* * * * *` makes exactly one row per minute and stops the moment you deactivate it. Then make a workflow that sleeps 90 seconds, schedule it every minute, and confirm the log says *skipped* instead of the machine getting slower and slower.

PHASE 5 · MAKE IT DO THINGS

STAGE 9 · WEEKS 7-8 · LEAD: C2

2.5 First, permission to use a Google account

WHO BUILDS THIS PART

C2

D2 keeps the runbook

WHAT YOU NEED TO KNOW

Reading **API documentation** carefully · the idea of **OAuth** (this page teaches it) · Node's built-in **crypto** · genuine care with secrets — this is the one place a mistake is expensive.

What we are doing. Getting permission to act for somebody — **without ever seeing their password**. The step most likely to cost us a week, so we do it early and once.

OAuth 2.0

What it is. The agreed way of saying “let this app act for me” without handing over a password.

What it does in this section. The seven arrows in Figure 13 are the whole of it.

How it connects. Every Google node depends on it, which is why this is the first thing built in Phase 5 and the earliest task in Week 4.

token

What it is. A temporary key a service gives you after login, used instead of a password.

What it does in this section. Google returns two: one lasting an hour, one lasting much longer.

How it connects. The long one is scrambled and stored; the shared helper in the next section quietly swaps it for a fresh short one whenever needed.

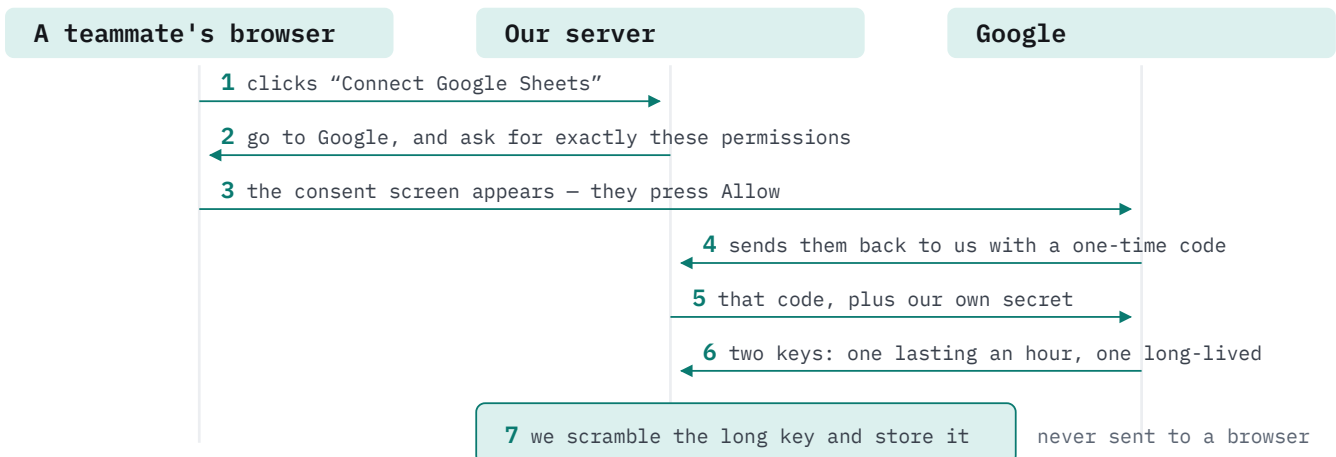


Figure 20. The whole thing in seven arrows. Steps 1–3 happen in front of the person; 4–7 happen invisibly in about half a second.

FAMILIAR FROM N8N

Adding a Google credential opened a small window, showed Google's “choose an account” screen, then said *Account connected*. These seven arrows are what happened inside it.

TOOLS WE USE IN THIS PHASE

Google Cloud Console

where the project, the four APIs and the permission screen live — free

node:crypto

built into Node; scrambles the stored keys

OAuth 2.0

the agreed way of saying “let this app act for me” — a sequence of requests, not a library

HOW WE PROCEED

- 1. One of us creates a Google Cloud project** and switches on the four APIs. Free, fifteen minutes.
- 2. Fill in the consent screen** and keep the app in **Testing** mode. Add every group member as a **test user**, plus our demonstrator: in Testing mode nobody outside that list can connect at all.
- 3. Build the round trip** from the figure above, then scramble the long-lived key before storing it — and never send it to a browser.

THE TWO TRAPS THAT END DEMOS

1 • Gmail permissions are “restricted”.

Publishing the app would need a Google security review we cannot finish this semester — and we do not need one: Testing mode plus a test-user list behaves identically for us.

2 • In Testing mode the long-lived key dies after seven days.

An account connected in Week 9 is dead by Week 14, and it looks like a broken node. **Reconnecting every account is step one of the demo-morning checklist.**

DONE WHEN

Somebody presses Connect, approves on Google's screen, and our server can read a real spreadsheet with the stored keys.

2.5b Build it: the Google permission round trip

PROGRAM VS Code — see §1.10 for the window, §1.11 for the loop

FILES apps/api/oauth.js, apps/api/crypto.js, schema.sql

CREATE THEM In the Explorer, right-click apps/api → New File → type the name with its extension → paste → **Ctrl S**

1 Create the project and switch on the four APIs

- 1 Go to console.cloud.google.com and sign in with the account that will own this
- 2 Click the **project dropdown** in the top bar → **NEW PROJECT** → name it **ai-automation** → **CREATE**
- 3 Wait for the notification, then use the same dropdown to **select the new project**. Everything after this happens inside it — this is the step people miss.
- 4 In the search bar type **Google Sheets API** → open it → **ENABLE**
- 5 Repeat step 4 for **Google Drive API**, **Google Docs API** and **Gmail API**. Four separate enables.

2 The consent screen, and the test-user list that makes it work

- 1 Left menu → **APIs & Services** → **OAuth consent screen**
- 2 Choose **External** → **CREATE**. Leave the publishing status as **Testing** — we never publish.
- 3 App name **AI Automation Platform**, your email for both support and developer contact → **SAVE AND CONTINUE**
- 4 **ADD OR REMOVE SCOPES** → filter and tick **.../auth/spreadsheets**, **.../auth/drive.file**, **.../auth/documents**, **.../auth/gmail.send** → **UPDATE** → **SAVE AND CONTINUE**
- 5 **Test users** → **ADD USERS** → paste all eight Gmail addresses and the demonstrator's → **SAVE AND CONTINUE**. Anybody not on this list simply cannot connect.

3 The client ID, and the exact redirect address

- 1 **APIs & Services** → **Credentials** → **CREATE CREDENTIALS** → **OAuth client ID**
- 2 Application type **Web application**, name it **server**
- 3 Under **Authorised redirect URIs** click **ADD URI** twice and paste both addresses below — character for character, including the trailing path
- 4 **CREATE**, then copy the **Client ID** and **Client secret** straight into **.env**. The secret is shown once.

```
http://localhost:5678/rest/oauth2-credential/callback
https://your-domain.com/rest/oauth2-credential/callback  # add the live one now too
```

4 Where the keys live

schema.sql — additions

```
CREATE TABLE IF NOT EXISTS credentials (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name        text NOT NULL,
  type        text NOT NULL,           -- googleSheets | googleDrive | googleDocs | gmail
  data        jsonb NOT NULL,          -- the ENCRYPTED envelope, never plain tokens
  scopes      text[] NOT NULL DEFAULT '{}',
  expires_at  timestampz,              -- when the short-lived access token dies
  created_at  timestampz NOT NULL DEFAULT now()
);

-- a one-use, short-lived note proving the browser that comes back is the one we sent

CREATE TABLE IF NOT EXISTS oauth_states (
  state        text PRIMARY KEY,
  credential_id uuid NOT NULL REFERENCES credentials(id) ON DELETE CASCADE,
  expires_at    timestampz NOT NULL
);
```

5 Encryption you can trust, in twenty-five lines

apps/api/crypto.js

```
import crypto from 'node:crypto';

const raw = process.env.ENCRYPTION_KEY ?? '';
if (!/^[0-9a-f]{64}$/i.test(raw)) {
  console.error('ENCRYPTION_KEY must be exactly 64 hex characters (32 bytes).');
  console.error("Generate one: node -e `\"console.log(crypto.randomBytes(32).toString('hex'))\"`");
  process.exit(1); // fail at boot, not at 2 a.m. during the demo
}
const key = Buffer.from(raw, 'hex');

// version the envelope: if we ever change algorithm, old rows still decrypt
export function encrypt(text) {
  const iv = crypto.randomBytes(12);
  const c = crypto.createCipheriv('aes-256-gcm', key, iv);
  const data = Buffer.concat([c.update(String(text), 'utf8'), c.final()]);
  return { v: 1, iv: iv.toString('base64'), tag: c.getAuthTag().toString('base64'),
    data: data.toString('base64') };
}

export function decrypt(envelope) {
  if (!envelope || envelope.v !== 1) throw new Error('Unknown credential format');
  const d = crypto.createDecipheriv('aes-256-gcm', key, Buffer.from(envelope.iv, 'base64'));
  d.setAuthTag(Buffer.from(envelope.tag, 'base64'));
  try {
    return Buffer.concat([d.update(Buffer.from(envelope.data, 'base64')), d.final()]).toString('utf8');
  } catch {
    throw new Error('Credential could not be decrypted – has ENCRYPTION_KEY changed?');
  }
}
```

GCM — the "authenticated" mode of AES. It does not only hide the token, it detects tampering: change one byte in the database and `final()` throws instead of quietly returning rubbish. That is the whole reason for the `tag` field.

6 Send the user to Google — with a state nobody can forge

apps/api/oauth.js

```
import crypto from 'node:crypto';
import { db } from './db.js';
import { encrypt, decrypt } from './crypto.js';

const SCOPES = {
  googleSheets: ['https://www.googleapis.com/auth/spreadsheets'],
  googleDrive: ['https://www.googleapis.com/auth/drive.file'],
  googleDocs: ['https://www.googleapis.com/auth/documents'],
  gmail: ['https://www.googleapis.com/auth/gmail.send'],
};

const REDIRECT = () => `${process.env.BASE_URL}/rest/oauth2-credential/callback`;

app.get('/rest/oauth2-credential/auth', route(async (req, res) => {
  const { type, credentialId } = req.query;
  if (!SCOPES[type]) throw new HttpError(400, `Unknown credential type "${type}"`);

  // one-use random state, stored with a 10-minute life. This is the CSRF defence:
  // a callback carrying a state we did not issue is thrown away.
  const state = crypto.randomBytes(24).toString('base64url');
  await db.query("INSERT INTO oauth_states (state, credential_id, expires_at) "
    + "VALUES ($1,$2, now() + interval '10 minutes')", [state, credentialId]);

  res.redirect('https://accounts.google.com/o/oauth2/v2/auth?' + new URLSearchParams({
    client_id: process.env.GOOGLE_CLIENT_ID,
    redirect_uri: REDIRECT(),
    response_type: 'code',
    scope: SCOPES[type].join(' '),
    access_type: 'offline', // REQUIRED, or no long-lived key comes back
    prompt: 'consent', // REQUIRED when re-authorising
    include_granted_scopes: 'true',
    state,
  }));
}));
```

7 Trade the code for keys, and store them scrambled

apps/api/oauth.js – continued

```
app.get('/rest/oauth2-credential/callback', route(async (req, res) => {
  // the user pressed Cancel, or Google refused – say so in words, not a stack trace
  if (req.query.error)
    return res.status(400).send(page('Not connected',
      req.query.error === 'access_denied'
        ? 'You pressed Cancel on Google's screen. Nothing was saved.'
        : `Google said: ${req.query.error}`));

  const { rows: [note] } = await db.query(
    'DELETE FROM oauth_states WHERE state = $1 AND expires_at > now() RETURNING credential_id',
    [String(req.query.state ?? '')]); // DELETE...RETURNING makes it one-use, atomically
  if (!note) return res.status(400).send(page('Not connected',
    'This link has expired or was not started here. Press Connect again.'));

  const r = await fetch('https://oauth2.googleapis.com/token', {
    method: 'POST',
    headers: { 'content-type': 'application/x-www-form-urlencoded' },
    signal: AbortSignal.timeout(15_000),
    body: new URLSearchParams({
      code: String(req.query.code ?? ''),
      client_id: process.env.GOOGLE_CLIENT_ID,
      client_secret: process.env.GOOGLE_CLIENT_SECRET,
      redirect_uri: REDIRECT(),
      grant_type: 'authorization_code',
    }),
  });
  const tokens = await r.json(); // { access_token, refresh_token, expires_in, scope }

  if (!r.ok) return res.status(502).send(page('Google refused',
    `${tokens.error}: ${tokens.error_description ?? ''}`));

  // the single most common Phase 5 bug, caught here instead of in three days' time
  if (!tokens.refresh_token) return res.status(400).send(page('Half-connected',
    'Google did not send a refresh token. Remove this app at myaccount.google.com/permissions '
    + 'and press Connect again – access_type=offline only returns one on a fresh consent.'));

  await db.query('UPDATE credentials SET data = $1, scopes = $2, expires_at = $3 WHERE id = $4',
    [encrypt(JSON.stringify(tokens)), (tokens.scope ?? '').split(' '),
      new Date(Date.now() + tokens.expires_in * 1000), note.credential_id]);

  res.send(page('Connected', 'You can close this window.'));
}));

const page = (title, body) =>
  `<!doctype html><meta charset="utf-8"><title>${title}</title>
  <body style="font:16px system-ui;padding:3rem;max-width:34rem">
  <h1 style="font-size:1.3rem">${title}</h1><p>${body}</p></body>`;
```

```
$ node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
# put the result in .env as ENCRYPTION_KEY – never commit it
```

Testing-mode refresh tokens expire after seven days. Put a reminder in the group chat for the morning of the demo: press Connect once on each credential. It takes forty seconds and it has ended more student demos than any bug.

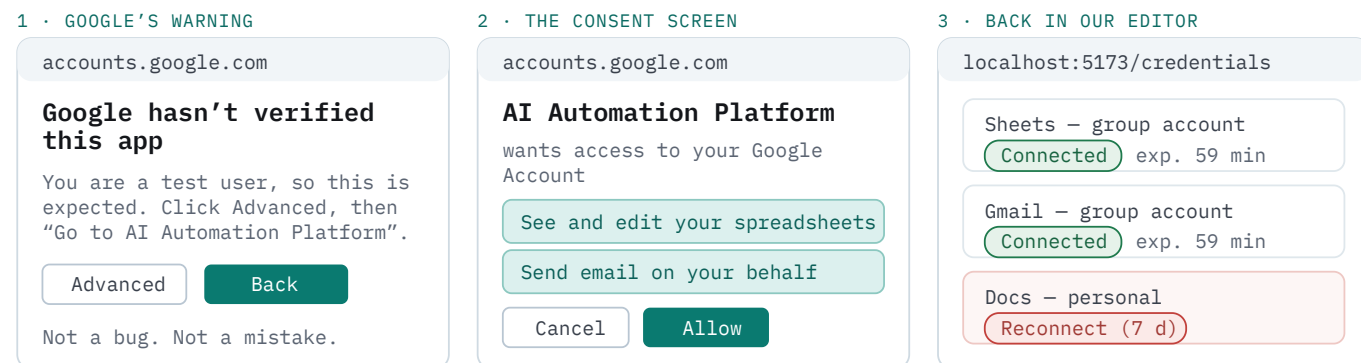
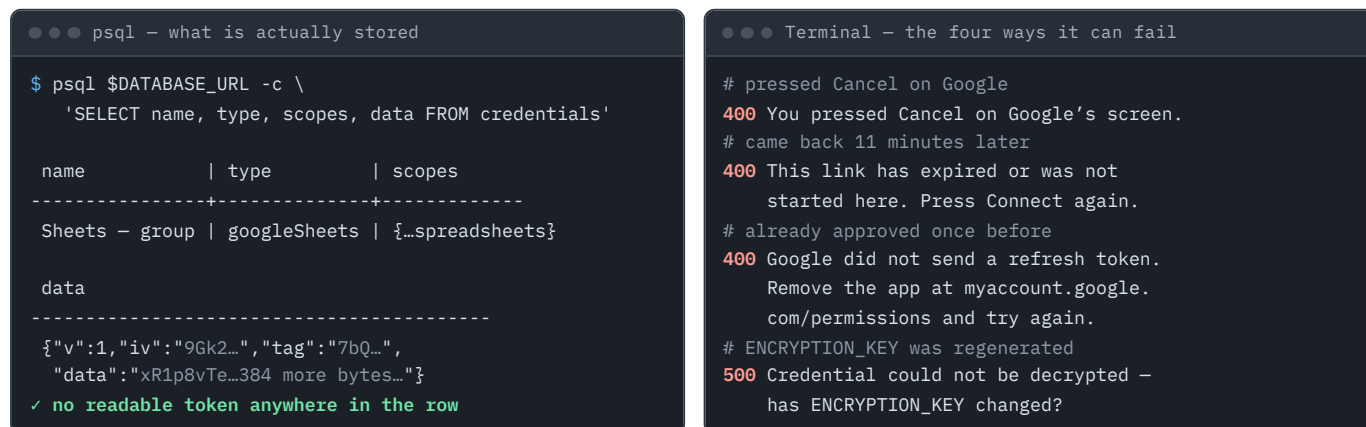
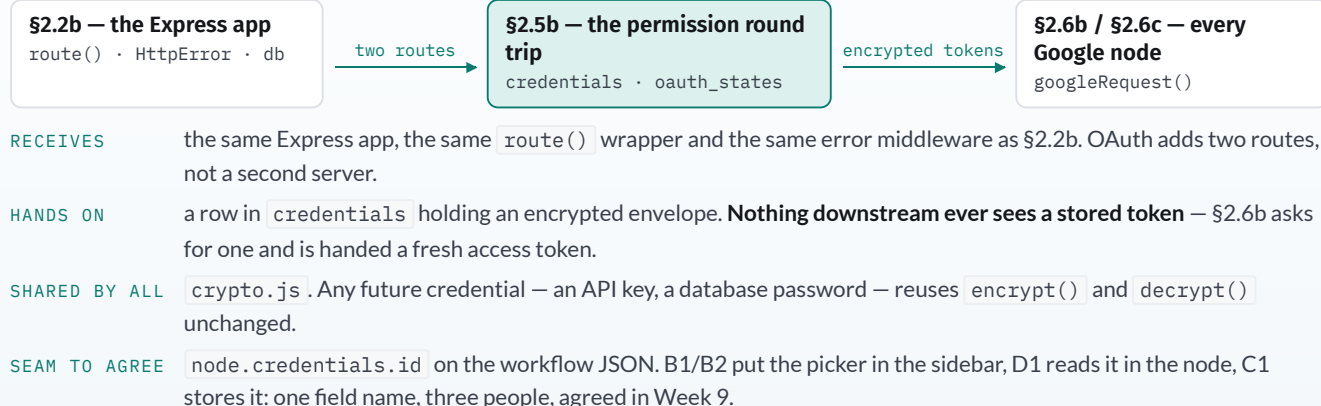


Figure 21. The whole round trip, as the user sees it. Panel 1 always appears while the app is in Testing mode — expect it, and tell your demonstrator to expect it too.



HOW THIS PART CONNECTS TO THE OTHERS



CHECK IT WORKS

Press Connect, approve on Google's screen, and the credential row appears with unreadable bytes in it. Then deliberately reproduce all four failures on the right — especially the missing refresh token, because it *will* happen to somebody in your group in Week 9 and the message above is what saves them an afternoon.

2.6 Then the nodes people actually came for

WHO BUILDS THIS PART

D1 leads

A1 A2 B2 C1 C2 build

one node each

WHAT YOU NEED TO KNOW

Reading **Google's API reference** · `fetch` and JSON shaping · for Gmail, patience with **MIME** · for the Code node, curiosity about how sandboxes fail.

What we are doing. Building the four Google nodes, the Code node and the AI node. With Phases 1–4 finished, each of these is a small, self-contained job — which is exactly why four of us can now work at the same time without getting in each other's way.

sandbox

What it is. A restricted place to run code you do not trust, where it cannot reach the rest of the program.

What it does in this section. Ours is a separate thread we can stop after five seconds, running the user's JavaScript in an isolated context.

How it connects. It is what makes the Code node safe enough for us — and being able to explain its limits is worth real marks in the report.

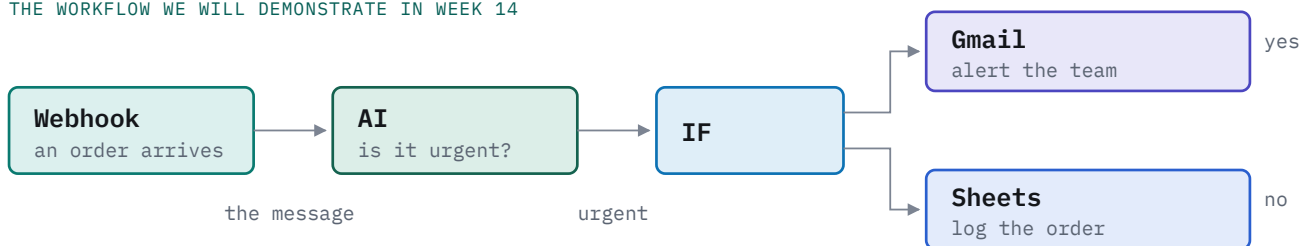
MIME

What it is. The old, strict format an email is written in before it is sent.

What it does in this section. Gmail wants the whole message as one encoded string, so we build the headers and parts by hand.

How it connects. It is the only genuinely fiddly part of Phase 5 — budget a day for it and it stops being a surprise.

THE WORKFLOW WE WILL DEMONSTRATE IN WEEK 14



Six nodes, five different phases, one screen. Everything in this document exists to make this work.

Figure 22. Each box is coloured by the phase that built it — which is also a picture of how the whole project fits together.

FAMILIAR FROM N8N

This is your weather workflow, grown up. Same shape: something starts it, something thinks about the data, something decides, and something delivers. The only new idea is the branch — and n8n's IF node worked exactly the same way.

HOW WE PROCEED

1. **Build the Google nodes easiest first: Sheets, then Docs, then Drive, then Gmail.** Each is a settings description plus one carefully built request. A single shared helper deals with expired keys, so no individual node has to think about it.
2. **Allow a full day for Gmail.** Sending mail means assembling the message in an old format by hand — knowing that in advance stops it feeling like failure.
3. **Use the Docs node for the best demo moment:** a template document with placeholders, filled in from a spreadsheet row. A mail merge in four nodes, and it looks impressive in ten seconds.
4. **The Code node's work is not running the code** — it is stopping bad code from taking the server down, which is the figure below.
5. **The AI node is one request and about half a day.** Because settings fields carry live data, somebody can type “Classify this message: `{{ ... }}`” straight into the sidebar and feed the answer to an IF node. Do not leave this until last: it is what makes ours an AI/automation platform.

THE CODE NODE – THREE WALLS, AND AN HONEST ADMISSION

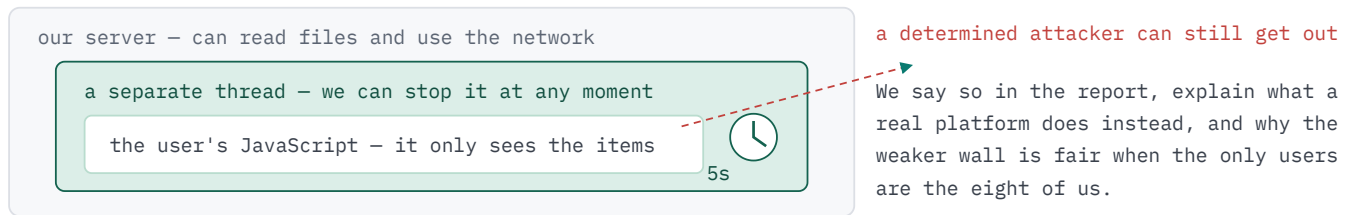


Figure 23. Being able to explain the limits of our own sandbox is worth more marks than quietly using a stronger one would have been.

DONE WHEN

The workflow above runs green from end to end, with a real email arriving and a real spreadsheet row appearing.

2.6b Build it: one helper, then four Google nodes

D1 writes the helper first. After that each node is a settings description plus one request, and six people can work in parallel without talking to each other.

PROGRAM VS Code — see §1.10 for the window, §1.11 for the loop

FILES `packages/nodes/google/request.js`, `sheets.js`, `docs.js`, `drive.js`

CREATE THEM In the Explorer, right-click `packages/nodes/google` → **New File** → type the name with its extension → paste → **Ctrl S**

0 First: let the engine hand a node its credential

Two more lines in the `ctx` object from §2.1b, and every node from here on can ask for the key it was configured with. Nothing else in the engine changes.

`packages/engine/engine.js` — inside the `ctx` object

```
credentialId: node.credentials?.id ?? null,
async getCredentials() {
  if (!node.credentials?.id)
    throw new WorkflowError(`${nodeName} has no credential selected — pick one in the sidebar`,
      { node: nodeName, code: 'NO_CREDENTIAL' });
  return loadDecryptedCredential(node.credentials.id); // apps/api/credentials.js
},
```

1 The shared helper — refresh, retry and rate limits, written once

Every Google node calls this and nothing else. Get it right and the other four files are twenty lines each.


```

import { db } from '../../apps/api/db.js';
import { encrypt, decrypt } from '../../apps/api/crypto.js';

const cache = new Map(); // credentialId → { token, expiresAt }
const inFlight = new Map(); // credentialId → Promise, so eight items refresh ONCE

const sleep = (ms) => new Promise(r => setTimeout(r, ms));

export async function googleRequest(credentialId, url, options = {}) {
  const { retries = 4, timeoutMs = 30_000, ...init } = options;
  let token = await getAccessToken(credentialId);

  for (let attempt = 0; attempt <= retries; attempt++) {
    const res = await fetch(url, { ...init,
      signal: AbortSignal.timeout(timeoutMs),
      headers: { 'content-type': 'application/json', ...init.headers,
        Authorization: `Bearer ${token}` } });

    if (res.ok) return res.status === 204 ? {} : res.json();

    // 401 – the hour is up. Refresh once and try the same request again.
    if (res.status === 401 && attempt === 0) { token = await refresh(credentialId); continue; }

    // 403 with reason "insufficient permissions" is a SCOPE problem, and no amount of
    // retrying fixes it. Say the real cause instead of "Google 403".
    const body = await res.text();
    if (res.status === 403 && /insufficient|scope/i.test(body))
      throw new Error('This credential was not granted the scope this node needs. '
        + 'Reconnect it and tick the missing permission.');
```

// 429 / 5xx – back off and try again, honouring Retry-After when Google sends it

```

    if ((res.status === 429 || res.status >= 500) && attempt < retries) {
      const after = Number(res.headers.get('retry-after')) * 1000;
      await sleep(after || Math.min(16_000, 2 ** attempt * 1000) + Math.random() * 400);
      continue;
    }
    throw new Error(`Google ${res.status}: ${body.slice(0, 300)}`);
  }
  throw new Error('Google kept refusing after every retry');
}

async function getAccessToken(credentialId) {
  const hit = cache.get(credentialId);
  if (hit && hit.expiresAt > Date.now() + 60_000) return hit.token; // 60 s safety margin
  return refresh(credentialId);
}

function refresh(credentialId) {
  // single-flight: if a refresh is already running, everybody waits for that one
  if (inFlight.has(credentialId)) return inFlight.get(credentialId);

  const p = (async () => {
    const { rows: [row] } = await db.query('SELECT data FROM credentials WHERE id = $1',
      [credentialId]);

    if (!row) throw new Error('That credential has been deleted');
```

```

const saved = JSON.parse(decrypt(row.data));
if (!saved.refresh_token) throw new Error('This credential has no refresh token – reconnect it');

const r = await fetch('https://oauth2.googleapis.com/token', {
  method: 'POST', signal: AbortSignal.timeout(15_000),
  headers: { 'content-type': 'application/x-www-form-urlencoded' },
  body: new URLSearchParams({
    refresh_token: saved.refresh_token, grant_type: 'refresh_token',
    client_id: process.env.GOOGLE_CLIENT_ID,
    client_secret: process.env.GOOGLE_CLIENT_SECRET }),
});
const t = await r.json();
if (!r.ok) {
  if (t.error === 'invalid_grant')
    throw new Error('Google has expired this connection (Testing mode lasts 7 days). '
      + 'Open Credentials and press Connect again. ');
  throw new Error(`Refresh failed: ${t.error_description ?? t.error}`);
}
const merged = { ...saved, ...t, refresh_token: t.refresh_token ?? saved.refresh_token };
await db.query('UPDATE credentials SET data = $1, expires_at = $2 WHERE id = $3',
  [encrypt(JSON.stringify(merged)),
    new Date(Date.now() + t.expires_in * 1000), credentialId]);
cache.set(credentialId, { token: t.access_token,
  expiresAt: Date.now() + t.expires_in * 1000 });
return t.access_token;
}()).finally(() => inFlight.delete(credentialId));

inFlight.set(credentialId, p);
return p;
}

```

Single-flight — when ten items all discover the token has expired at the same millisecond, ten refresh requests go to Google, nine of them invalidate each other, and the node fails for no visible reason. Holding one shared promise in a Map fixes it in three lines and is worth a paragraph in the report.

2 Google Sheets — append every item in *one* request

The obvious version sends one request per item. Fifty items is fifty round trips and a rate limit. Collect first, send once — this is measurable, so measure it for the report.

```

// "Sheet1" is fine; "Q3 Orders" and "Bob's list" are not, until they are quoted
const quoteSheetName = (name) => `${String(name).replace(/'/g, "'")}`;

// accepts a full URL or a bare id, and says so clearly when it is neither
export function parseSheetId(value) {
  const s = String(value ?? '').trim();
  const fromUrl = s.match(/\/spreadsheets\/d\/([a-zA-Z0-9-_]+)/?.[1];
  const id = fromUrl ?? s;
  if (!/^[a-zA-Z0-9-_]{20,}$/.test(id))
    throw new Error(`${s} is not a spreadsheet id or URL`);
  return id;
}

export const sheetsAppend = {
  async execute(ctx) {
    const items = ctx.getInputData();
    if (items.length === 0) return [[]];

    const id = parseSheetId(ctx.getNodeParameter('documentId', 0));
    const range = `${quoteSheetName(ctx.getNodeParameter('sheetName', 0))!A:Z`;

    // read the header row once, so column order follows the sheet, not the object
    const header = await googleRequest(ctx.credentialId,
      `https://sheets.googleapis.com/v4/spreadsheets/${id}/values/`
      + `${encodeURIComponent(range.replace(':Z', '1:Z1'))}`);
    const columns = header.values?.[0] ?? null;

    const values = items.map((_, i) => {
      const row = ctx.getNodeParameter('columns', i); // { name, email, ... }
      return columns ? columns.map(c => row[c] ?? '') : Object.values(row);
    });

    // Sheets caps a write at 10 MB / ~10 000 rows; chunk so a big run cannot fail as a whole
    const results = [];
    for (let start = 0; start < values.length; start += 500) {
      const chunk = values.slice(start, start + 500);
      results.push(await googleRequest(ctx.credentialId,
        `https://sheets.googleapis.com/v4/spreadsheets/${id}/values/`
        + `${encodeURIComponent(range)}:append`
        + `?valueInputOption=USER_ENTERED&insertDataOption=INSERT_ROWS`,
        { method: 'POST', body: JSON.stringify({ values: chunk }) }));
    }

    ctx.logger.info(`appended ${values.length} row(s) in ${results.length} request(s)`);
    return [items.map((item, i) => ({ json: { ...item.json,
      updatedRange: results[0]?.updates?.updatedRange }, pairedItem: i }))]
  },
};

```

3 Google Docs — copy the template, then mail-merge the copy

packages/nodes/google/docs.js

```
// 1 · never write into the template itself – copy it with Drive first
const copy = await googleRequest(ctx.credentialId,
  `https://www.googleapis.com/drive/v3/files/${templateId}/copy`,
  { method: 'POST', body: JSON.stringify({ name: `${docName} - ${new Date().toISOString().slice(0,10)}` } ) });

// 2 · one batchUpdate replaces every placeholder atomically: all of them, or none
const replacements = ctx.getNodeParameter('replacements', i); // { customer_name: 'Alice', ... }
const requests = Object.entries(replacements).map(([k, v]) => ({
  replaceAllText: { containsText: { text: `${k}`, matchCase: true },
    replaceText: String(v ?? '') },
}));
if (requests.length === 0) throw new Error('No replacements were given – the copy would be blank');

const result = await googleRequest(ctx.credentialId,
  `https://docs.googleapis.com/v1/documents/${copy.id}:batchUpdate`,
  { method: 'POST', body: JSON.stringify({ requests }) });

// 3 · tell the user how many placeholders actually matched – a zero here means a typo
const replaced = result.replies.reduce((n, r) => n + (r.replaceAllText?.occurrencesChanged ?? 0), 0);
ctx.logger.info(`replaced ${replaced} placeholder(s) in ${copy.id}`);
out.push({ json: { documentId: copy.id, replaced,
  url: `https://docs.google.com/document/d/${copy.id}/edit` }, pairedItem: i });
```

4 Google Drive — upload, which is multipart and slightly odd

packages/nodes/google/drive.js

```
const MAX_SIMPLE_UPLOAD = 5 * 1024 * 1024; // above this, Google wants a resumable upload

export async function uploadFile(ctx, { fileName, mimeType, fileBuffer, folderId }) {
  if (!Buffer.isBuffer(fileBuffer)) throw new Error('The file must arrive as binary data');
  if (fileBuffer.length > MAX_SIMPLE_UPLOAD)
    throw new Error(`${fileName} is ${(fileBuffer.length / 1e6).toFixed(1)} MB – use the `
      + 'resumable endpoint for files over 5 MB');

  const boundary = 'b' + crypto.randomUUID();
  const meta = { name: fileName, ...(folderId ? { parents: [folderId] } : {}) };

  const body = Buffer.concat([
    Buffer.from(`--${boundary}\r\ncontent-type: application/json; charset=UTF-8\r\n\r\n`,
    Buffer.from(JSON.stringify(meta)),
    Buffer.from(`\r\n--${boundary}\r\ncontent-type: ${mimeType}\r\n\r\n`,
    fileBuffer,
    Buffer.from(`\r\n--${boundary}--\r\n`), // the trailing CRLF matters; without it: 400
  ]);

  return googleRequest(ctx.credentialId,
    'https://www.googleapis.com/upload/drive/v3/files'
    + '?uploadType=multipart&fields=id,name,webViewLink,size',
    { method: 'POST', body,
      headers: { 'content-type': `multipart/related; boundary=${boundary}` } });
}
```

`drive.file` scope only ever sees files *our app created* — that is why it does not trigger Google's restricted-scope review, and it is a good sentence for the report's security section.

docs.google.com/spreadsheets/d/1aB.../edit

A	B	C	D
customer	email	total	received_at
Alice Nguyen	alice@example.com	42.00	14:32:07
Bilal Rahman	bilal@example.com	17.50	14:32:07
Chen Wu	chen@example.com	88.20	14:32:07

3 rows arrived together, in one request, in column order taken from the header row. The timestamps are identical because it was a single append, not three.

Figure 24. What batching looks like from the user's side. Three items in, one API call out, identical timestamps — and the same code handles five hundred.

Engine log — the happy path

```

info Webhook 3 item(s) received
info Sheets appended 3 row(s) in 1 request(s)
done Sheets 3 item(s) in 241 ms
info Docs replaced 6 placeholder(s)
done Docs 1 item(s) in 890 ms

```

Engine log — under pressure, and wrong

```

warn Sheets 429 — retrying in 1.2 s (1/4)
warn Sheets 429 — retrying in 2.3 s (2/4)
done Sheets 3 item(s) in 3 780 ms
# wrong scope:
error Gmail This credential was not granted
the scope this node needs.
Reconnect it and tick the
missing permission.

```

HOW THIS PART CONNECTS TO THE OTHERS

\$2.5b credentials · \$2.3b properties
decrypt() · description

googleRequest()

\$2.6b — the Google nodes
Sheets · Docs · Drive

items with json{}

\$2.4b triggers · \$2.6c Gmail
and the Week 14 demo workflow

- RECEIVES** a credential id from the workflow JSON, and the property vocabulary of \$2.3b. It never touches OAuth itself — that is entirely \$2.5b's job.
- HANDS OUT** `googleRequest(credentialId, url, options)` — the only function any Google node calls. Refresh, retry, rate limits and scope errors live inside it, once, for all four nodes.
- HANDS ON** ordinary items `{ json, pairedItem }` to the next node, exactly as \$2.1b requires. A Sheets node and an HTTP node are indistinguishable to the engine.
- SEAM TO AGREE** who owns `request.js`. D1 writes it in Week 9 and freezes it; D2, A1 and A2 then write four nodes against it in parallel without one merge conflict.

CHECK IT WORKS

Webhook → Sheets append puts a row into a spreadsheet you have open on screen, within about a second. Send **three** items and confirm the log says `1 request(s)`, not three — that is the batching working. Then rename a column in the sheet and confirm the new order is followed without touching any code. Finally, connect a Sheets-only credential to the Gmail node and confirm you get the scope sentence, not a bare 403.

2.6c Build it: Gmail, the sandbox, and the AI node

PROGRAM	VS Code — see §1.10 for the window, §1.11 for the loop
FILES	<code>packages/nodes/google/gmail.js</code> , <code>core/code.js</code> , <code>ai/anthropic.js</code>
CREATE THEM	In the Explorer, right-click <code>packages/nodes</code> → <code>New File</code> → type the name with its extension → paste → <code>Ctrl S</code>

5 Gmail — build the message by hand, then base64url it

This is the only genuinely fiddly node. Knowing that in advance stops it feeling like failure. Three things bite: non-English characters in the subject, addresses that are not addresses, and attachments over Gmail's limit.

```
packages/nodes/google/gmail.js
```

```

const MAX_MESSAGE = 35 * 1024 * 1024; // Gmail's hard limit for a send

// "Résumé" in a Subject: header is illegal ASCII. RFC 2047 is how you smuggle it through.
const encodeHeader = (s) => /^[^\x20-\x7E]*$/.test(s)
  ? s
  : `=?UTF-8?B?${Buffer.from(s, 'utf8').toString('base64')}?=`;

const isEmail = (s) => /^[^\s@,;]+\@[^\s@,;]+\.[^\s@,;]+$/i.test(String(s).trim());

function addressList(value, field) {
  const list = String(value ?? '').split(',').map(s => s.trim()).filter(Boolean);
  if (list.length === 0) throw new Error(`${field} is empty`);
  const bad = list.filter(a => !isEmail(a));
  if (bad.length) throw new Error(`${field} contains an invalid address: ${bad[0]}`);
  return list.join(', ');
}

export function buildMime({ to, cc, subject, html, text, attachments = [], inReplyTo }) {
  const b = 'x' + crypto.randomUUID();
  const headers = [
    `To: ${addressList(to, 'To')}`,
    cc ? `Cc: ${addressList(cc, 'Cc')}` : null,
    `Subject: ${encodeHeader(subject ?? '(no subject)')}`,
    inReplyTo ? `In-Reply-To: ${inReplyTo}` : null, // makes it a reply, not a new thread
    inReplyTo ? `References: ${inReplyTo}` : null,
    'MIME-Version: 1.0',
    `Content-Type: multipart/mixed; boundary="${b}"`,
  ].filter(Boolean).join('\r\n');

  const bodyPart =
    `--${b}\r\nContent-Type: text/html; charset="UTF-8"\r\n`
    + `Content-Transfer-Encoding: base64\r\n\r\n`
    + `${Buffer.from(html ?? text ?? '', 'utf8').toString('base64')}\r\n`;

  const fileParts = attachments.map(a =>
    `--${b}\r\nContent-Type: ${a.mimeType || 'application/octet-stream'}\r\n`
    + `Content-Disposition: attachment; filename="${a.fileName.replace(/"/g, '"')}"\r\n`
    + `Content-Transfer-Encoding: base64\r\n\r\n${a.data}\r\n`).join('');

  const mime = `${headers}\r\n\r\n${bodyPart}${fileParts}--${b}--\r\n`;
  if (Buffer.byteLength(mime) > MAX_MESSAGE)
    throw new Error('That message is over Gmail's 35 MB limit – put the file in Drive and link it!');
  return mime;
}

const base64url = (s) => Buffer.from(s).toString('base64')
  .replace(/+/g, '-').replace(/\//g, '_').replace(/>=+$/, '');

// dry-run is a real feature: it renders and validates without sending, so the whole
// workflow can be tested a hundred times without a hundred emails to a real person
if (ctx.getNodeParameter('dryRun', i, false)) {
  out.push({ json: { dryRun: true, preview: mime.slice(0, 400) }, pairedItem: i });
} else {
  const sent = await googleRequest(ctx.credentialId,
    'https://gmail.googleapis.com/gmail/v1/users/me/messages/send',

```

```
{ method: 'POST', body: JSON.stringify({ raw: base64url(mime),  
                                     threadId: ctx.getNodeParameter('threadId', i, undefined) }) });  
out.push({ json: { messageId: sent.id, threadId: sent.threadId }, pairedItem: i });  
}
```

6 The Code node — a thread we can stop, and starve

Two limits, not one. A timeout stops slow code; only a *memory* limit stops `const a = []; while (true) a.push('x')` from taking the server with it.


```

import { Worker } from 'node:worker_threads';

const WORKER = `
  const vm = require('node:vm');
  const { parentPort, workerData } = require('node:worker_threads');
  const logs = [];
  const sandbox = {
    items: structuredClone(workerData.items),    // a copy: user code cannot mutate ours
    console: { log: (...a) => logs.push(a.map(String).join(' ')).slice(0, 100) },
    JSON, Math, Date, String, Number, Boolean, Array, Object, RegExp,
  };
  try {
    const result = vm.runInNewContext(workerData.code, vm.createContext(sandbox),
                                      { timeout: workerData.timeoutMs, displayErrors: true });
    parentPort.postMessage({ ok: true, result, logs });
  } catch (e) {
    parentPort.postMessage({ ok: false, error: e && e.message ? e.message : String(e), logs });
  }
`;

export function runUserCode(code, items, { timeoutMs = 5000, memoryMb = 64 } = {}) {
  return new Promise((resolve, reject) => {
    if (typeof code !== 'string' || code.trim() === '')
      return reject(new Error('The Code node is empty'));

    const worker = new Worker(WORKER, {
      eval: true,
      workerData: { code, items, timeoutMs },
      resourceLimits: { maxOldGenerationSizeMb: memoryMb, maxYoungGenerationSizeMb: 16 },
      stdout: true, stderr: true,    // user code cannot spam our server log
    });

    let settled = false;
    const finish = (fn, value) => { if (settled) return; settled = true;
                                   clearTimeout(timer); worker.terminate(); fn(value); };

    // the vm timeout can be beaten by a tight async loop, so the thread gets a hard deadline too
    const timer = setTimeout(
      () => finish(reject, new Error(`Code node timed out after ${timeoutMs} ms`)),
      timeoutMs + 500);

    worker.on('message', (m) => {
      if (!m.ok) return finish(reject, new Error(`Code node: ${m.error}`));
      if (!Array.isArray(m.result))
        return finish(reject, new Error('Code must return an array of items, e.g. return items;'));
      finish(resolve, { items: m.result.map(x => ('json' in x ?? {}) ? x : { json: x })),
                logs: m.logs });
    });

    worker.on('error', (e) => finish(reject,
      /heap out of memory|Worker terminated/i.test(e.message)
        ? new Error(`Code node used more than ${memoryMb} MB and was stopped`)
        : e));

    worker.on('exit', (code) => { if (!settled && code !== 0)
      finish(reject, new Error(`Code node exited unexpectedly (${code})`)); });
  });
}

```

```
});  
}
```

For the report: run this snippet inside the Code node and show it reaching the host — then explain what a production platform uses instead (`isolated-vm`, or a container per execution). It is the strongest security content available to us for about an hour of work.

the escape, for the report only

```
return this.constructor.constructor('return process.version')(); // vm is not a sandbox
```


7 The AI node — one request, and everything that can go wrong with it

packages/nodes/ai/anthropic.js

```
const MODELS = ['claude-opus-5', 'claude-sonnet-5', 'claude-haiku-4-5-20251001'];

async function ask(ctx, i) {
  const prompt = String(ctx.getNodeParameter('userPrompt', i));
  if (prompt.length > 20_000) throw new Error('Prompt is too long – trim the data you send in');

  const body = {
    model:      ctx.getNodeParameter('model', i, MODELS[1]),
    max_tokens: Math.min(Math.max(ctx.getNodeParameter('maxTokens', i, 512), 1), 4096),
    system:     ctx.getNodeParameter('systemPrompt', i, ''),
    messages:   [{ role: 'user', content: prompt }],
  };

  for (let attempt = 0; attempt < 4; attempt++) {
    const res = await fetch(`${ctx.credentials.baseUrl} ?? 'https://api.anthropic.com'}/v1/messages`, {
      method: 'POST', signal: AbortSignal.timeout(60_000),
      headers: { 'x-api-key': ctx.credentials.apiKey, 'anthropic-version': '2023-06-01',
        'content-type': 'application/json' },
      body: JSON.stringify(body),
    });

    if (res.status === 429 || res.status === 529 || res.status >= 500) {
      const wait = Number(res.headers.get('retry-after')) * 1000 || 2 ** attempt * 1000;
      ctx.logger.info(`model busy (${res.status}) – waiting ${wait} ms`);
      await new Promise(r => setTimeout(r, wait));
      continue;
    }

    const data = await res.json();
    if (!res.ok) throw new Error(`AI ${res.status}: ${data.error?.message ?? 'unknown error'}`);
    if (data.stop_reason === 'max_tokens')
      ctx.logger.info('answer was cut off – raise Max tokens');

    const text = (data.content ?? []).filter(c => c.type === 'text').map(c => c.text).join('');
    return { text, usage: data.usage, model: data.model, stopReason: data.stop_reason };
  }

  throw new Error('The model stayed busy after four attempts');
}

// asking for JSON and getting prose is the classic failure. Ask, then check, then repair once.
export function parseJsonAnswer(text, expectedKeys = []) {
  const block = text.match(/````(?:json)?\s*(?![\s\S]*?)````)??.[1] ?? text;
  const slice = block.slice(block.indexOf('{'), block.lastIndexOf('}') + 1);
  let parsed;
  try { parsed = JSON.parse(slice); }
  catch { throw new Error(`The model did not return JSON. It said: ${text.slice(0, 120)}...`); }
  const missing = expectedKeys.filter(k => !(k in parsed));
  if (missing.length) throw new Error(`The answer is missing: ${missing.join(', ')}`);
  return parsed;
}
```

Because `userPrompt` runs through the expression resolver, somebody can type `Classify this: {{ $json.body.message }}`. Reply as `JSON: {"urgency": "urgent|routine"}` in the sidebar and feed the answer straight into an IF node. Point a second

credential's `baseUrl` at `http://localhost:11434/v1` to use a local Ollama model instead – the same node, no code change, and a good paragraph for the report.

WHAT YOU SHOULD SEE

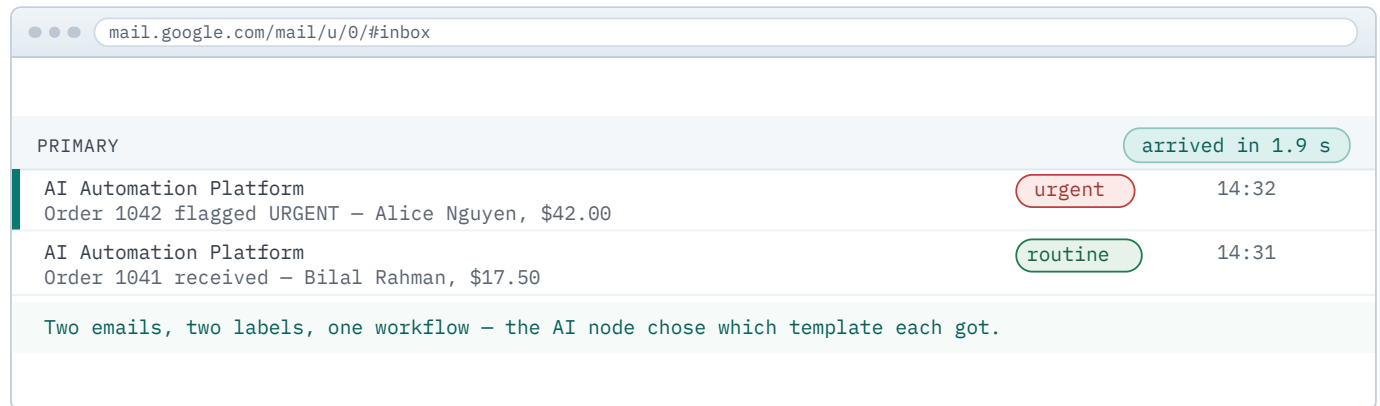
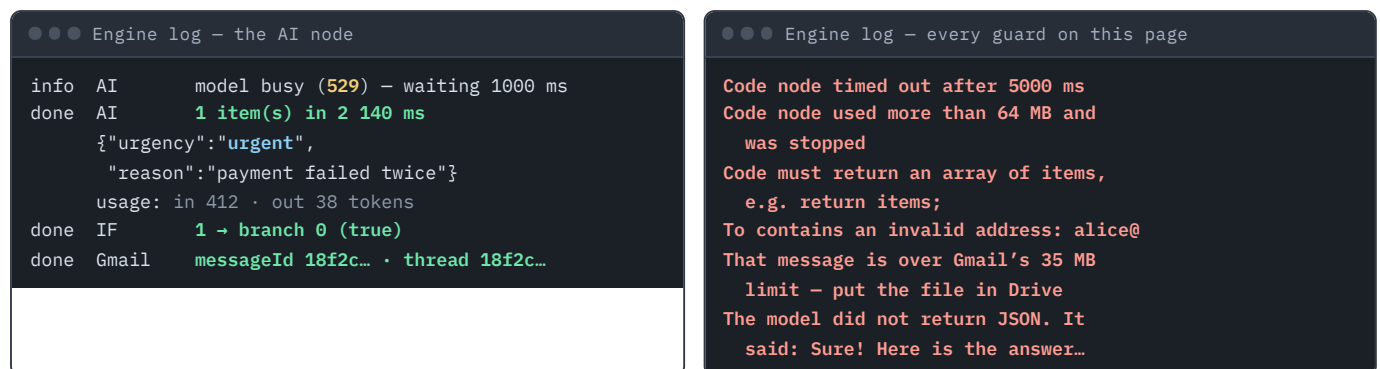
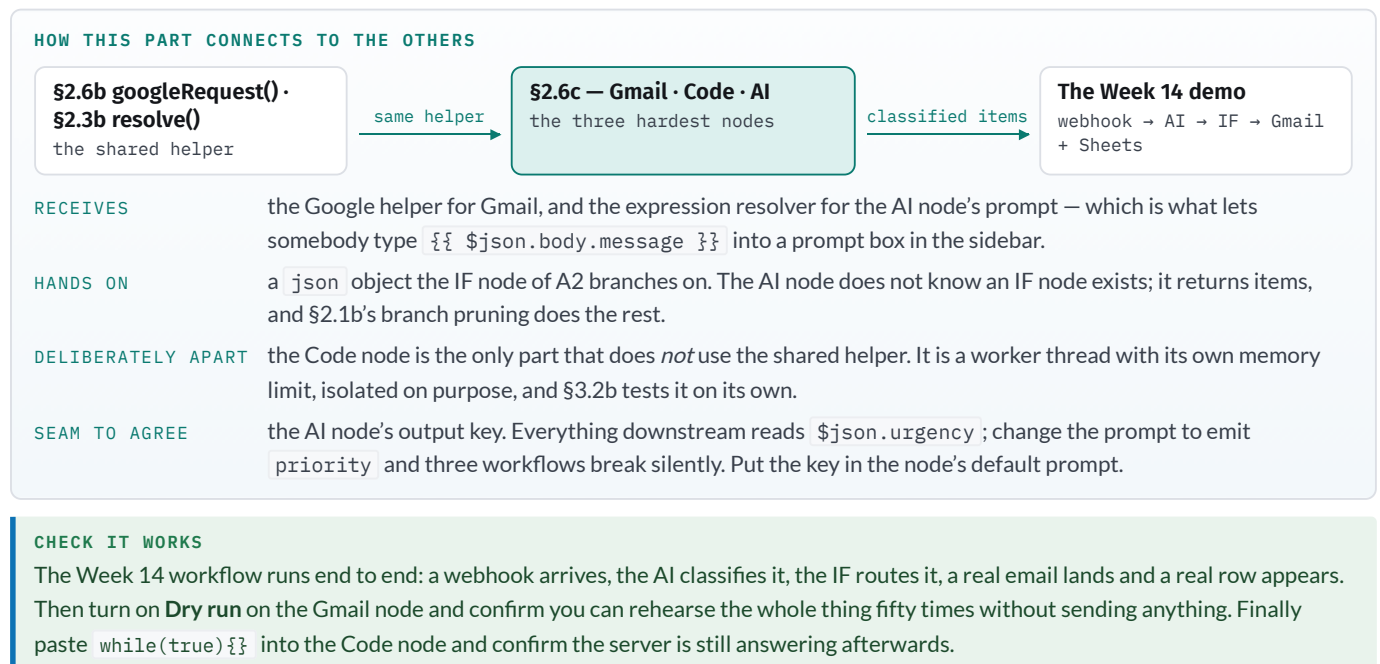


Figure 25. The end of the Week 14 demo, in the demonstrator's own inbox. The pill on the right is the AI node's answer, carried through an IF node into two different email bodies.



Every line on the right is a deliberate test, not an accident. Run all six before the demo and screenshot them: they are the evidence that the platform fails safely, which is most of what "robust" means to a marker.



PHASE 6 · SHIP IT STAGES 13-15 · WEEKS 11-15 · LEAD: D2 · ALL HANDS

2.7 Online, measured, and written up

WHO BUILDS THIS PART

D2 leads

all hands

WHAT YOU NEED TO KNOW

Docker Compose · the Linux command line over **SSH** · basic **DNS** (one A record) · reading a load-test summary without over-claiming.

What we are doing. Putting the platform on a real computer so our web addresses work from anywhere, then measuring it honestly against n8n, then writing the report. **At the end of Week 11 we stop adding features** — everything after that is fixing, measuring and writing.

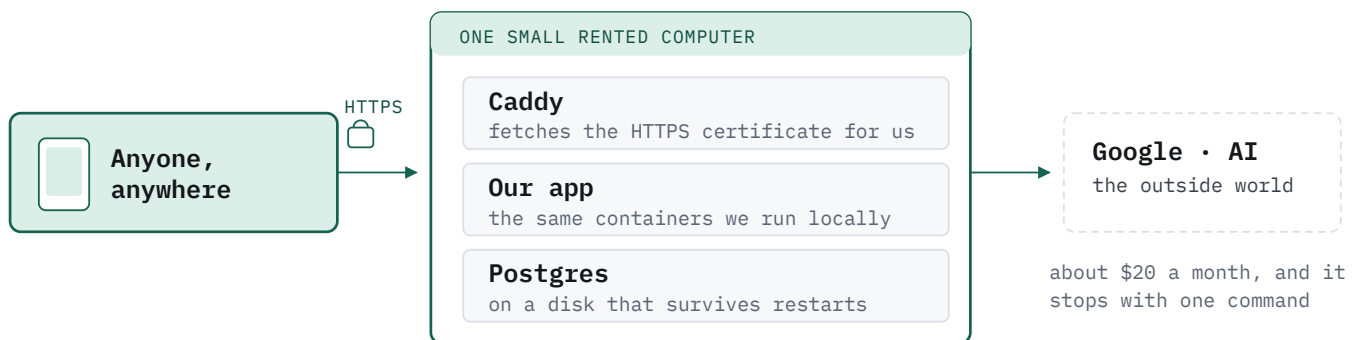


Figure 26. Not a cloud architecture diagram — one machine running the same three containers we already use. An afternoon's work, not two weeks.

FAMILIAR FROM N8N

This is simply n8n's own self-hosting story pointed at our code. The reason your `docker run n8nio/n8n` worked first time is the reason our deploy will: the container carries everything it needs with it.

TOOLS WE USE IN THIS PHASE

AWS EC2	one small rented machine that runs the whole platform	Caddy	gets and renews the HTTPS certificate by itself; three lines of configuration
n8n (Docker)	the platform we compare against, set up the same way as ours	k6	sends measured traffic at both so the comparison is real

HOW WE PROCEED

1. **Freeze features at the end of Week 11.** A group decision made in advance, so it is not an argument later.
2. **Rent one small machine, point a cheap domain at it,** and let Caddy handle HTTPS. Set a spending alarm on day one, before launching anything.
3. **Run n8n on the same machine, configured the same way** — not its simplest setup, or the comparison is unfair and a marker will notice.
4. **Build the same three workflows on both,** run each three times, and keep the raw numbers rather than only the averages.
5. **Time ourselves adding a new node to each platform.** We will win this clearly, and nobody else will think to measure it.
6. **Write honestly.** n8n does far more per run than we do, so being faster mostly means “we do less”. Saying that plainly is worth more marks than the number itself.

DONE WHEN

The canvas and a webhook address both work from a phone on mobile data, the benchmark numbers are recorded, and the report draft is circulating.

WATCH OUT

Write the report from Week 5 onwards, a section at a time, and screenshot everything as it is built. You cannot re-screenshot a server you have already shut down.

2.7b Build it: online, and measured against n8n

PROGRAM	VS Code locally, then a terminal over SSH on the server
FILES	Dockerfile, docker-compose.prod.yml, Caddyfile, benchmarks/k6/w1.js
CREATE THEM	In the Explorer, right-click the project root, then benchmarks/k6 → New File → type the name with its extension → paste → Ctrl S

1 The image — small, and not running as root

Dockerfile

```
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
COPY apps/api/package*.json apps/api/
COPY packages/engine/package*.json packages/engine/
RUN npm ci --omit=dev # deterministic install from the lockfile, no dev tools
COPY . .
RUN npm run build --workspace apps/editor

FROM node:22-alpine
WORKDIR /app
ENV NODE_ENV=production
COPY --from=build /app /app
USER node # a container escape lands on an unprivileged account
EXPOSE 5678
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
  CMD wget -qO- http://localhost:5678/healthz || exit 1
CMD ["node", "apps/api/server.js"]
```

Multi-stage build — the first FROM compiles, the second copies only the result. The shipped image has no compilers and no dev dependencies in it, so it is smaller and there is less inside it to attack.

2 The production stack — three containers and a certificate

docker-compose.prod.yml

```
services:
  caddy:
    image: caddy:2
    restart: unless-stopped
    ports: ["80:80", "443:443"]
    volumes: [".:/Caddyfile:/etc/caddy/Caddyfile:ro", "caddy_data:/data"]
    depends_on: { api: { condition: service_healthy } }
  api:
    build: .
    restart: unless-stopped
    env_file: .env
    environment:
      DATABASE_URL: postgres://postgres:${DB_PASSWORD}@postgres:5432/automation
      NODE_ENV: production
    depends_on: { postgres: { condition: service_healthy } }
    deploy: { resources: { limits: { cpus: "1.0", memory: 1G } } }
    logging: { driver: json-file, options: { max-size: "10m", max-file: "3" } }
  postgres:
    image: postgres:16
    restart: unless-stopped
    environment:
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: automation
    volumes: ["pgdata:/var/lib/postgresql/data"]
    # NO ports: - the database is reachable only from the other containers
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d automation"]
      interval: 10s
      retries: 5
volumes:
  caddy_data: {}
  pgdata: {}
```

Caddyfile — TLS, headers and compression in nine lines

```
your-domain.com {
  encode gzip
  header {
    Strict-Transport-Security "max-age=31536000"
    X-Content-Type-Options "nosniff"
    Referrer-Policy "strict-origin-when-cross-origin"
    -Server
  }
  reverse_proxy api:5678
}
```

Caddy obtains and renews the Let's Encrypt certificate by itself. There is no step 2.

3 Rent the machine — the AWS console, click by click

- 1 `console.aws.amazon.com` → search `EC2` → `Instances` → `Launch instances`
- 2 Name `automation-prod` · image `Ubuntu Server 24.04 LTS` · instance type `t3.small`
- 3 **Key pair** → `Create new key pair` → `RSA`, `.pem` → it downloads once. Lose it and you lose the machine.
- 4 **Network settings** → `Edit` → allow `SSH` from `My IP`, and `HTTP` + `HTTPS` from anywhere
- 5 Storage `20 GiB gp3` → `Launch instance`
- 6 `EC2` → `Elastic IPs` → `Allocate Elastic IP address` → `Associate` it with the instance, so the address survives a reboot
- 7 At your domain registrar add one **A record** pointing `your-domain.com` at that address
- 8 **Before anything else:** `Billing` → `Budgets` → `Create budget` → cost budget, **\$50**, alert to all eight emails

```
$ chmod 400 automation-prod.pem
$ ssh -i automation-prod.pem ubuntu@your-domain.com
$ curl -fsSL https://get.docker.com | sh && sudo usermod -aG docker ubuntu
$ exit && ssh -i automation-prod.pem ubuntu@your-domain.com # re-login for the group
```

4 Put the project on it — and take a backup before you ever need one

```
$ git clone https://github.com/<org>/ai-automation && cd ai-automation
$ cp .env.example .env && nano .env # real keys go here only
$ docker compose -f docker-compose.prod.yml up -d --build
$ docker compose -f docker-compose.prod.yml ps # all three must say healthy
# later deploys:
$ git pull && docker compose -f docker-compose.prod.yml up -d --build
```

`infra/backup.sh` — add to crontab, 03:00 daily

```
#!/usr/bin/env bash
set -euo pipefail # stop on the first error, not after it
cd "$(dirname "$0")/.."
mkdir -p backups
STAMP=$(date +%F-%H%M)
docker compose -f docker-compose.prod.yml exec -T postgres \
  pg_dump -U postgres automation | gzip > "backups/automation-$STAMP.sql.gz"
find backups -name '*.sql.gz' -mtime +14 -delete # keep a fortnight
echo "backup ok: backups/automation-$STAMP.sql.gz"
```

```
$ chmod +x infra/backup.sh
$ (crontab -l 2>/dev/null; echo "0 3 * * * $PWD/infra/backup.sh >> $PWD/backups/cron.log 2>&1") | crontab -
# then RESTORE one into a scratch database this week — a backup you have never
# restored is not a backup, and "we tested our restore path" is a report sentence
```

5 Stand n8n up beside it, configured the same way

benchmarks/n8n/docker-compose.yml

```
services:
  n8n:
    image: n8nio/n8n:1.70.1           # pin it, and cite this version in the report
    environment:
      DB_TYPE: postgresdb
      DB_POSTGRESDB_HOST: n8n-db
      EXECUTIONS_MODE: queue
      QUEUE_BULL_REDIS_HOST: redis
      GENERIC_TIMEZONE: Australia/Sydney
    ports: ["5679:5678"]
    deploy: { resources: { limits: { cpus: "1.0", memory: 1G } } }
  n8n-worker:
    image: n8nio/n8n:1.70.1
    command: worker
    deploy: { replicas: 2, resources: { limits: { cpus: "1.0", memory: 1G } } }
```

Give our own containers the identical `cpus` and `memory` limits, or the comparison is not a comparison. Say in the report that n8n ran two workers and we ran one process — that difference *is* the finding, not a flaw in the method.

6 Measure both with the same script

benchmarks/k6/w1.js

```
import http from 'k6/http';
import { check } from 'k6';
import { Trend, Rate } from 'k6/metrics';

const e2e = new Trend('workflow_end_to_end_ms');
const failed = new Rate('workflow_failed');

export const options = {
  scenarios: {
    warmup: { executor: 'constant-vus', vus: 2, duration: '20s', tags: { phase: 'warmup' } },
    load: { executor: 'constant-vus', vus: 10, duration: '60s', startTime: '25s',
      gracefulStop: '10s', tags: { phase: 'load' } },
  },
  // the run FAILS if these are missed, so a bad result cannot be quietly reported as good
  thresholds: {
    'http_req_duration{phase:load}': ['p(95)<1000', 'p(99)<2500'],
    'workflow_failed': ['rate<0.01'],
    'http_req_failed': ['rate<0.01'],
  },
  summaryTrendStats: ['avg', 'min', 'med', 'p(95)', 'p(99)', 'max'],
};

export default function () {
  const started = Date.now();
  const res = http.post(`${__ENV.TARGET}/webhook/bench-w1`,
    JSON.stringify({ id: __VU, iteration: __ITER, ts: started }),
    { headers: { 'content-type': 'application/json' }, timeout: '30s' });

  const ok = check(res, {
    'status is 200': (r) => r.status === 200,
    'body has a row id': (r) => String(r.body).includes('sheetRow'),
  });
  failed.add(!ok);
  e2e.add(Date.now() - started);
}
```

```
$ k6 run -e TARGET=http://localhost:5678 --summary-export=ours-w1-run1.json benchmarks/k6/w1.js
$ k6 run -e TARGET=http://localhost:5679 --summary-export=n8n-w1-run1.json benchmarks/k6/w1.js
# three runs each, commit every JSON file – the raw data is what makes it credible
```

WHAT YOU SHOULD SEE

Terminal – k6 run, ours

```
scenarios: (100.00%) 2 scenarios, 12 max VUs, 1m25s max duration

✓ status is 200
✓ body has a row id

checks.....: 100.00% ✓ 4318 x 0
http_req_duration{phase:load}..: avg=182ms med=161ms p(95)=337ms p(99)=612ms
http_req_failed.....: 0.00% ✓ 0 x 2159
iterations.....: 2159 35.9/s
workflow_end_to_end_ms.....: avg=184ms med=163ms p(95)=340ms

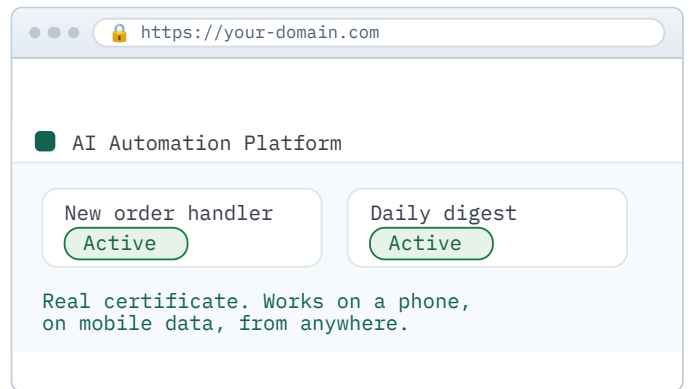
✓ THRESHOLDS PASSED · summary written to ours-w1-run1.json
```



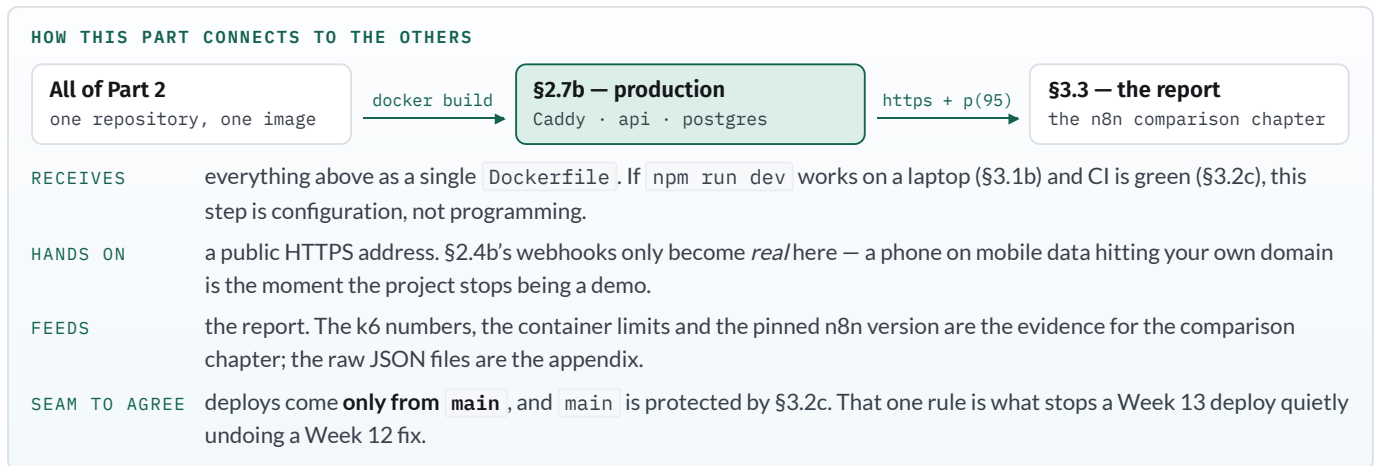
```
Terminal - the stack, healthy

$ docker compose -f docker-compose.prod.yml ps
NAME          STATUS
caddy         Up 6 min   80,443
api           Up 6 min   (healthy)
postgres      Up 6 min   (healthy) - no host port

$ curl -sI https://your-domain.com/healthz
HTTP/2 200
strict-transport-security: max-age=31536000
x-content-type-options: nosniff
```



The line that matters in the report is `p(95)`. Quote it beside `n8n`'s, from three runs each, with the container limits stated. An honest number that loses to `n8n` is worth more marks than a flattering number nobody can reproduce.



CHECK IT WORKS

The editor and a webhook address both work from a phone on mobile data, and you have six JSON result files for workflow 1. Then repeat for workflows 2 and 3. Reboot the EC2 instance from the AWS console and confirm all three containers come back by themselves and the schedules re-register — `restart: unless-stopped` plus `restoreAll()` is what makes that true.

2.8 How all the parts connect to each other

WHO THIS PAGE IS FOR

A1 A2 B1 B2 C1 C2 D1 D2

WHAT IT IS

Every arrow below is a promise one pair makes to another. Print this page, put it on the wall, and settle arguments with it.

Each section of Part 2 ended with a small strip showing what it receives and what it hands on. This is all of them at once. **Nothing crosses a box boundary except the labelled arrows** — that single rule is what lets four pairs work in four folders without waiting for each other.

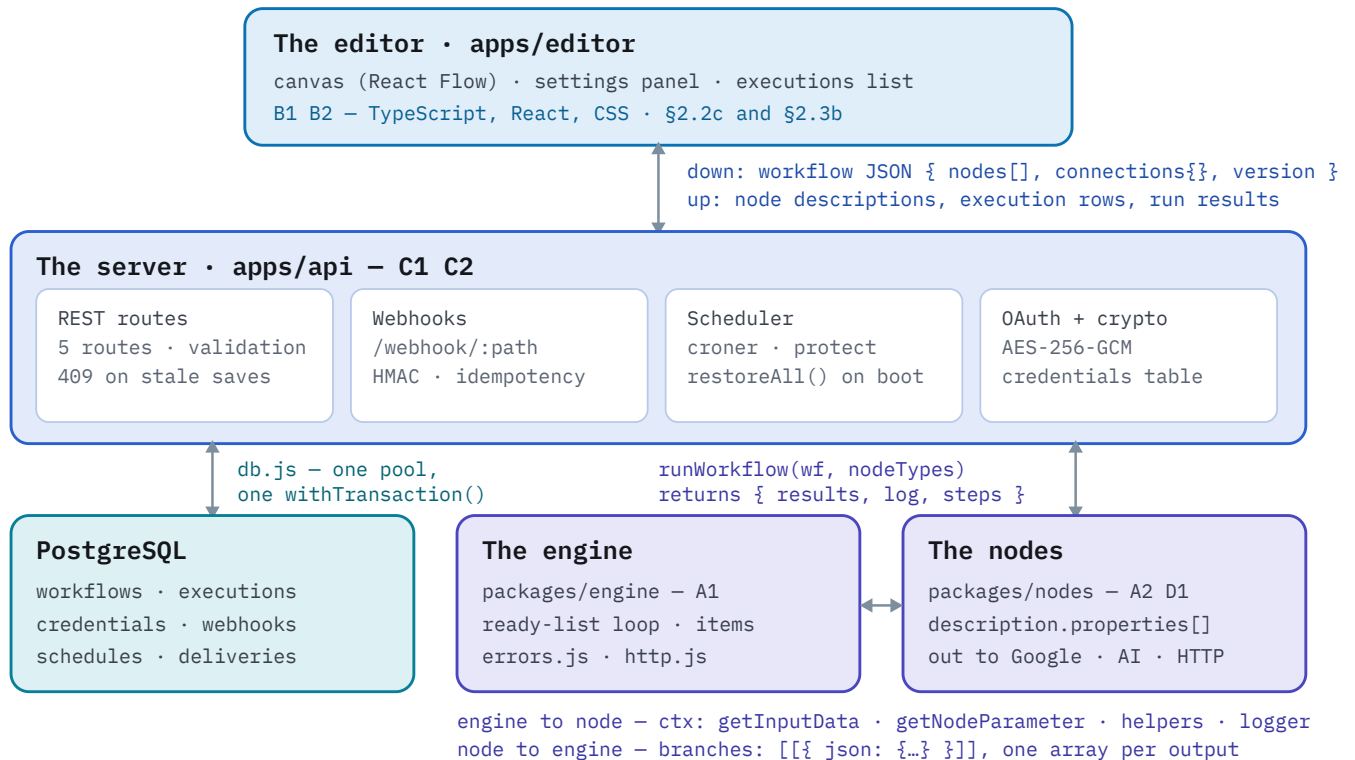


Figure 27. Every seam in the system. Four boxes, four owners, eight labelled promises. If a change would add a ninth arrow, it needs a conversation first — that is the whole governance rule for the semester.

READING THE PICTURE AS A WORK PLAN

- WEEK 3–4** Only the two purple boxes exist. A1 freezes `ctx` and the branch shape; **everybody else writes against a written promise, not against code that exists yet.**
- WEEK 5** The blue and cyan boxes join, and the top arrow becomes real. This is the hard gate: the editor can save and reload a workflow.
- WEEK 8** The right-hand purple box grows six new files at once. Nothing else changes — that is the payoff for §2.3b.
- WEEK 9–10** The OAuth sub-box and the outward arrows from the nodes. D1 owns `request.js`; D2, A1 and A2 write nodes against it in parallel.
- WEEK 12** The whole picture is put on one server (§2.7b) and measured against n8n running the same three workflows.

USE IT LIKE THIS

When two people disagree about where code should live, find the arrow it crosses. Code that translates between two boxes belongs to *one* of them, never both — that is what `convert.ts`, `runner.js` and `request.js` each are. If a new idea needs a new arrow, that is a ten-minute group conversation, not a pull request somebody discovers on Thursday.

3.1 How six phases become one working system

WHO BUILDS THIS PART

D2

one of the eight rotates each week

WHAT YOU NEED TO KNOW

git branches and merges · npm workspaces · Docker Compose · the discipline to run the whole thing from a clean clone every Thursday.

Each phase produced a piece, and each of us has been working in our own folder. Integration is not a week at the end — it is a habit from Week 4: **one repository, one command, and everything runs**. If assembling the project is ever difficult, something went wrong earlier.

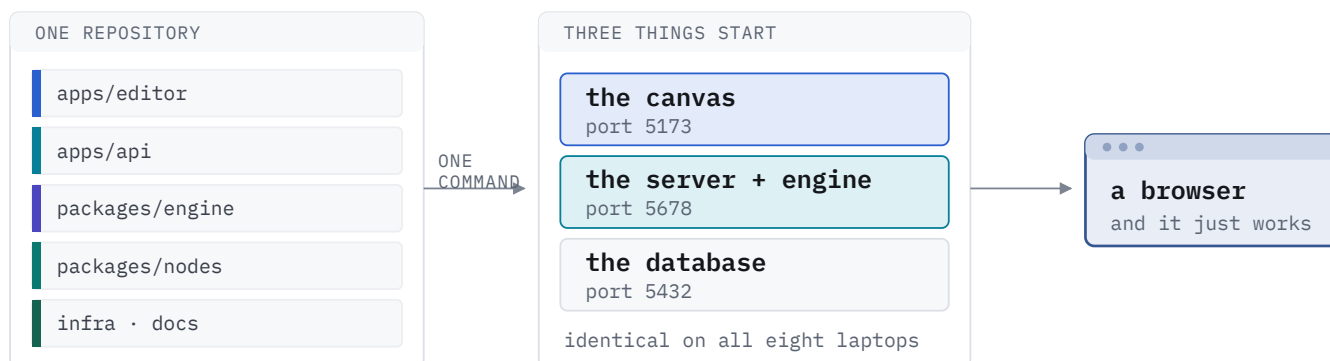


Figure 28. Five folders, one command, three programs. Anyone who can clone the repository can run the whole platform.

HOW WE PROCEED

1. **One repository from day one**, everyone on their own branch, merging into the main one. Never a zip file passed around, and never “I will send you my folder”.
2. **One command starts everything**. If that command ever stops working, fixing it matters more than any feature.
3. **One example settings file** in the repository listing every key the project needs, with fake values. Real keys live only in each person's own untracked copy.
4. **A starter workflow loaded on first run**, so a fresh clone shows something working instead of an empty screen.
5. **Integration every Thursday**. Everything merges, and one person — a different one of the eight each week — clones the repository fresh and runs it. Whatever breaks is that week's priority, and D2 chairs it.

DONE WHEN

Someone who has never touched the project can clone it, run one command, and have a working platform in under ten minutes without asking a question.

WATCH OUT

The classic failure is eight people whose parts each work alone and have never run together — and with eight it happens faster. Merging weekly from Week 4 makes integration boring, which is exactly what we want it to be.

3.1b Build it: one repository, one command

PROGRAM VS Code — see §1.10 for the window, §1.11 for the loop

FILES `package.json`, `.env.example`, `seed.sql`, `scripts/doctor.js`

CREATE THEM In the Explorer, right-click the project root → `New File` → type the name with its extension → paste → `Ctrl S`

1 The layout everyone clones

the whole tree

```
ai-automation/
├─ apps/
│  ├─ api/           # Express, the five routes, webhooks, schedules, OAuth C1 C2
│  └─ editor/        # React + React Flow + the parameter panel B1 B2
├─ packages/
│  ├─ engine/        # the loop, items, expressions, errors, http helper A1
│  └─ nodes/
│     ├─ core/       # IF, Set, Merge, Code, HTTP Request A2
│     └─ google/     # Sheets, Drive, Docs, Gmail D1
│        └─ ai/       # the LLM node A1
├─ benchmarks/      # k6 scripts, the n8n stack, raw results D2
├─ infra/            # Caddyfile, docker-compose.prod.yml, backup.sh D2
├─ scripts/          # doctor.js — the "why won't it start" answer machine C1
├─ docs/             # this guide, ADRs, the report as it is written D2
├─ e2e/              # the one Playwright test B2
├─ docker-compose.yml
├─ .nvmrc            # 22 — so nvm picks the right Node with no arguing
└─ .env.example
```

2 One command starts everything

package.json — root

```
{
  "type": "module",
  "workspaces": ["apps/*", "packages/*"],
  "engines": { "node": ">=22" },
  "scripts": {
    "doctor": "node scripts/doctor.js",
    "predev": "npm run doctor",
    "dev": "docker compose up -d --wait && npm run dev --workspaces --if-present",
    "migrate": "psql $DATABASE_URL -f schema.sql && psql $DATABASE_URL -f seed.sql",
    "test": "vitest run --coverage",
    "e2e": "playwright test",
    "lint": "eslint . && tsc --noEmit",
    "format": "prettier --write .",
    "secrets": "gitleaks detect --no-git --redact"
  }
}
```

```
$ git clone ... && cd ai-automation
$ cp .env.example .env
$ npm install
$ npm run migrate
$ npm run dev           # editor 5173 · api 5678 · postgres 5432
```

Five commands, ten minutes, no questions asked of anybody. That is the target. `--wait` makes docker block until the healthcheck passes, and `predev` makes npm run the doctor first, automatically, every single time.

3 The doctor — forty lines that answer "why won't it start?"

Eight people on four operating systems will hit the same six problems. Write the answers down once, in code, and stop answering them in the group chat.


```

import { execSync } from 'node:child_process';
import fs from 'node:fs';
import pg from 'pg';

const GREEN = '\x1b[32m', RED = '\x1b[31m', OFF = '\x1b[0m';
const results = [];

const check = (name, fn, fix) => {
  try { results.push(['ok', name, fn() ?? '']); }
  catch (e) { results.push(['fail', name, `${e.message} → ${fix}`]); }
};

check('Node 22 or newer', () => {
  const major = Number(process.versions.node.split('.')[0]);
  if (major < 22) throw new Error(`found v${process.versions.node}`);
  return `v${process.versions.node}`;
}, 'install Node 22 (npm install 22), then reopen the terminal');

check('Docker is running',
  () => execSync('docker info --format "{{.ServerVersion}}"',
    { stdio: ['ignore', 'pipe', 'ignore'] }).toString().trim(),
  'start Docker Desktop and wait for the whale to stop animating');

check('.env exists',
  () => { if (!fs.existsSync('.env')) throw new Error('missing'); return ''; },
  'run: cp .env.example .env');

for (const key of ['DATABASE_URL', 'BASE_URL', 'ENCRYPTION_KEY']) {
  check(`${key} is set`, () => {
    if (!process.env[key]) throw new Error('empty');
    if (key === 'ENCRYPTION_KEY' && /^[0-9a-f]{64}$/i.test(process.env[key]))
      throw new Error('must be 64 hex characters');
    return '';
  }, `fill ${key} in .env – see .env.example`);
}

const db = new pg.Pool({ connectionString: process.env.DATABASE_URL,
  connectionTimeoutMillis: 3000 });

try {
  const { rows } = await db.query("SELECT count(*)::int AS n FROM information_schema.tables "
    + "WHERE table_name IN ('workflows','executions')");
  results.push(rows[0].n === 2
    ? ['ok', 'Tables exist', 'workflows, executions']
    : ['fail', 'Tables exist', 'not migrated → run: npm run migrate']);
} catch (e) {
  results.push(['fail', 'Postgres answers', `${e.message} → run: docker compose up -d`]);
} finally {
  await db.end();
}

for (const [state, name, detail] of results)
  console.log(`${state} === 'ok' ? ${GREEN} ok ${OFF} : ${RED}FAIL${OFF}` +
    + `${name.padEnd(22)} ${detail}`);

const failed = results.filter(r => r[0] === 'fail').length;

```

```
console.log(failed ? `n${failed} problem(s) – fix the arrows above.` : `nEverything is ready.`);
process.exit(failed ? 1 : 0); // a non-zero exit is what stops 'npm run dev' continuing
```

4 Every key the project needs, with fake values

```
.env.example – committed; .env is never committed
```

```
DATABASE_URL=postgres://postgres:dev@localhost:5432/automation
BASE_URL=http://localhost:5678
EDITOR_ORIGIN=http://localhost:5173
ENCRYPTION_KEY=0000000000000000000000000000000000000000000000000000000000000000
GOOGLE_CLIENT_ID=replace-me.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=replace-me
ANTHROPIC_API_KEY=replace-me
```

```
$ printf '.env\nnode_modules\nbackups\n*.pem\n' >> .gitignore
$ npm run secrets # run it once a week; a leaked key costs marks
```

5 A starter workflow, so a fresh clone shows something

```
seed.sql
```

```
INSERT INTO workflows (name, nodes, connections) VALUES (
  'Hello weather',
  [{ "name": "Start", "type": "manualTrigger", "parameters": {}, "position": { "x": 80, "y": 80 },
    { "name": "Weather", "type": "httpRequest",
      "parameters": { "url": "https://wttr.in/Newcastle?format=j1", "method": "GET", "timeout": 15000 },
      "position": { "x": 360, "y": 80 } } ],
  { "Start": [ "Weather" ] } ]
) ON CONFLICT DO NOTHING;
```

The difference between a new teammate being productive in ten minutes or in two days.

WHAT YOU SHOULD SEE

```
Terminal – npm run dev, everything ready

$ npm run dev

ok Node 22 or newer      v22.11.0
ok Docker is running     27.3.1
ok .env exists
ok DATABASE_URL is set
ok BASE_URL is set
ok ENCRYPTION_KEY is set
ok Tables exist          workflows, executions

Everything is ready.

✓ Container automation-postgres Healthy
api on http://localhost:5678
restored 0 schedule(s)
VITE v6.0.1 ready in 431 ms
→ Local: http://localhost:5173/
```

```
Terminal – a teammate's first clone

$ npm run dev

ok Node 22 or newer      v22.11.0
FAIL Docker is running   not found
→ start Docker Desktop and wait for
the whale to stop animating
FAIL .env exists         missing
→ run: cp .env.example .env
FAIL ENCRYPTION_KEY is set empty
→ fill ENCRYPTION_KEY in .env
FAIL Postgres answers    ECONNREFUSED
→ run: docker compose up -d

4 problem(s) – fix the arrows above.

npm ERR! code 1 (dev never started)
```

The right-hand screen is the point of this page. Nobody had to ask anybody anything. Every line names the problem *and* the exact command that fixes it — and because `predev` runs it, a broken start can never happen silently.

HOW THIS PART CONNECTS TO THE OTHERS

Four areas, four folders

engine · editor · api ·
integrations

npm workspaces

\$3.1b — one repository

npm run dev · doctor

the same five
commands

\$3.2c — CI, and \$2.7b — deploy

run exactly what you run

RECEIVES

the four areas as workspace folders. npm links them for you, so `import { runWorkflow } from './../packages/engine/engine.js'` works with no build step and nothing published anywhere.

HANDS ON

one command. CI (\$3.2c) runs `npm run lint / test / e2e` and the server (\$2.7b) builds the same `Dockerfile` — so “works on my machine” and “works in CI” stop being different questions.

BINDS TOGETHER

the `.env` contract. Every part above reads its keys from the same file, and `doctor.js` is the one place that knows which keys must exist.

SEAM TO AGREE

nobody adds a dependency without saying so in the group chat. A lockfile conflict at 11 p.m. in Week 13 is the most avoidable disaster available to this project.

CHECK IT WORKS

Every Thursday, one of the eight — a different one each week — clones the repository into a brand-new folder and runs those five commands. Whatever breaks is that week's first priority, and the fix goes into `doctor.js` so it can never cost anybody an hour twice.

3.2 Four levels of testing, and what each one catches

WHO BUILDS THIS PART

D2 sets it up

every owner writes their own

WHAT YOU NEED TO KNOW

Vitest · a little Node `http` for the fake server · **Playwright** · **YAML** for the CI file.

Not testing everything — testing the things that break quietly. The rule: **the lower the level, the more tests and the faster they run**. We do not need all four from day one; each arrives with the phase that needs it.

1 · Engine tests

about 20 · milliseconds

Does the loop run nodes in the right order? Does a branch with zero items correctly stop everything after it?

2 · Node tests

one per node · seconds

Does the Gmail node build the right request from its settings? Runs against a fake server, never real Google.

3 · Browser test

one or two · a minute

Can a person drag out two nodes, connect them, press Run and see the result? One test covering the whole path.

4 · The real run

by hand · before demos

Does a real email actually arrive? Does a row appear in a real spreadsheet? Only a person can confirm this.

Levels 1–3 run automatically on every change. Level 4 is a checklist we work through before every demo.

Figure 29. Each level catches a kind of bug the level above it cannot. Skipping level 2 is why integrations break silently the week before a demo.

TOOLS WE USE FOR TESTING

Vitest

runs levels 1 and 2 — fast, and the default for a Node project

A fake server

a tiny local program that answers like Google does, so node tests never touch the real thing

Playwright

drives a real browser for level 3

GitHub Actions

runs levels 1–3 on every change, so nobody has to remember

HOW WE PROCEED

1. **Start with the engine (level 1), in Phase 1.** The cheapest tests we will ever write, and they protect the piece everything else depends on.
2. **Add one test per node as we build it (level 2),** not afterwards. The trick is the fake server: our node calls a local address instead of Google's, so the test is fast, free, and works with no internet.
3. **Write exactly one browser test (level 3)** once the canvas can save. One is enough — browser tests are slow and break for boring reasons.
4. **Keep level 4 as a written checklist,** not as tests. Section 3.3 is that checklist.
5. **Turn the automatic checks on early,** so that from Week 5 a broken change is caught when it is pushed rather than the night before the demo.

DONE WHEN

Someone pushes a change that breaks the IF node and a red cross appears within two minutes — without anybody noticing by hand.

DO NOT TEST

Google's APIs, React Flow, or Postgres. They are not our code and testing them wastes time we do not have. Test *our* logic and use a fake for everything else.

3.2b Build it: the four levels, in code

```
$ npm i -D vitest @vitest/coverage-v8 @playwright/test
$ npx playwright install chromium
```

PROGRAM	VS Code — see §1.10 for the window, §1.11 for the loop
FILES	packages/engine/engine.test.js, convert.test.ts, google/sheets.test.js
CREATE THEM	In the Explorer, right-click beside the file each one tests → New File → type the name with its extension → paste → Ctrl S

1 Level 1 — the engine, in milliseconds

Test the happy path once, then test **every guard you wrote in §2.1b**. Those six tests are the proof that "robust" is not just a word in the report.

```
packages/engine/engine.test.js
```

```

import { test, expect } from 'vitest';
import { runWorkflow, validateWorkflow } from './engine.js';
import { WorkflowError } from './errors.js';

const fake = (name, out, group = 'action') => ({
  description: { name, group, properties: [] },
  async execute() { return typeof out === 'function' ? out() : out; },
});

const wf = (nodes, connections) => ({ nodes, connections });

test('runs nodes in order and passes items along', async () => {
  const types = { start: fake('start', [[{ json: { a: 1 } }]], 'trigger'),
    next: fake('next', [[{ json: { b: 2 } }]]) };
  const { results, log } = await runWorkflow(
    wf([
      { name: 'S', type: 'start', parameters: {} },
      { name: 'N', type: 'next', parameters: {} }, { S: [['N']] }], types);
  expect(results.N[0][0].json).toEqual({ b: 2 });
  expect(log.filter(l => l.level === 'done')).toHaveLength(2);
});

test('an empty branch stops everything after it', async () => {
  const types = { start: fake('start', [], 'trigger'), never: fake('never', [[{ json: {} }]]) };
  const { results } = await runWorkflow(
    wf([
      { name: 'S', type: 'start', parameters: {} },
      { name: 'X', type: 'never', parameters: {} }, { S: [['X']] }], types);
  expect(results.X).toBeUndefined(); // it never ran - and that is correct
});

test('a misspelled target is refused before anything runs', () => {
  const types = { start: fake('start', [[{ json: {} }]], 'trigger') };
  expect(() => validateWorkflow(
    wf([
      { name: 'S', type: 'start', parameters: {} }, { S: [['Wether']] }], types))
    .toThrow(/missing node "Wether"/));
});

test('a loop is stopped by the step limit, not by the laptop', async () => {
  const types = { start: fake('start', [[{ json: {} }]], 'trigger'),
    echo: fake('echo', [[{ json: {} }]]) };
  await expect(runWorkflow(
    wf([
      { name: 'S', type: 'start', parameters: {} },
      { name: 'E', type: 'echo', parameters: {} }, { S: [['E']], E: [['E']] },
      types, { maxSteps: 25 })).rejects.toThrow(/loops/));
});

test('a hanging node is cut off and names itself', async () => {
  const types = { start: fake('start', [[{ json: {} }]], 'trigger'),
    slow: { description: { name: 'slow', group: 'action', properties: [] },
      execute: () => new Promise(() => {})} }; // never resolves
  await expect(runWorkflow(
    wf([
      { name: 'S', type: 'start', parameters: {} },
      { name: 'Slow', type: 'slow', parameters: {} }, { S: [['Slow']] },
      types, { nodeTimeoutMs: 40 })).rejects.toThrow(/"Slow" ran longer than/));
});

test('continueOnFail carries the error downstream instead of stopping', async () => {

```

```

const types = { start: fake('start', [[{ json: {} }]], 'trigger'),
  boom: { description: { name: 'boom', group: 'action', properties: [] },
    execute() { throw new Error('kaboom'); } },
  after: fake('after', [[{ json: { reached: true } }]] );
const { results } = await runWorkflow(
  wf([
    { name: 'S', type: 'start', parameters: {} },
    { name: 'B', type: 'boom', parameters: {}, continueOnFail: true },
    { name: 'A', type: 'after', parameters: {} },
    { S: [['B']], B: [['A']] },
  ], types);
expect(results.B[0][0].error).toMatch(/kaboom/);
expect(results.A[0][0].json.reached).toBe(true);
});

```

2 The one test that will save the editor team a week

apps/editor/src/convert.test.ts

```

import { test, expect } from 'vitest';
import { toReactFlow, fromReactFlow } from './convert';

const sample = {
  name: 'demo',
  nodes: [
    { name: 'A', type: 'x', parameters: {}, position: { x: 0, y: 0 } },
    { name: 'B', type: 'y', parameters: {}, position: { x: 200, y: 0 } },
    { name: 'C', type: 'y', parameters: {}, position: { x: 200, y: 120 } },
  ],
  connections: { A: [['B'], ['C']] }, // branch 0 → B, branch 1 → C (an IF node)
};

test('a workflow survives a round trip through React Flow unchanged', () => {
  const { nodes, edges } = toReactFlow(sample);
  expect(fromReactFlow(nodes, edges).connections).toEqual(sample.connections);
});

test('an edge to a deleted node is dropped, not crashed on', () => {
  const broken = { ...sample, connections: { A: [['B'], ['GHOST']] } };
  expect(toReactFlow(broken).edges).toHaveLength(1);
});

```

3 Level 2 — a node against a fake Google, with no internet

packages/nodes/google/sheets.test.js

```
import { test, expect, beforeAll, afterAll } from 'vitest';
import http from 'node:http';

let server, calls = [], nextStatus = 200;

beforeAll(() => new Promise(done => {
  server = http.createServer((req, res) => {
    let body = ''; req.on('data', c => body += c);
    req.on('end', () => {
      calls.push({ url: req.url, method: req.method, body: JSON.parse(body || '{}') });
      if (nextStatus !== 200) { // let a test ask for a failure
        res.writeHead(nextStatus, { 'retry-after': '0' }); return res.end('{}');
      }
      res.writeHead(200, { 'content-type': 'application/json' });
      res.end(JSON.stringify({ values: [['name', 'email']],
        updates: { updatedRows: 1, updatedRange: 'Orders!A18' } }));
    });
  }).listen(4010, done);
}));

afterAll(() => server.close());

test('append sends one row, in header order, to the right range', async () => {
  await sheetsAppend.execute(makeCtx({
    base: 'http://localhost:4010',
    documentId: 'https://docs.google.com/spreadsheets/d/ABC123/edit',
    sheetName: 'Orders', columns: { email: 'a@example.com', name: 'Alice' },
  }));
  const write = calls.at(-1);
  expect(write.method).toBe('POST');
  expect(write.url).toContain('/ABC123/values/');
  expect(write.body.values).toEqual(['Alice', 'a@example.com']); // sheet order, not object order
});

test('three items become ONE append request', async () => {
  calls = [];
  await sheetsAppend.execute(makeCtx({ base: 'http://localhost:4010', items: 3, /* ... */ }));
  expect(calls.filter(c => c.method === 'POST')).toHaveLength(1);
});

test('a 429 is retried, not thrown', async () => {
  calls = []; nextStatus = 429;
  setTimeout(() => { nextStatus = 200; }, 30); // recover after the first retry
  await expect(sheetsAppend.execute(makeCtx({ base: 'http://localhost:4010' })))
    .resolves.toBeDefined();
  expect(calls.length).toBeGreaterThan(1);
});

test('a sheet name with an apostrophe is quoted properly', () => {
  expect(quoteSheetName("Bob's list")).toBe("'Bob's list'");
});
```

Thirty lines, and every Google node gets the same treatment. No credentials, no quota, no network — so it runs on every push in about two seconds.

```

Terminal - npm test

✓ packages/engine/engine.test.js      (6 tests)   41 ms
✓ apps/editor/src/convert.test.ts      (2 tests)    9 ms
✓ packages/nodes/google/sheets.test.js (4 tests)  118 ms
✓ packages/nodes/core/code.test.js     (3 tests)   612 ms

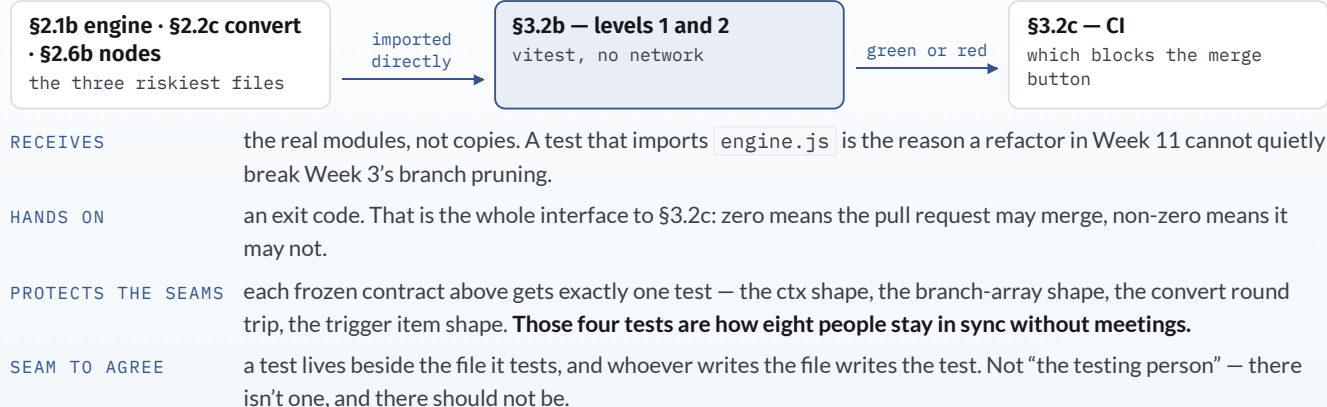
Test Files  4 passed (4)
Tests      15 passed (15)
Duration   2.14 s (transform 340ms, tests 780ms)

% Coverage report from v8
-----|-----|-----|-----|
File      | % Stmts | % Branch | % Funcs |
-----|-----|-----|-----|
engine.js |   94.1  |   88.0   |   100   |
http.js   |   91.6  |   85.7   |   100   |
google/request.js | 78.2 | 70.5 | 100 |
-----|-----|-----|-----|

```

Two numbers to quote in the report: the duration (under three seconds, so nobody is tempted to skip it) and the branch coverage on `engine.js`. Branch coverage is the honest one — it says the *error* paths were executed, not just the happy path.

HOW THIS PART CONNECTS TO THE OTHERS



CHECK LEVELS 1 AND 2

`npm test` finishes in under five seconds, with no internet connection and no Google credentials. If it needs either, the fake server is not being used. Then delete one guard from `engine.js` — say the step limit — and confirm a test goes red. A test suite that stays green when you break the code is worse than no test suite.

3.2c Build it: the browser test, and running it all automatically

PROGRAM	VS Code, then github.com in a browser to watch it run
FILES	playwright.config.ts, e2e/build-and-run.spec.ts, .github/workflows/ci.yml
CREATE THEM	In the Explorer, right-click new e2e and .github/workflows folders → New File → type the name with its extension → paste → Ctrl S

4 Let Playwright start the app itself

playwright.config.ts

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  testDir: 'e2e',
  timeout: 30_000,
  expect: { timeout: 8_000 },           // retry an assertion for 8 s before failing it
  retries: process.env.CI ? 2 : 0,     // a flaky test in CI is retried, but still reported
  reporter: process.env.CI ? [['github'], ['html', { open: 'never' }]] : 'list',
  use: { baseURL: 'http://localhost:5173', trace: 'retain-on-failure',
        screenshot: 'only-on-failure', video: 'retain-on-failure' },
  webServer: {                        // no "did you remember to start the server?" ever again
    command: 'npm run dev',
    url: 'http://localhost:5173',
    reuseExistingServer: !process.env.CI,
    timeout: 120_000,
  },
});
```

Trace — a recording of the whole browser session: every click, every network call, a screenshot before and after each step. When CI fails at 1 a.m., you open the trace instead of guessing. `retain-on-failure` keeps it only when it is needed.

5 Level 3 — one browser test, and only one

e2e/build-and-run.spec.ts

```
import { test, expect } from '@playwright/test';

test('draw two nodes, connect them, run, see a result', async ({ page, request }) => {
  // start from a known state: make the workflow through the API, not by clicking
  const created = await request.post('/rest/workflows', {
    data: { name: `e2e ${Date.now()}`, nodes: [], connections: {} } });
  const { id } = await created.json();

  await page.goto(`/workflow/${id}`);
  await expect(page.getByTestId('canvas')).toBeVisible();

  await page.getByRole('button', { name: 'Add node' }).click();
  await page.getByText('Manual Trigger').click();
  await page.getByRole('button', { name: 'Add node' }).click();
  await page.getByText('Edit Fields').click();

  await page.getByTestId('handle-Manual Trigger-0')
    .dragTo(page.getByTestId('target-Edit Fields'));

  await page.getByRole('button', { name: 'Save' }).click();
  await expect(page.getByTestId('save-status')).toHaveText('Saved');

  await page.getByRole('button', { name: 'Run' }).click();
  await expect(page.getByTestId('output-Edit Fields')).toContainText('json');

  // it really persisted – reload and check again, which is the bug this test exists to catch
  await page.reload();
  await expect(page.getByTestId('node-Edit Fields')).toBeVisible();
});
```

6

```
.github/workflows/ci.yml
```

```
name: ci
on: [push, pull_request]

# a second push cancels the first run instead of queueing behind it
concurrency: { group: "ci-${{ github.ref }}", cancel-in-progress: true }

jobs:
  check:
    runs-on: ubuntu-latest
    timeout-minutes: 15
    env:
      DATABASE_URL: postgres://postgres:dev@localhost:5432/automation
      BASE_URL: http://localhost:5678
      ENCRYPTION_KEY: "00000000000000000000000000000000000000000000000000000000000000ff"
    services:
      postgres:
        image: postgres:16
        env: { POSTGRES_PASSWORD: dev, POSTGRES_DB: automation }
        ports: ["5432:5432"]
        options: --health-cmd pg_isready --health-interval 5s --health-retries 10
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: npm }
      - run: npm ci
      - run: npm run lint
      - run: psql $DATABASE_URL -f schema.sql
      - run: npm test
      - name: Look for leaked keys
        uses: gitleaks/gitleaks-action@v2
      - run: npx playwright install --with-deps chromium
      - run: npm run e2e
      - uses: actions/upload-artifact@v4
        if: failure() # only when something broke, and it self-deletes
        with: { name: playwright-report, path: playwright-report/, retention-days: 7 }
```

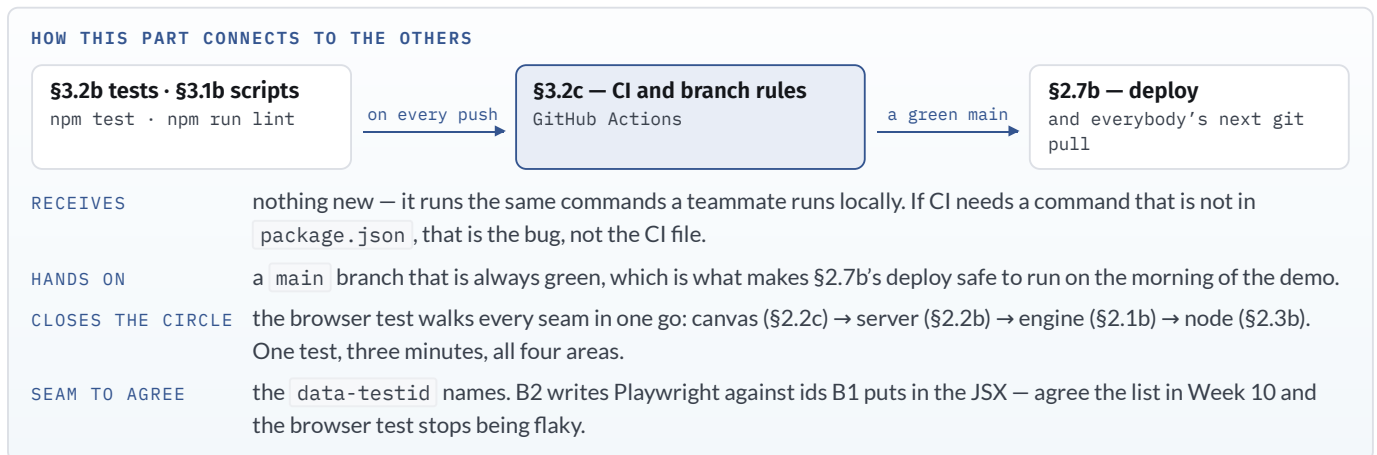
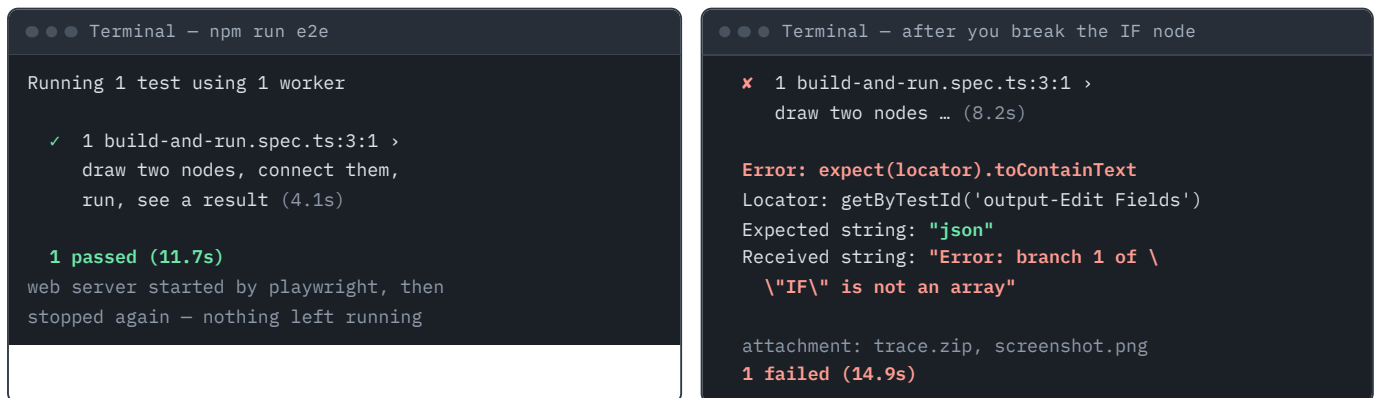
7

- 1 On GitHub open the repository → [Settings](#) → [Branches](#)
- 2 Add branch ruleset → target `main`
- 3 Tick `Require a pull request before merging` → approvals 1
- 4 Tick `Require status checks to pass` → search `check` → select it
- 5 Create `.` From now on nobody — including you — can push red code to `main`.

WHAT YOU SHOULD SEE



Figure 30. The green state every pull request must reach before it can be merged. This screenshot, in the report, is the evidence for the "process" marks.



CHECK IT WORKS

Break the IF node on purpose, push, and a red cross appears within three minutes — without anybody noticing by hand. Open the failed run, download the `playwright-report` artefact, and watch the trace replay the exact click that failed. Then confirm the **Merge** button is greyed out until it is green again. That is the whole point of levels 1–3.

3.3 Proving the whole thing works, on the day

WHO BUILDS THIS PART

D2 directs

A1 · D1 present

WHAT YOU NEED TO KNOW

Nothing technical — a printed checklist, a second laptop, a phone hotspot, and somebody willing to say “stop, do it again”.

The last two weeks are not for building. They are for making sure that what we built works **in front of somebody else, on a day we do not control**.

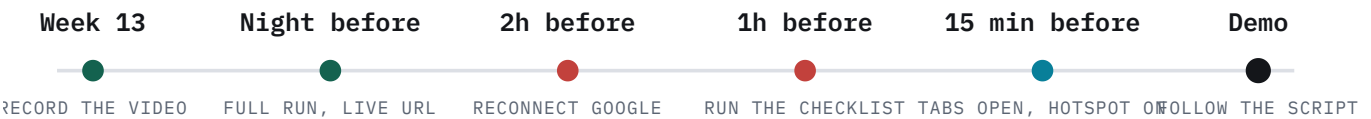


Figure 31. The two red markers are the ones that have ended other groups' demos. Neither takes more than fifteen minutes.

THE ACCEPTANCE CHECKLIST — EVERYTHING THE BRIEF ASKS FOR, AND HOW WE PROVE IT

WHAT IS REQUIRED	WHAT WE SHOW
Schedule trigger	A workflow set to every minute; the run list fills up while we talk, and stops when we switch it off.
Webhook	Somebody in the room sends a request from their phone; the run appears on screen immediately.
HTTP request	The demo workflow calls a real outside API and shows what came back.
Google Sheets	A row appears in a spreadsheet that is open on screen.
Google Docs	A template document comes back with its placeholders filled in from that row.
Google Drive	That finished document appears in a Drive folder.
Gmail	A real email arrives in somebody's inbox, with the document attached.
Code node	A snippet turns one item into ten; then an endless loop is stopped after five seconds and the server stays up.
AI node	The model classifies an incoming message and the IF node routes it to the right branch.
Extensibility	A brand-new node is written live in about ninety seconds and appears with a full settings panel.
Comparison with n8n	One slide: our numbers beside n8n's, the raw data available, and an honest sentence about why we are faster.

DONE WHEN

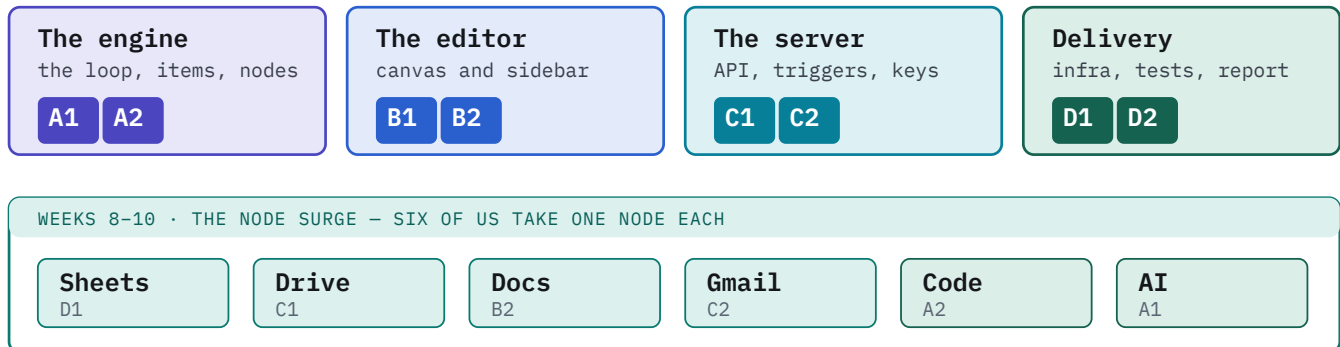
We have run this entire checklist twice, on the deployed address, on two different machines — and the fallback video exists.

THE LAST THING TO SAY

Every row above maps to a sentence in the brief. If we can tick all eleven, the demo is finished — everything else is polish, and polish is not what is being marked.

4.1 Eight people, eight areas

We own **parts of the system**, not tasks — so nobody has to ask whose decision something is, and we rarely edit the same file at the same time. Eight people means **four areas of two**: within a pair you swap work freely and review each other; across pairs you talk through the written agreements.



B1 builds the code-editing box and the run viewer in the same weeks;
D2 writes the tests and the benchmark harness for all six.

Figure 32. Areas are owned all semester. The nodes in Weeks 8–10 are a *surge* — six of us take one each, which is only possible because Phase 3 made nodes independent of the editor.

	OWNS	RESPONSIBLE FOR, ALL SEMESTER
A1	packages/engine	Engine core and technical lead. The loop, the item model, the expression resolver. Writes Phase 1 alone. Holds the casting vote on architecture. Takes the AI node in the surge.
A2	packages/nodes/core	Control flow and isolation. IF, Set, Merge, per-node retry and error handling; later the Code node's sandbox. Reviews A1's work and vice versa.
B1	apps/editor/canvas	The canvas. React Flow, node rendering, and the conversion between its format and ours — the fiddliest code in the editor. Later the code-editing box.
B2	apps/editor/panel	The sidebar and run viewer. The settings-panel generator — the single highest-leverage piece in the project — plus the execution list and log. Takes Docs in the surge.
C1	apps/api	Server and database. The five addresses, the schema and its migrations, and the query layer. Takes Drive in the surge.
C2	apps/api/triggers	Triggers and credentials. Webhook ingress, the schedule registry, encryption, and the Google permission flow. Takes Gmail in the surge — they already know that auth path.
D1	packages/nodes/google	Integrations lead. The shared Google request helper and the node-writing conventions everyone else copies. Takes Sheets first, so the pattern exists before the surge starts.
D2	infra/ · docs/	Infrastructure, testing and the report. Docker, automatic checks, deployment, the benchmark. Editor-in-chief of the report and director of the demo. Starts on day one, not in Week 12.

THREE WRITTEN AGREEMENTS HOLD EIGHT PEOPLE TOGETHER

The **node shape** (frozen Phase 1, A1 owns it), the **five addresses** (frozen Phase 2, C1 owns them), and the **settings-description format** (frozen Phase 3, B2 owns it). Everything else can change without a meeting. These three cannot change without one.

4.2 The hard part of being eight, and how we solve it

Eight people is an advantage in Weeks 8 to 10 and a **liability in Weeks 4 to 6**. Phase 1 is genuinely one person's work and Phase 2 is about three. If we do not plan for that, five of us spend three weeks either idle or — much worse — rewriting each other's code.

WEEKS 4-6 · EIGHT INDEPENDENT TRACKS THAT CONVERGE IN WEEK 6

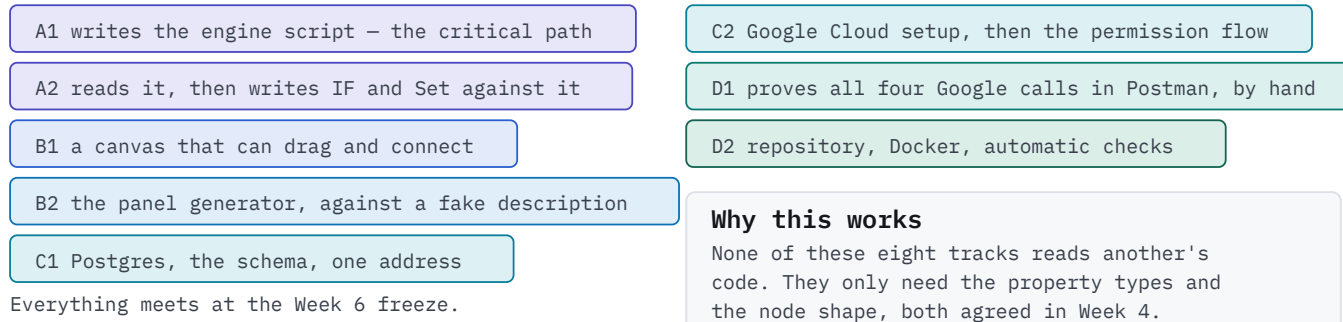


Figure 33. Eight tracks, no shared code. The only Week 4 decisions everyone needs are the node shape and the list of setting types — both are one meeting.

THE TRICK FOR B2

Build against a fake

The settings generator does not need real nodes — it needs a *description*. Write three fake ones by hand in Week 4 and build the whole panel against them. When real nodes arrive in Week 6 they simply work.

THE TRICK FOR D1

Prove the API calls by hand first

Every Google call can be made in Postman before a single node exists. Doing all four in Week 4 — and saving the exact requests — turns Weeks 8–10 from research into typing, and finds the permission problems while there is still time.

HOW EIGHT PEOPLE WORK WITHOUT GETTING IN EACH OTHER'S WAY

- 1. Directory ownership is real.** A change inside someone's directory is reviewed by them. One approval plus green checks merges it; nobody waits for all eight.
- 2. Pairs review each other.** A1↔A2, B1↔B2, C1↔C2, D1↔D2. Reviews stay fast because the reviewer already knows the code.
- 3. Two short meetings, not one long one.** Monday, twenty minutes, in two groups — engine and editor, then server and delivery. Thursday, forty minutes, all eight, and everyone shows something running.
- 4. Never eight people on one problem.** If a thing is stuck, at most two people work on it and the rest keep moving. Eight people debugging one webhook is a wasted afternoon for six.
- 5. Write the contribution statement weekly,** from the commit history, while it is still accurate. With eight names it is a marked component and it is impossible to reconstruct in Week 15.

WHAT EIGHT BUYS US

Capacity in Weeks 8–10, and therefore scope. With five people the AI Agent node, the Merge node, per-node retry and the execution queue were all “only if we are ahead”. With eight they are **expected** — and they are the four things most likely to make our comparison with n8n interesting rather than apologetic.

4.3 Three risks that end projects, and six habits that save them

Google keys expire after seven days

While our app is in Testing mode the long-lived key stops working after a week. Connected in Week 9, dead by Week 14 — and it looks like a broken node, not an expired key.

C2 owns this: reconnect every Google account on the morning of the demo. Step one of the checklist, not a footnote.

Gmail permissions are restricted

Making the app public needs a Google security review we cannot complete this semester.

C2 owns this: stay in Testing mode and add all eight of us — plus our demonstrator — as test users in Week 4, not Week 8.

Eight people, five of them idle

The failure specific to a large group: too few tracks early, so people either wait or start rewriting the core.

A1 owns this: the eight tracks on the previous page are assigned in Week 4, before anyone asks what to do.

Six habits that will save us weeks

- 01 Build the boring version first.** An HTTP node that only does the simplest request is worth more in Week 6 than a perfect one in Week 9, because seven other people can then start.
- 02 One person owns the engine.** Two people editing it at once produces a mess nobody can untangle — which is exactly why A2 reviews it rather than writing it.
- 03 We are not building user accounts.** Logins and password resets earn no marks here and will eat a week of somebody's time.
- 04 When a node will not work, test the plain request in Postman first.** Most “my node is broken” turns out to be a missing permission or a wrong header — nothing to do with our code.
- 05 Print the data between nodes, constantly.** Almost every confusing bug is the shape of the data, not the logic — and section 1.4 explains the shape it should have.
- 06 Copy n8n's good ideas deliberately, and say so in the report.** Explaining *why* a design is right is exactly what the assignment asks for. Copying without understanding is what costs marks.

IF WE FALL BEHIND

We cut *nodes*, never phases. Losing the Drive node costs one row in the acceptance checklist. Losing Phase 3 means every later node needs a hand-built settings panel and the six-person surge becomes a queue behind one person. The order of what to drop, if it comes to it: the Merge node, then Drive, then Docs, then the second half of the benchmark. Never the AI node, and never a phase.

4.4 What each of us does this week

Eight tasks, one per person, none of which needs anybody else's code to exist first. Phase 1 cannot properly start until the first two are done.

	TASK	BLOCKS
A1	Write the engine script from Figure 5 and walk all eight of us through it. Then write the node shape down and freeze it.	everything
A2	Read A1's script line by line, then write the IF and Set nodes against the agreed shape. Nothing that calls an outside service.	Phase 3
B1	A React Flow canvas that can add a box and connect two boxes. Nothing needs to save yet.	Phase 2
B2	Agree the list of setting types with A1, then build the panel generator against three hand-written fake descriptions .	Phase 3
C1	Postgres in a container, the two tables, and a server that answers one address.	Phase 2
C2	Create the Google Cloud project , switch on the four APIs, fill in the consent screen, add all eight of us and the demonstrator as test users.	Phase 5
D1	Prove all four Google calls by hand in Postman — append a row, fill a template, upload a file, send a message — and save the exact requests in the repository.	Phase 5
D2	Repository set up, the shared Docker file, automatic checks on every change, an AWS account with a spending alarm, and the report skeleton.	Phases 2 and 6
All	Read Parts 0 and 1 before Monday, and put the Week 13 rehearsal in the calendar now.	—

WHY C2'S AND D1'S TASKS ARE ON THIS LIST

Both look like Week 8 work. They are here because Google's permission screen has waiting periods we do not control, restricted permissions we cannot argue with, and a test-user list that must include our demonstrator — and because proving the four API calls by hand now turns three weeks of research into three weeks of typing.

OUR WEEKLY RHYTHM FROM HERE

Monday, twenty minutes, in two groups: engine and editor, then server and delivery. Thursday, forty minutes, all eight — everyone shows what actually runs, and one person clones the repository fresh and starts it. Nothing else.

THE ONE SENTENCE TO REMEMBER

Every phase ends in something we can run and show. If a phase takes twice as long as this document says, that is normal — we cut a node, never a phase, and we never start a new phase before the previous one's "done when" is actually true.